

**1. 面向对象的程序设计思想是什么？**

答：把数据结构和对数据结构进行操作的方法封装形成一个个的对象。

**2. 什么是类？**

答：把一些具有共性的对象归类后形成一个集合，也就是所谓的类。

**3. 对象都具有的两方面特征是什么？分别是什么含义？**

答：对象都具有的特征是：静态特征和动态特征。

静态特征是指能描述对象的一些属性（成员变量），动态特征是指对象表现出来的行为（成员函数）

**4. 在头文件中进行类的声明，在对应的实现文件中进行类的定义有什么意义？**

答：这样可以提高编译效率，因为分开的话只需要编译一次生成对应的.obj 文件后，再次应用该类的地方，这个类就不会被再次编译，从而大大的提高了编译效率。

**5. 在类的内部定义成员函数的函数体，这种函数会具备那种属性？**

答：这种函数会自动为内联函数，这种函数在函数调用的地方在编译阶段都会进行代码替换。

**6. 成员函数通过什么来区分不同对象的成员数据？为什么它能够区分？**

答：通过 this 指针指向对象的首地址来区分的。

**7. C++编译器自动为类产生的四个缺省函数是什么？**

答：默认构造函数，拷贝构造函数，析构函数，赋值函数。

**8. 拷贝构造函数在哪几种情况下会被调用？**

答：

- 1.当类的一个对象去初始化该类的另一个对象时；
- 2.如果函数的形参是类的对象，调用函数进行形参和实参结合时；
- 3.如果函数的返回值是类对象，函数调用完成返回时。

**9. 构造函数与普通函数相比在形式上有什么不同？（构造函数的作用，它的声明形式来分析）**

答：构造函数是类的一种特殊成员函数，一般情况下，它是专门用来初始化对象成员变量的。构造函数的名字必须与类名相同，它不具有任何类型，不返回任何值。

**10. 什么时候必须重写拷贝构造函数？**

答：当构造函数涉及到动态存储分配空间时，要自己写拷贝构造函数，并且要深拷贝。

**11. 构造函数的调用顺序是什么？**

答：1.先调用基类构造函数  
2.按声明顺序初始化数据成员  
3.最后调用自己的构造函数。

**12. 哪几种情况必须用到初始化成员列表？**

答：类的成员是常量成员初始化；  
类的成员是对象成员初始化，而该对象没有无参构造函数。  
类的成员为引用时。

### 13. 什么是常对象？

答：常对象是指在任何场合都不能对其成员的值进行修改的对象。

### 14. 静态函数存在的意义？

答：静态私有成员在类外不能被访问，可通过类的静态成员函数来访问；  
当类的构造函数是私有的时候，不像普通类那样实例化自己，只能通过静态成员函数来调用构造函数。

### 15. 在类外有什么办法可以访问类的非公有成员？

答：友元，继承，公有成员函数。

### 16. 什么叫抽象类？

答：不用来定义对象而只作为一种基本类型用作继承的类。

### 17. 运算符重载的意义？

答：为了对用户自定义数据类型的数据的操作与内定义的数据类型的数据的操作形式一致。

### 18. 不允许重载的 5 个运算符是哪些？

答：

1. .\*（成员指针访问运算符）
2. :: 域运算符
3. sizeof 长度运算符
4. ?: 条件运算符
5. .（成员访问符）

### 19. 运算符重载的三种方式？

答：普通函数，友元函数，类成员函数。

### 20. 流运算符为什么不能通过类的成员函数重载？一般怎么解决？

答：因为通过类的成员函数重载必须是运算符的第一个是自己，而对流运算的重载要求第一个参数是流对象。所以一般通过友元来解决。

### 21. 赋值运算符和拷贝构造函数的区别与联系？

答：相同点：都是将一个对象 copy 到另一个中去。  
不同点：拷贝构造函数涉及到要新建立一个对象。

### 22. 在哪种情况下要调用该类的析构函数？

答：对象生命周期结束时。

### 23. 对象间是怎样实现数据的共享的？

答: 通过类的静态成员变量来实现对象间的数据共享。静态成员变量占有自己独立的空间不为某个对象所私有。

#### 24. 友元关系有什么特性?

答: 单向的, 非传递的, 不能继承的。

#### 25. 对对象成员进行初始化的次序是什么?

答: 它的次序完全不受它们在初始化表中次序的影响, 只有成员对象在类中声明的次序来决定的。

#### 26. 类和对象之间的关系是什么?

答: 类是对象的抽象, 对象是类的实例。

#### 27. 对类的成员的访问属性有什么?

答: public, protected, private。

#### 28. const char \*p 和 char \* const p; 的区别

答:

如果 const 位于星号的左侧, 则 const 就是用来修饰指针所指向的变量, 即指针指向为常量; 如果 const 位于星号的右侧, const 就是修饰指针本身, 即指针本身是常量。

#### 29. 是不是一个父类写了一个 virtual 函数, 如果子类覆盖它的函数不加 virtual ,也能实现多态?

答:

virtual 修饰符会被隐形继承的。

virtual 可加可不加, 子类覆盖它的函数不加 virtual ,也能实现多态。

#### 30. 函数重载是什么意思? 它与虚函数的概念有什么区别?

答: 函数重载是一个同名函数完成不同的功能, 编译系统在编译阶段通过函数参数个数、参数类型不同, 函数的返回值来区分该调用哪一个函数, 即实现的是静态的多态性。但是记住: 不能仅仅通过函数返回值不同来实现函数重载。而虚函数实现的是在基类中通过使用关键字 virtual 来申明一个函数为虚函数, 含义就是该函数的功能可能在将来的派生类中定义或者在基类的基础之上进行扩展, 系统只能在运行阶段才能动态决定该调用哪一个函数, 所以实现的是动态的多态性。它体现的是一个纵向的概念, 也即在基类和派生类间实现。

#### 31. 构造函数和析构函数是否可以被重载,为什么?

答: 构造函数可以被重载, 析构函数不可以被重载。因为构造函数可以有多个且可以带参数, 而析构函数只能有一个, 且不能带参数。

#### 32. 如何定义和实现一个类的成员函数为回调函数?

答:

所谓的回调函数, 就是预先在系统的对函数进行注册, 让系统知道这个函数的存在, 以后, 当某个事件发生时, 再调用这个函数对事件进行响应。

定义一个类的成员函数时在该函数前加 CALLBACK 即将其定义为回调函数, 函数的实现和

普通成员函数没有区别

**33. 虚函数是怎么实现的？**

答：简单说来使用了虚函数表。

**34. 抽象类不会产生实例，所以不需要有构造函数。 错**

**35. 从一个模板类可以派生新的模板类，也可以派生非模板类。 对**

**36. main 函数执行以前，还会执行什么代码？**

答案：全局对象的构造函数会在 main 函数之前执行。

**37. 当一个类 A 中没有生命任何成员变量与成员函数,这时 sizeof(A)的值是多少，如果不是零，请解释一下编译器为什么没有让它为零。（Autodesk）**

答案：肯定不是零。举个反例，如果是零的话，声明一个 class A[10]对象数组，而每一个对象占用的空间是零，这时就没办法区分 A[0],A[1]...了。

**38. delete 与 delete []区别：**

答：delete 只会调用一次析构函数，而 delete[]会调用每一个成员的析构函数。

**39. 子类析构时要调用父类的析构函数吗？**

答：会调用。析构函数调用的次序是先派生类的析构后基类的析构，也就是说在基类的析构调用的时候,派生类的信息已经全部销毁了

**40. 继承的优缺点。**

1、类继承是在编译时刻静态定义的，且可直接使用，

2、类继承可以较方便地改变父类的实现。

缺点：

1、因为继承在编译时刻就定义了，所以无法在运行时刻改变从父类继承的实现

2、父类通常至少定义了子类的部分行为，父类的任何改变都可能影响子类的行为

3、如果继承下来的实现不适合解决新的问题，则父类必须重写或被其他更适合的类替换。

这种依赖关系限制了灵活性并最终限制了复用性。

**41. 解释堆和栈的区别。**

答：栈区（stack）— 由编译器自动分配释放，存放函数的参数值，局部变量的值等。

堆（heap）一般由程序员分配释放，若程序员不释放，程序结束时可能由 OS 回收。

**42. 一个类的构造函数和析构函数什么时候被调用,是否需要手工调用？**

答：构造函数在创建类对象的时候被自动调用，析构函数在类对象生命期结束时，由系统自动调用。

**43. 何时需要预编译：**

答：总是使用不经常改动的大型代码体。

程序由多个模块组成，所有模块都使用一组标准的包含文件和相同的编译选项。在这种情况下

下，可以将所有包含文件预编译为一个预编译头。

#### 44. 多态的作用？

答：主要是两个：

1. 隐藏实现细节，使得代码能够模块化；扩展代码模块，实现代码重用；
2. 接口重用：为了类在继承和派生的时候，保证使用家族中任一类的实例的某一属性时的正确调用

#### 45. 虚拟函数与普通成员函数的区别？内联函数和构造函数能否为虚拟函数？

答案：区别：虚拟函数有 `virtual` 关键字，有虚拟指针和虚函数表，虚拟指针就是虚拟函数的接口，而普通成员函数没有。内联函数和构造函数不能为虚拟函数。

#### 46. 构造函数和析构函数的调用顺序？析构函数为什么要虚拟？

答案：构造函数的调用顺序：基类构造函数—对象成员构造函数—派生类构造函数；析构函数的调用顺序与构造函数相反。析构函数虚拟是为了防止析构不彻底，造成内存的泄漏。

#### 47. C++中类型为 `private` 的成员变量可以由哪些函数访问？

答：只可以由本类中的成员函数和友元函数访问

#### 48. 请说出类中 `private`，`protect`，`public` 三种访问限制类型的区别

答：`private` 是私有类型，只有本类中的成员函数访问；`protect` 是保护型的，本类和继承类可以访问；`public` 是公有类型，任何类都可以访问。

#### 49. 类中成员变量怎么进行初始化？

答：可以通过构造函数的初始化列表或构造函数的函数体实现。

#### 50. 在什么时候需要使用“常引用”？

答：如果既要利用引用提高程序的效率，又要保护传递给函数的数据不在函数中被改变，就应使用常引用。

#### 51. 引用与指针有什么区别？

答 、1) 引用必须被初始化，指针不必。

2) 引用初始化以后不能被改变，指针可以改变所指的对象。

3) 不存在指向空值的引用，但是存在指向空值的指针。

#### 52. 描述实时系统的基本特性

答 、在特定时间内完成特定的任务，实时性与可靠性。

#### 54. 全局变量和局部变量在内存中是否有区别？如果有，是什么区别？

答 、全局变量储存在静态数据区，局部变量在堆栈中。

#### 55. 堆栈溢出一般是由什么原因导致的？

答 、没有回收垃圾资源

**56. 什么函数不能声明为虚函数？**

答 构造函数（constructor）

**57. IP 地址的编码分为哪俩部分？**

答 IP 地址由两部分组成，网络号和主机号。

**58. 不能做 switch() 的参数类型是：**

答 、switch 的参数不能为实型。

**59. 如何引用一个已经定义过的全局变量？**

答 、可以用引用头文件的方式，也可以用 extern 关键字，如果用引用头文件方式来引用某个在头文件中声明的全局变理，假定你将那个变写错了，那么在编译期间会报错，如果你用 extern 方式引用时，假定你犯了同样的错误，那么在编译期间不会报错，而在连接期间报错

**60. 对于一个频繁使用的短小函数,在 C 语言中应用什么实现,在 C++中应用什么实现？**

答 、c 用宏定义，c++用 inline

**61. C++是不是类型安全的？**

答案：不是。两个不同类型的指针之间可以强制转换（用 reinterpret cast）

**62. 当一个类 A 中没有生命任何成员变量与成员函数,这时 sizeof(A)的值是多少，请解释一下编译器为什么没有让它为零。**

答案：为 1。举个反例，如果是零的话，声明一个 class A[10]对象数组，而每一个对象占用的空间是零，这时就没办法区分 A[0],A[1]...了。

**63. 简述数组与指针的区别？**

答：数组要么在静态存储区被创建（如全局数组），要么在栈上被创建。指针可以随时指向任意类型的内存块。

(1)修改内容上的区别

```
char a[] = "hello";
```

```
a[0] = 'X';
```

```
char *p = "world"; // 注意 p 指向常量字符串
```

```
p[0] = 'X'; // 编译器不能发现该错误，运行时错误
```

(2) 用运算符 sizeof 可以计算出数组的容量（字节数）。sizeof(p),p 为指针得到的是一个指针变量的字节数，而不是 p 所指的内存容量。

**64. C++函数中值的传递方式**

答：有三种方式：值传递、指针传递、引用传递

**65. 内存的分配方式**

答：分配方式有三种，

1、静态存储区，是在程序编译时就已经分配好的，在整个运行期间都存在，如全局变量、常量。

2、 栈上分配，函数内的局部变量就是从这分配的，但分配的内存容易有限。

3、 堆上分配，也称动态分配，如我们用 `new,malloc` 分配内存，用 `delete,free` 来释放的内存。

#### 66. `extern“C”`有什么作用？

答：Extern “C”是由 C++ 提供的一个连接交换指定符号，用于告诉 C++ 这段代码是 C 函数。这是因为 C++ 编译后库中函数名会变得很长，与 C 生成的不一致，造成 C++ 不能直接调用 C 函数，加上 `extren “c”` 后，C++ 就能直接调用 C 函数了。

Extern “C”主要使用正规 DLL 函数的引用和导出 和 在 C++ 包含 C 函数或 C 头文件时使用。使用时在前面加上 `extern “c”` 关键字即可。可以用一句话概括 `extern “C”` 这个声明的真实目的：实现 C++ 与 C 及其它语言的混合编程。

#### 67. 用什么函数开启新进程、线程。

答案：

线程：CreateThread/AfxBeginThread 等

进程：CreateProcess 等

#### 68. SendMessage 和 PostMessage 有什么区别

答案：SendMessage 是阻塞的，等消息被处理后，代码才能走到 SendMessage 的下一行。PostMessage 是非阻塞的，不管消息是否已被处理，代码马上走到 PostMessage 的下一行。

#### 69. CMemoryState 主要功能是什么

答案：查看内存使用情况，解决内存泄露问题。

#### 70. `#include <filename.h>` 和 `#include “filename.h”` 有什么区别？

答：对于 `#include <filename.h>`，编译器从标准库路径开始搜索 `filename.h`  
对于 `#include “filename.h”`，编译器从用户的工作路径开始搜索 `filename.h`

#### 71. 处理器标识 `#error` 的目的是什么？

答：编译时输出一条错误信息，并中止继续编译。

#### 72. `#ifndef(AFX_..._HADE_H)`

`#define(AFX_..._HADE_H)`

.....

`#endif` 作用？

答：防止该头文件被重复引用。

#### 73. 在定义一个宏的时候要注意什么？

答：定义部分的每个形参和整个表达式都必须用括号括起来，以避免不可预料的错误发生

#### 74. 数组在做函数实参的时候会转变为什么类型？

答：数组在做实参时会变成指针类型。

75. 系统会自动打开和关闭的 3 个标准的文件是？

- (1) 标准输入----键盘---stdin
- (2) 标准输出----显示器---stdout
- (3) 标准出错输出----显示器---stderr

76. 在 Win32 下 char, int, float, double 各占多少位？

- (1) Char 占用 8 位
- (2) Int 占用 32 位
- (3) Float 占用 32 位
- (4) Double 占用 64 位

77. strcpy()和 memcpy()的区别？

答：strcpy()和 memcpy()都可以用来拷贝字符串，strcpy()拷贝以'\0'结束，但 memcpy()必须指定拷贝的长度。

78. 说明 define 和 const 在语法和含义上有什么不同？

答：(1) #define 是 C 语法中定义符号变量的方法，符号常量只是用来表达一个值，在编译阶段符号就被值替换了，它没有类型；

(2) Const 是 C++语法中定义常变量的方法，常变量具有变量特性，它具有类型，内存中存在以它命名的存储单元，可以用 sizeof 测出长度。

79. 说出字符常量和字符串常量的区别，并使用运算符 sizeof 计算有什么不用？

答：字符常量是指单个字符，字符串常量以'\0'结束，使用运算符 sizeof 计算多占一字节的存储空间。

80. 简述全局变量的优缺点？

答：全局变量也称为外部变量，它是在函数外部定义的变量，它属于一个源程序文件，它保存上一次被修改后的值，便于数据共享，但不方便管理，易引起意想不到的错误。

81. 总结 static 的应用和作用？

答：(1) 函数体内 static 变量的作用范围为该函数体，不同于 auto 变量，该变量的内存只被分配一次，因此其值在下次调用时仍维持上次的值；

(2) 在模块内的 static 全局变量可以被模块内所用函数访问，但不能被模块外其它函数访问；

(3) 在模块内的 static 函数只可被这一模块内的其它函数调用，这个函数的使用范围被限制在声明它的模块内；

(4) 在类中的 static 成员变量属于整个类所拥有，对类的所有对象只有一份拷贝；

(5) 在类中的 static 成员函数属于整个类所拥有，这个函数不接收 this 指针，因而只能访问类的 static 成员变量。

82. 总结 const 的应用和作用？

答：(1) 欲阻止一个变量被改变，可以使用 const 关键字。在定义该 const 变量时，通常需要对它进行初始化，因为以后就没有机会再去改变它了；

(2) 对指针来说，可以指定指针本身为 const，也可以指定指针所指的数据为 const，或二

者同时指定为 `const`;

(3) 在一个函数声明中, `const` 可以修饰形参, 表明它是一个输入参数, 在函数内部不能改变其值;

(4) 对于类的成员函数, 若指定其为 `const` 类型, 则表明其是一个常函数, 不能修改类的成员变量;

(5) 对于类的成员函数, 有时候必须指定其返回值为 `const` 类型, 以使得其返回值不为“左值”。

### 83. 什么是指针? 谈谈你对指针的理解?

答: 指针是一个变量, 该变量专门存放内存地址;

指针变量的类型取决于其指向的数据类型, 在所指数据类型前加\*

指针变量的特点是它可以访问所指向的内存。

### 84. 什么是常指针, 什么是指向常变量的指针?

答: 常指针的含义是该指针所指向的地址不能变, 但该地址所指向的内容可以变化, 使用常指针可以保证我们的指针不能指向其它的变量,

指向常变量的指针是指该指针的变量本身的地址可以变化, 可以指向其它的变量, 但是它所指的内容不可以被修改。指向长变量的指针定义,

### 85. 函数指针和指针函数的区别?

答: 函数指针是指向一个函数入口的指针;

指针函数是函数的返回值是一个指针类型。

### 87. 简述 Debug 版本和 Release 版本的区别?

答: Debug 版本是调试版本, Release 版本是发布给用户的最终非调试的版本,

### 88. 指针的几种典型应用情况?

答:

`int *p[n];`-----指针数组, 每个元素均为指向整型数据的指针。

`int (*p)[n];`---p 为指向一维数组的指针, 这个一维数组有 n 个整型数据。

`int *p();`-----函数带回指针, 指针指向返回的值。

`int (*p)();`---p 为指向函数的指针。

### 89. static 函数与普通函数有什么区别?

答: static 函数在内存中只有一份, 普通函数在每个被调用中维持一份拷贝

### 90. struct(结构) 和 union(联合)的区别?

答: 1. 结构和联合都是由多个不同的数据类型成员组成, 但在任何同一时刻, 联合中只存放了一个被选中的成员(所有成员共用一块地址空间), 而结构的所有成员都存在(不同成员的存放地址不同)。

2. 对于联合的不同成员赋值, 将会对其它成员重写, 原来成员的值就不存在了, 而对于结构的不同成员赋值是互不影响的。

### 91. class 和 struct 的区别?

答: **struct** 的成员默认是公有的, 而类的成员默认是私有的。

## 92. 简述枚举类型?

答: 枚举方便一次定义一组常量, 使用起来很方便;

## 93. **assert()**的作用?

答: **ASSERT()**是一个调试程序时经常使用的宏, 在程序运行时它计算括号内的表达式, 如果表达式为 **FALSE (0)**, 程序将报告错误, 并终止执行。如果表达式不为 **0**, 则继续执行后面的语句。这个宏通常用来判断程序中是否出现了明显非法的数据, 如果出现了终止程序以免导致严重后果, 同时也便于查找错误。

## 94. 局部变量和全局变量是否可以同名?

答: 能。局部会屏蔽全局。要用全局变量, 需要使用"**::**"(域运算符)。

95. 程序的局部变量存在于(堆栈)中, 全局变量存在于(静态区)中, 动态申请数据存在于(堆)中。

## 96. 在什么时候使用常引用?

答: 如果既要利用引用提高程序的效率, 又要保护传递给函数的数据不在函数中被改变, 就应使用常引用。

## 97. 类的声明和实现的分开的好处?

- 答: 1. 起保护作用;  
2. 提高编译的效率。

## 98. windows 消息系统由哪几部分构成?

答: 由一下 3 部分组成:

1. 消息队列: 操作系统负责为进程维护一个消息队列, 程序运行时不断从该消息队列中获取消息、处理消息;
2. 消息循环: 应用程序通过消息循环不断获取消息、处理消息。
3. 消息处理: 消息循环负责将消息派发到相关的窗口上使用关联的窗口过程函数进行处理。

## 99. 什么是消息映射?

答: 消息映射就是让程序员指定 **MFC** 类(有消息处理能力的类)处理某个消息。然后由程序员完成对该处理函数的编写, 以实现消息处理功能。

## 100. 什么是 UDP 和 TCP 的区别是什么?

答: **TCP** 的全称为传输控制协议。这种协议可以提供面向连接的、可靠的、点到点的通信。**UDP** 全称为用户报文协议, 它可以提供非连接的不可靠的点到多点的通信。用 **TCP** 还是 **UDP**, 那要看你的程序注重哪一个方面? 可靠还是快速?

## 101. winsock 建立连接的主要实现步骤?

答:

服务器端：**socket()**建立套接字，绑定（**bind**）并监听（**listen**），用 **accept**（）等待客户端连接，**accept**（）发现有客户端连接，建立一个新的套接字，自身重新开始等待连接。该新产生的套接字使用 **send()**和 **recv**（）写读数据，直至数据交换完毕，**closesocket()**关闭套接字。

客户端：**socket()**建立套接字，连接（**connect**）服务器，连接上后使用 **send()**和 **recv**（），在套接字上写读数据，直至数据交换完毕，**closesocket()**关闭套接字。

#### 102. 进程间主要的通讯方式？

答：信号量，管道，消息，共享内存

#### 103. 构成 Win32 API 函数的三个动态链接库是什么？

答：内核库，用户界面管理库，图形设备界面库。

#### 104. 创建一个窗口的步骤是？

答：填充一个窗口类结构->注册这个窗口类->然后再创建窗口->显示窗口->更新窗口。

#### 105. 模态对话框和非模态对话框有什么区别？

- 答：1.调用规则不同：前者是用 **DoModal()**调用，后者通过属性和 **ShowWindow()**来显示。  
2. 模态对话框在没有关闭前用户不能进行其他操作，而非模态对话框可以。  
3. 非模态对话框创建时必须编写自己的共有构造函数，还要调用 **Create()**函数。

#### 106. 从 EDIT 框中取出数据给关联的变量，已经把关联的变量的数据显示在 EDIT 框上的函数是什么？

答： **UpdateData(TRUE)**， **Updatedata(FALSE)**。

#### 107. 简单介绍 GDI？

答： GDI 是 Graphics Device Interface 的缩写，译为：图形设备接口；是一个在 Windows 应用程序中执行与设备无关的函数库，这些函数在不同的输出设备上产生图形以及文字输出。

#### 108. windows 消息分为几类？并对各类做简单描述。

答：

- 1.窗口消息：与窗口相关的消息，除 **WM\_COMMAND** 之外的所有以 **WM\_**开头的消息；
- 2.命令消息：用于处理用户请求，以 **WM\_COMMAND** 表示的消息；
- 3.控件通知消息：统一由 **WM\_NOTIFY** 表示，
- 4.用户自定义消息。

#### 109. 如何自定义消息？

答：使用 **WM\_USER** 和 **WM\_APP** 两个宏来自定义消息，

#### 110. 简述 Visual C++ 、 Win32 API 和 MFC 之间的关系？

- 答：(1) Visual C++是一个以 C++程序设计语言为基础的、集成的、可视化的编程环境；  
(2) Win32 API 是 32 位 Windows 操作系以 C/C++形式提供的一组应用程序接口；  
(3) MFC 是对 Win32 API 的封装，简化了开发过程。

### 111.怎样消除多重继承中的二义性?

答: 1. 成员限定符 2. 虚基类

### 112 什么叫静态关联, 什么叫动态关联

答: 在多态中, 如果程序在编译阶段就能确定实际执行动作, 则称静态关联, 如果等到程序运行才能确定叫动态关联。

### 113 多态的两个必要条件

答: 1.一个基类的指针或引用指向一个派生类对象 2.虚函数

### 114.什么叫智能指针?

答: 当一个类中, 存在一个指向另一个类对象的指针时, 对指针运算符进行重载, 那么当前类对象可以通过指针像调用自身成员一样调用另一个类的成员。

### 115.什么时候需要用虚析构函数?

答: 当基类指针指向用 new 运算符生成的派生类对象时, delete 基类指针时, 派生类部分没有释放掉而造成释放不彻底现象, 需要虚析构函数。补充: 虚函数就是让派生类调用基类的虚函数。

### 116. MFC 中, 大部分类是从哪个类继承而来?

答: CObject

### 117.什么是平衡二叉树?

答: 左右子树都是平衡二叉树, 而且左右子树的深度差值的绝对值不大于 1。

### 118.语句 for( ; 1 ; )有什么问题? 它是什么意思?

答: 无限循环, 和 while(1)相同。

### 119. 派生新类的过程要经历三个步骤

答: 1.吸收基类成员 2.改造基类成员 3.添加新成员

### 121. TCP/IP 建立连接的过程

答: 在 TCP/IP 协议中, TCP 协议提供可靠的连接服务, 采用三次握手建立一个连接。

第一次握手: 建立连接时, 客户端发送连接请求到服务器, 并进入 SYN\_SEND 状态, 等待服务器确认;

第二次握手: 服务器收到客户端连接请求, 向客户端发送允许连接应答, 此时服务器进入 SYN\_RECV 状态;

第三次握手: 客户端收到服务器的允许连接应答, 向服务器发送确认, 客户端和服务器进入通信状态, 完成三次握手

### 122. memset ,memcpy 的区别

答: memset 用来对一段内存空间全部设置为某个字符, 一般用在对定义的字符串进行初始化为'\0'。

memcpy 用来做内存拷贝,你可以拿它拷贝任何数据类型的对象,可以指定拷贝的数据长度:

**123. 在 C++ 程序中调用被 C 编译器编译后的函数,为什么要加 extern “C”?**

答: C++语言支持函数重载, C 语言不支持函数重载。函数被 C++编译后在库中的名字与 C 语言的不同。假设某个函数的原型为: void foo(int x, int y);该函数被 C 编译器编译后在库中的名字为\_foo , 而 C++编译器则会产生像\_foo\_int\_int 之类的名字。C++提供了 C 连接交换指定符号 extern“C”来解决名字匹配问题。

**124 怎样定义一个纯虚函数? 含有纯虚函数的类称为什么?**

答: 在虚函数的后面加=0, 含有虚函数的类称为抽象类。

**125. 已知 strcpy 函数的原型是:**

**char \* strcpy(char \* strDest,const char \* strSrc);**不调用库函数, 实现 strcpy 函数。其中, strSrc 是原字符串, strDest 是目标字符串 。

答案:

```
char *strcpy(char *strDest, const char *strSrc)
{
    if ( strDest == NULL || strSrc == NULL)
        return NULL ;
    if ( strDest == strSrc)
        return strDest ;
    char *tempPtr = strDest ; //指针 tempPtr 指向 strDest 的地址;
    while( (*strDest++ = *strSrc++) != '\0') //注意: 别忘了转义符;
        ;
    return tempPtr ; //返回指针向的地址;
}
```

**126. 已知类 String 的原型为:**

```
class String
{
public:
    String(const char *str = NULL); // 普通构造函数
    String(const String &other); // 拷贝构造函数
    ~String(void); // 析构函数
    String & operate =(const String &other); // 赋值函数
private:
    char *m_data; // 用于保存字符串
};
```

请编写 String 的上述 4 个函数。

答案:

// 普通构造函数

String::String(const char \*str)

```
{
    if ( str == NULL ) //strlen 在参数为 NULL 时会抛异常才会有这步判断
```

```

    {
        m_data = new char[1];
        m_data[0] = " ";
    }
    else
    {
        m_data = new char[strlen(str) + 1];
        strcpy(m_data, str);
    }
}
//拷贝构造函数
String::String(const String &other)
{
    m_data = new char[strlen(other.m_data) + 1];
    strcpy(m_data, other.m_data);
}
//赋值函数(重载运算符)
String & String::operator =(const String &other)
{
    if ( this == &other)
        return *this ;
    delete []m_data;
    m_data = new char[strlen(other.m_data) + 1];
    strcpy(m_data, other.m_data);
    return *this ;
}
//析构函数
String::~~ String(void)
{
    delete []m_data ;
}

```

## 127. 类成员函数的重载、覆盖和隐藏的区别

答案：

成员函数被重载的特征：

- (1) 相同的范围（在同一个类中）；
- (2) 函数名字相同；
- (3) 参数不同；
- (4) **virtual** 关键字可有可无。

覆盖是指派生类函数覆盖基类函数，特征是：

- (1) 不同的范围（分别位于派生类与基类）；
- (2) 函数名字相同；
- (3) 参数相同；
- (4) 基类函数必须有 **virtual** 关键字。

“隐藏”是指派生类的函数屏蔽了与其同名的基类函数，规则如下：

（1）如果派生类的函数与基类的函数同名，但是参数不同。此时，不论有无 **virtual** 关键字，基类的函数将被隐藏（注意别与重载混淆）。

（2）如果派生类的函数与基类的函数同名，并且参数也相同，但是基类函数没有 **virtual** 关键字。此时，基类的函数被隐藏（注意别与覆盖混淆）

## 128. 如何打印出当前源文件的文件名以及源文件的当前行号？

答案：

```
cout << __FILE__ ;
```

```
cout<<__LINE__ ;
```

`__FILE__` 和 `__LINE__` 是系统预定义宏，这种宏并不是在某个文件中定义的，而是由编译器定义的。

## 129. 文件中有一组整数，要求排序后输出到另一个文件中

答案：

```
void Order(vector<int> &data) //冒泡排序
```

```
{
    int count = data.size();
    int tag = false ;
    for ( int i = 0 ; i < count ; i++)
    {
        for ( int j = 0 ; j < count - i - 1 ; j++)
        {
            if ( data[j] > data[j+1])
            {
                tag = true ;
                int temp = data[j] ;
                data[j] = data[j+1] ;
                data[j+1] = temp ;
            }
        }
        if ( !tag )
            break ;
    }
}
```

```
void main( void )
```

```
{
    vector<int>data;
    ifstream in("c:\\data.txt");
    if ( !in)
    {
        cout<<"file error!";
        exit(1);
    }
}
```

```

    }
    int temp;
    while (!in.eof())
    {
        in>>temp;
        data.push_back(temp);
    }
    in.close();
    Order(data);
    ofstream out("c:\\result.txt");
    if ( !out)
    {
        cout<<"file error!";
        exit(1);
    }
    for ( i = 0 ; i < data.size() ; i++)
        out<<data[i]<<" ";
    out.close();
}

```

### 130. 一个链表的结点结构

```
struct Node
```

```
{
```

```
int data ;
```

```
Node *next ;
```

```
};
```

```
typedef struct Node Node ;
```

已知链表的头结点 head,写一个函数把这个链表逆序 ( Intel)

答案:

```

Node * ReverseList(Node *head) //链表逆序
{
    if ( head == NULL || head->next == NULL )
        return head;
    Node *p1 = head ;
    Node *p2 = p1->next ;
    Node *p3 = p2->next ;
    p1->next = NULL ;
    while ( p3 != NULL )
    {
        p2->next = p1 ;
        p1 = p2 ;
        p2 = p3 ;
        p3 = p3->next ;
    }
}

```

```

    p2->next = p1 ;
    head = p2 ;
    return head ;
}

```

### 131. 一个链表的结点结构

```

struct Node
{
    int data ;
    Node *next ;
};

```

```

typedef struct Node Node ;

```

已知两个链表 head1 和 head2 各自有序，请把它们合并成一个链表依然有序。

答案：

```

Node * Merge(Node *head1 , Node *head2)
{
    if ( head1 == NULL)
        return head2 ;
    if ( head2 == NULL)
        return head1 ;
    Node *head = NULL ;
    Node *p1 = NULL;
    Node *p2 = NULL;
    if ( head1->data < head2->data )
    {
        head = head1 ;
        p1 = head1->next;
        p2 = head2 ;
    }
    else
    {
        head = head2 ;
        p2 = head2->next ;
        p1 = head1 ;
    }
    Node *pcurrent = head ;
    while ( p1 != NULL && p2 != NULL)
    {
        if ( p1->data <= p2->data )
        {
            pcurrent->next = p1 ;
            pcurrent = p1 ;
            p1 = p1->next ;
        }
    }
}

```

```

else
{
pcurrent->next = p2 ;
pcurrent = p2 ;
p2 = p2->next ;
}
}
if ( p1 != NULL )
pcurrent->next = p1 ;
if ( p2 != NULL )
pcurrent->next = p2 ;
return head ;
}

```

132.已知两个链表 head1 和 head2 各自有序，请把它们合并成一个链表依然有序，这次要求用递归方法进行。 (Autodesk)

答案：

```

Node * MergeRecursive(Node *head1 , Node *head2)
{
if ( head1 == NULL )
return head2 ;
if ( head2 == NULL )
return head1 ;
Node *head = NULL ;
if ( head1->data < head2->data )
{
head = head1 ;
head->next = MergeRecursive(head1->next,head2);
}
else
{
head = head2 ;
head->next = MergeRecursive(head1,head2->next);
}
return head ;
}

```

133. 分析一下这段程序的输出 (Autodesk)

```

class B
{
public:
B()
{
cout<<"default constructor"<<endl;

```

```

}
~B()
{
cout<<"destroyed"<<endl;
}
B(int i):data(i)
{
cout<<"constructed by parameter" << data <<endl;
}
private:
int data;
};
B Play( B b)
{
return b ;
}
int main(int argc, char* argv[])
{
B temp = Play(5);
return 0;
}

```

### 133 将“引用”作为函数参数有哪些特点？

（1）传递引用给函数与传递指针的效果是一样的。这时，被调函数的形参就成为原来主调函数中的实参变量或对象的一个别名来使用，所以在被调函数中对形参变量的操作就是对其相应的目标对象（在主调函数中）的操作。

（2）使用引用传递函数的参数，在内存中并没有产生实参的副本，它是直接对实参操作；而使用一般变量传递函数的参数，当发生函数调用时，需要给形参分配存储单元，形参变量是实参变量的副本；如果传递的是对象，还将调用拷贝构造函数。因此，当参数传递的数据较大时，用引用比用一般变量传递参数的效率和所占空间都好。

（3）使用指针作为函数的参数虽然也能达到与使用引用的效果，但是，在被调函数中同样要给形参分配存储单元，且需要重复使用“\*指针变量名”的形式进行运算，这很容易产生错误且程序的阅读性较差；另一方面，在主调函数的调用点处，必须用变量的地址作为实参。而引用更容易使用，更清晰。

### 134. 什么时候需要“引用”？

流操作符（<<、>>）和赋值操作符（=）的返回值、拷贝构造函数的参数、赋值操作符的参数、其它情况都推荐使用引用。

### 135. 面向对象的三个基本特征，并简单叙述之？

1. 封装：将客观事物抽象成类，每个类对自身的数据和方法实行 `protection(private, protected, public)`

2. 继承：广义的继承有三种实现形式：实现继承（指使用基类的属性和方法而无需额外编码的能力）、可视继承（子窗体使用父窗体的外观和实现代码）、接口继承（仅使用属性和方法，实现滞后到子类实现）。前两种（类继承）和后一种（对象组合=>接口继承以及纯虚函数）构成了功能复用的两种方式。

3. 多态：是将父对象设置成为和一个或更多的他的子对象相等的技术，赋值之后，父对象就可以根据当前赋值给它的子对象的特性以不同的方式运作。简单的说，就是一句话：允许将子类类型的指针赋值给父类类型的指针。

136.求下面函数的返回值（微软）

```
int func(x)
{
    int countx = 0;
    while(x)
    {
        countx ++;
        x = x&(x-1);
    }
    return countx;
}
```

假定 `x = 9999`。 答案： 8

思路：将 `x` 转化为 2 进制，看含有的 1 的个数。

137、写出下列代码的输出内容

```
#include<stdio.h>
int inc(int a)
{
    return(++a);
}
int multi(int*a,int*b,int*c)
{
    return(*c=*a**b);
}
typedef int(FUNC1)(int in);
typedef int(FUNC2) (int*,int*,int*);
void show(FUNC2 fun,int arg1, int*arg2)
{
    INCp=&inc;
```

```

int temp =p(arg1);
fun(&temp,&arg1, arg2);
printf("%d\n",*arg2);
}
main()
{
int a;
show(multi,10,&a);
return 0;
}

```

答: 110

138. 编写一个 C 函数，该函数在一个字符串中找到可能的最长的子字符串，且该字符串是由同一字符组成的。

```

char * search(char *cpSource, char ch)
{
    char *cpTemp=NULL, *cpDest=NULL;
    int iTemp, iCount=0;
    while(*cpSource)
    {
        if(*cpSource == ch)
        {
            iTemp = 0;
            cpTemp = cpSource;
            while(*cpSource == ch)
                ++iTemp, ++cpSource;
            if(iTemp > iCount)
                iCount = iTemp, cpDest = cpTemp;
            if(!*cpSource)
                break;
        }
        ++cpSource;
    }
    return cpDest;
}

```

139. 请编写一个 C 函数，该函数在给定的内存区域搜索给定的字符，并返回该字符所在位置索引值。

```

int search(char *cpSource, int n, char ch)
{
    int i;
    for(i=0; i<n && *(cpSource+i) != ch; ++i);
    return i;
}

```

140. 一个单向链表，不知道头节点,一个指针指向其中的一个节点，问如何删除这个指针指

向的节点？

将这个指针指向的 next 节点值 copy 到本节点，将 next 指向 next->next,并随后删除原 next 指向的节点。

**141、用指针的方法，将字符串“ABCD1234efgh”前后对调显示**

```
#include <stdio.h>
#include <string.h>
#include <dos.h>
int main()
{
    char str[] = "ABCD1234efgh";
    int length = strlen(str);
    char * p1 = str;
    char * p2 = str + length - 1;
    while(p1 < p2)
    {
        char c = *p1;
        *p1 = *p2;
        *p2 = c;
        ++p1;
        --p2;
    }
    printf("str now is %s\n",str);
    system("pause");
    return 0;
}
```

142、有一分数序列：1/2,1/4,1/6,1/8.....，用函数调用的方法，求此数列前 20 项的和

```
#include <stdio.h>
double getValue()
{
    double result = 0;
    int i = 2;
    while(i < 42)
    {
        result += 1.0 / i;//一定要使用 1.0 做除数，不能用 1，否则结果将自动转化成整数，
        即 0.000000
        i += 2;
    }
    return result;
}
int main()
{
    printf("result is %f\n", getValue());
    system("pause");
}
```

```

    return 0;
}

```

143、有一个数组 `a[1000]` 存放 `0--1000`; 要求每隔二个数删掉一个数，到末尾时循环至开头继续进行，求最后一个被删掉的数的原始下标位置。

以 7 个数为例：

`{0,1,2,3,4,5,6,7}` `0-->1-->2`（删除）`-->3-->4-->5`（删除）`-->6-->7-->0`（删除），如此循环直到最后一个数被删除。

方法 1：数组

```

#include <iostream>
using namespace std;
#define null 1000
int main()
{
    int arr[1000];
    for (int i=0;i<1000;++i)
        arr[i]=i;
    int j=0;
    int count=0;
    while(count<999)
    {
        while(arr[j%1000]==null)
            j=(++j)%1000;
        j=(++j)%1000;
        while(arr[j%1000]==null)
            j=(++j)%1000;
        j=(++j)%1000;
        while(arr[j%1000]==null)
            j=(++j)%1000;
        arr[j]=null;
        ++count;
    }
    while(arr[j]==null)
        j=(++j)%1000;
    cout<<j<<endl;
    return 0;
}

```

方法 2：链表

```

#include<iostream>
using namespace std;
#define null 0
struct node
{
    int data;
    node* next;
}

```

```

};
int main()
{
    node* head=new node;
    head->data=0;
    head->next=null;
    node* p=head;
    for(int i=1;i<1000;i++)
    {
        node* tmp=new node;
        tmp->data=i;
        tmp->next=null;
        head->next=tmp;
        head=head->next;
    }
    head->next=p;
    while(p!=p->next)
    {
        p->next->next=p->next->next->next;
        p=p->next->next;
    }
    cout<<p->data;
    return 0;
}

```

方法 3: 通用算法

```

#include <stdio.h>
#define MAXLINE 1000 //元素个数
/*
MAXLINE    元素个数
a[]         元素数组
R[]         指针场
suffix      下标
index       返回最后的下标序号
values      返回最后的下标对应的值
start       从第几个开始
K           间隔
*/
int find_n(int a[],int R[],int K,int& index,int& values,int s=0) {
    int suffix;
    int front_node,current_node;
    suffix=0;
    if(s==0) {
        current_node=0;

```

```

        front_node=MAXLINE-1;
    }
    else {
        current_node=s;
        front_node=s-1;
    }
    while(R[front_node]!=front_node) {
        printf("%d\n",a[current_node]);
        R[front_node]=R[current_node];
        if(K==1) {
            current_node=R[front_node];
            continue;
        }
        for(int i=0;i<K;i++){
            front_node=R[front_node];
        }
        current_node=R[front_node];
    }
    index=front_node;
    values=a[front_node];
    return 0;
}
int main(void) {
    int a[MAXLINE],R[MAXLINE],suffix,index,values,start,i,K;
    suffix=index=values=start=0;
    K=2;
    for(i=0;i<MAXLINE;i++) {
        a[i]=i;
        R[i]=i+1;
    }
    R[i-1]=0;
    find_n(a,R,K,index,values,2);
    printf("the value is %d,%d\n",index,values);
    return 0;
}

```

144、指出下列程序有什么错误：

```

void test2()
{
    char string[10], str1[10];
    int i;
    for(i=0; i<10; i++)
    {
        str1[i] = 'a';
    }
}

```

```

    strcpy( string, str1 );
}

```

解答：对试题 2，如果面试者指出字符数组 **str1** 不能在数组内结束可以给 3 分；如果面试者指出 **strcpy(string, str1)**调用使得从 **str1** 内存起复制到 **string** 内存起所复制的字节数具有不确定性可以给 7 分，在此基础上指出库函数 **strcpy** 工作方式的给 10 分；

**str1** 不能在数组内结束:因为 **str1** 的存储为: {a,a,a,a,a,a,a,a,a,a},没有'\0'(字符串结束符),所以不能结束

**strcpy( char \*s1,char \*s2)**他的工作原理是，扫描 **s2** 指向的内存，逐个字符付到 **s1** 所指向的内存，直到碰到'\0',因为 **str1** 结尾没有'\0'，所以具有不确定性，不知道他后面还会付什么东东。

正确应如下

```

void test2()
{
    char string[10], str1[10];
    int i;
    for(i=0; i<9; i++)
    {
        str1[i] = 'a'+i; //把 abcdefghi 赋值给字符数组
    }
    str[i]='\0';//加上结束符
    strcpy( string, str1 );
}

```

#### 145、实现 strcmp

```

int StrCmp(const char *str1, const char *str2)
{
    assert(str1 && str2);
    while(*str1 && *str1++ == *str2++);
    return *str1-*str2;
}

```

#### 146. 字符串 A 和 B, 输出 A 和 B 中的最大公共子串。

比如 A="aocdfe" B="pmcdfa" 则输出"cdf"

```

*/
//Author: azhen
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
char *commonstring(char shortstring[], char longstring[])
{
    int i, j;
    char *substring=malloc(256);
    if(strstr(longstring, shortstring)!=NULL)                //如果....., 那么返回 shortstring
        return shortstring;
    for(i=strlen(shortstring)-1; i>0; i--)                    //否则, 开始循环计算
    {

```

```

for(j=0; j<=strlen(shortstring)-i; j++){
    memcpy(substring, &shortstring[j], i);
    substring[i]='\0';
    if(strstr(longstring, substring)!=NULL)
        return substring;
}
}
return NULL;
}

```

```

main()
{
    char *str1=malloc(256);
    char *str2=malloc(256);
    char *comman=NULL;
    gets(str1);
    gets(str2);
    if(strlen(str1)>strlen(str2)) //将短的字符串放前面
        comman=commanstring(str2, str1);
    else
        comman=commanstring(str1, str2);
    printf("the longest comman string is: %s\n", comman);
}

```

147、写一个函数比较两个字符串 **str1** 和 **str2** 的大小，若相等返回 0，若 **str1** 大于 **str2** 返回 1，若 **str1** 小于 **str2** 返回 -1

```

int strcmp ( const char * src,const char * dst)
{
    int ret = 0 ;
    while( ! (ret = *(unsigned char *)src - *(unsigned char *)dst) && *dst)
    {
        ++src;
        ++dst;
    }
    if ( ret < 0 )
        ret = -1 ;
    else if ( ret > 0 )
        ret = 1 ;
    return( ret );
}

```

148、判断一个字符串是不是回文

```

int IsReverseStr(char *aStr)
{
    int i,j;
}

```

```

int found=1;
if(aStr==NULL)
return -1;
j=strlen(aStr);
for(i=0;i<j/2;i++)
if(*(aStr+i)!=*(aStr+j-i-1))
{
found=0;
break;
}
return found;

```

```

149 #include main()
{
    int c[3][3]={1,2,3,4,5,6,7,8,9};
    for(int i=0;i<3;i++)
    for(int j=0;j<3;j++)
    printf("%ld\n",&c[j]);
    printf("-----\n");
    printf("%ld\n",(c+1));
    printf("%ld\n",(*c+1));
    printf("%ld\n",&c[0][0]);
    printf("%ld\n",**c);
    printf("%ld\n",*c[0]);
    if(int(c)==int(*c)) printf("equal");
}

```

为什么 `c`, `*c` 的值相等, `(c+1)`, `(*c+1)` 的值不等 `c`, `*c`, `**c`, 代表什么意思?

参考答案:

`c` 是第一个元素的地址, `*c` 是第一行元素的首地址, 其实第一行元素的地址就是第一个元素的地址, 这容易理解。 `**c` 是提领第一个元素。

为什么 `c`, `*c` 的值相等?

`int c` 因为直接用 `c` 表示数组 `c[0][0]` `printf("%ld\n",*c[0]);` 语句已将指针移到数组头。 `int(*c)` 表示 `c0` 的值为 1, 所以相等。数组 `c` 的存放空间示意如下: (机器中是行优先存放的) `c[0][0]` `c[0][1]` `c[0][2]` `c[1][0]` `c[1][1]` `c[1][2]` `c[2][0]` `c[2][1]` `c[2][2]` `c` 是一个二维数组名, 实际上它是一个指针常量, 不能进行自加、自减运算, 即: `c++`、`c--`、`++c`、`--c` 都是不允许的;

`c`: 数组名; 是一个二维指针, 它的值就是数组的首地址, 也即第一行元素的首地址 (等于 `*c`), 也 等于第一行第一个元素的地址 ( `&c[0][0]`); 可以说成是二维数组的行指针。 `*c`: 第一行元素的首地址; 是一个一维指针, 可以说成是二维数组的列指针。 `**c`: 二维数组中的第一个元素的值; 即: `c[0][0]` 所以: `c` 和 `*c` 的值是相等的, 但他们两者不能相互赋值, (类型不同); `(c+1)` : `c` 是行指针, `(c+1)` 是在 `c` 的基础上加上二维数组一行的地址长度, 即从 `&c[0][0]` 变到了 `&c[1][0]`; `(*c+1)`: `*c` 是列指针, `(*c+1)` 是在 `*c` 的基础上加上二维数组一个元素的所占的长度, 即从 `&c[0][0]` 变到了 `&c[0][1]` 从而 `(c+1)` 和 `(*c+1)` 的值就不相等了

150、定义 `int **a[3][4]`, 则变量占有的内存空间为: 32 参考答案:

`int **p; /*16 位下 sizeof(p)=2, 32 位下 sizeof(p)=4*/`

总共 `3*4*sizeof(p)`

151、写出判断 ABCD 四个表达式的是否正确, 若正确, 写出经过表达式中 `a` 的值

`int a = 4;`

(A) `a += (a++);` (B) `a += (++a);` (C) `(a++) += a;` (D) `(++a) += (a++);`

`a = ?`

答: C 错误, 左侧不是一个有效变量, 不能赋值, 可改为 `(++a) += a;`

改后答案依次为 9,10,10,11

152、某 32 位系统下, C++ 程序, 请计算 `sizeof` 的值

`char str[] = "http://www.ibegroup.com/"`

`char *p = str;`

`int n = 10;`

请计算

(1) `sizeof (str) = ?`

(2) `sizeof ( p ) = ?`

(3) `sizeof ( n ) = ?`

`void Foo ( char str[100]){`

请计算

`sizeof( str ) = ? (4)`

`}`

`void *p = malloc( 100 );`

请计算

`sizeof ( p ) = ? (5)`

答: (1) 25 (2) 4 (3) 4 (4) 4 (5) 4

153、 回答下面的问题

(1). `Void GetMemory(char **p, int num){`

`*p = (char *)malloc(num);`

`}`

`void Test(void){`

`char *str = NULL;`

`GetMemory(&str, 100);`

`strcpy(str, "hello");`

`printf(str);`

`}`

请问运行 `Test` 函数会有什么样的结果?

答: 输出“hello”

154、 `void Test(void)`

`{`

```

char *str = (char *) malloc(100);
strcpy(str, "hello");
free(str);
if(str != NULL){
strcpy(str, "world");
printf(str);
}
}

```

请问运行 Test 函数会有什么样的结果？

答：输出“world”

```

155、 char *GetMemory(void){
char p[] = "hello world";
return p;
}
void Test(void){
char *str = NULL;
str = GetMemory();
printf(str);
}

```

请问运行 Test 函数会有什么样的结果？

答：无效的指针，输出不确定

### 156、编写 strcat 函数

已知 **strcat** 函数的原型是 **char \*strcat (char \*strDest, const char \*strSrc);**

其中 **strDest** 是目的字符串，**strSrc** 是源字符串。

(1) 不调用 C++/C 的字符串库函数，请编写函数 **strcat**

答：

VC 源码：

```

char * __cdecl strcat (char * dst, const char * src)
{
char * cp = dst;
while( *cp )
cp++;
while( *cp++ = *src++ );
return( dst );
}

```

157、**strcat** 能把 **strSrc** 的内容连接到 **strDest**，为什么还要 **char \*** 类型的返回值？

答：方便赋值给其他变量

158、MFC 中 **CString** 是类型安全类么？

答：不是，其它数据类型转换到 **CString** 可以使用 **CString** 的成员函数 **Format** 来转换

159.C++中什么数据分配在栈或堆中？

答：栈：存放局部变量，函数调用参数,函数返回值，函数返回地址。由系统管理

堆：程序运行时动态申请，new 和 malloc 申请的内存就在堆上

160、函数模板与类模板有什么区别？

答：函数模板的实例化是由编译程序在处理函数调用时自动完成的，而类模板的实例化必须由程序员在程序中显式地指定。

161、 int i=10, j=10, k=3; k\*=i+j; k 最后的值是？

答：60，此题考察优先级，实际写成： k\*=(i+j);，赋值运算符优先级最低

162、do.....while 和 while.....do 有什么区别？

答 、前一个循环一遍再判断，后一个判断以后再循环

163、请写出下列代码的输出内容

```
#include
main()
{
int a,b,c,d;
a=10;
b=a++;
c=++a;
d=10*a++;
printf("b, c, d: %d, %d, %d", b, c, d);
return 0;
}
```

答 、 10, 12, 120

164.在 c 语言库函数中将一个字符转换成整型的函数是 atol()吗，这个函数的原型是什么？

答 、函数名: atol

功 能: 把字符串转换成长整型数

用 法: long atol(const char \*nptr);

程序例:

```
#include
#include
int main(void)
{
    long l;
```

```

char *str = "98765432";
l = atol(lstr);
printf("string = %s integer = %ld\n", str, l);
return(0);
}

```

165. 以下三条输出语句分别输出什么？

```

char str1[] = "abc";
char str2[] = "abc";
const char str3[] = "abc";
const char str4[] = "abc";
const char* str5 = "abc";
const char* str6 = "abc";
cout << boolalpha << ( str1==str2 ) << endl; // 输出什么？
cout << boolalpha << ( str3==str4 ) << endl; // 输出什么？
cout << boolalpha << ( str5==str6 ) << endl; // 输出什么？

```

答：分别输出 **false**,**false**,**true**。**str1** 和 **str2** 都是字符数组，每个都有其自己的存储区，它们的值则是各存储区首地址，不等；**str3** 和 **str4** 同上，只是按 **const** 语义，它们所指向的数据区不能修改。**str5** 和 **str6** 并非数组而是字符指针，并不分配存储区，其后的“abc”以常量形式存于静态数据区，而它们自己仅是指向该区首地址的指针，相等。

166 以下代码中的两个 **sizeof** 用法有问题吗？

```

void UpperCase( char str[] ) // 将 str 中的小写字母转换成大写字母
{
for( size_t i=0; i<sizeof(str)/sizeof(str[0]); ++i )
if( 'a'<=str[i] && str[i]<='z' )
str[i] -= ('a'-'A' );
}
char str[] = "aBcDe";
cout << "str 字符长度为: " << sizeof(str)/sizeof(str[0]) << endl;
UpperCase( str );
cout << str << endl;

```

答：函数内的 **sizeof** 有问题。根据语法，**sizeof** 如用于数组，只能测出静态数组的大小，无法检测动态分配的或外部数组大小。函数外的 **str** 是一个静态定义的数组，因此其大小为 6，函数内的 **str** 实际只是一个指向字符串的指针，没有任何额外的与数组相关的信息，因此 **sizeof** 作用于上只将其当指针看，一个指针为 4 个字节，因此返回 4。

167 非 C++ 内建型别 **A** 和 **B**，在哪几种情况下 **B** 能隐式转化为 **A**？

答：

- a. `class B : public A { ..... }` // **B** 公有继承自 **A**，可以是间接继承的
- b. `class B { operator A( ); }` // **B** 实现了隐式转化为 **A** 的转化
- c. `class A { A( const B& ); }` // **A** 实现了 **non-explicit** 的参数为 **B**（可以有其他带默认值的参数）构造函数

d. `A& operator= ( const A& );` // 赋值操作，虽不是正宗的隐式类型转换，但也可以勉强算一个

168. 以下代码有什么问题？

```
struct Test
{
    Test( int ) {}
    Test() {}
    void fun() {}
};

void main( void )
{
    Test a(1);
    a.fun();
    Test b();
    b.fun();
}
```

答：变量 **b** 定义出错。按默认构造函数定义对象，不需要加括号。

169 以下代码有什么问题？

```
cout << (true?"1":"") << endl;
```

答：三元表达式“?:”问号后面的两个操作数必须为同一类型。

170 以下代码能够编译通过吗，为什么？

```
unsigned int const size1 = 2;
char str1[ size1 ];
unsigned int temp = 0;
cin >> temp;
unsigned int const size2 = temp;
char str2[ size2 ];
```

答：**str2** 定义出错，**size2** 非编译器期间常量，而数组定义要求长度必须为编译期常量。

171. 以下代码中的输出语句输出 0 吗，为什么？

```
struct CLS
{
    int m_i;
    CLS( int i ) : m_i(i) {}
    CLS()
    {
        CLS(0);
    }
};

CLS obj;
```

```
cout << obj.m_i << endl;
```

答：不能。在默认构造函数内部再调用带参的构造函数属用户行为而非编译器行为，亦即仅执行函数调用，而不会执行其后的初始化表达式。只有在生成对象时，初始化表达式才会随相应的构造函数一起调用。

172 C++中的空类，默认产生哪些类成员函数？

答：

```
class Empty
{
public:
    Empty(); // 缺省构造函数
    Empty( const Empty& ); // 拷贝构造函数
    ~Empty(); // 析构函数
    Empty& operator=( const Empty& ); // 赋值运算符
    Empty* operator&(); // 取址运算符
    const Empty* operator&() const; // 取址运算符 const
};
```

173 以下两条输出语句分别输出什么？

```
float a = 1.0f;
cout << (int)a << endl;
cout << (int&)a << endl;
cout << boolalpha << ( (int)a == (int&)a ) << endl; // 输出什么？
float b = 0.0f;
cout << (int)b << endl;
cout << (int&)b << endl;
cout << boolalpha << ( (int)b == (int&)b ) << endl; // 输出什么？
```

答：分别输出 **false** 和 **true**。注意转换的应用。(int)a 实际上是以浮点数 a 为参数构造了一个整型数，该整数的值是 1，(int&)a 则是告诉编译器将 a 当作整数看（并没有做任何实质上的转换）。因为 1 以整数形式存放和以浮点形式存放其内存数据是不一样的，因此两者不等。对 b 的两种转换意义同上，但是 0 的整数形式和浮点形式其内存数据是一样的，因此在这种特殊情形下，两者相等（仅仅在数值意义上）。

注意，程序的输出会显示(int&)a=1065353216，这个值是怎么来的呢？前面已经说了，1 以浮点数形式存放在内存中，按 **ieee754** 规定，其内容为 0x0000803F（已考虑字节反序）。这也就是 a 这个变量所占据的内存单元的值。当(int&)a 出现时，它相当于告诉它的上下文：“把这块地址当做整数看待！不要管它原来是什么。”这样，内容 0x0000803F 按整数解释，其值正好就是 1065353216（十进制数）。

通过查看汇编代码可以证实“(int)a 相当于重新构造了一个值等于 a 的整型数”之说，而(int&)的作用则仅仅是表达了一个类型信息，意义在于为 cout<<及==选择正确的重载版

174、请简述以下两个 for 循环的优缺点（5 分）

```
for (i=0; i<N; i++)
{
    if (condition)
        DoSomething();
    else
        DoOtherthing();
}
if (condition)
{
    for (i=0; i<N; i++)
        DoSomething();
}
else
{
    for (i=0; i<N; i++)
        DoOtherthing();
}
```

优点：程序简洁

缺点：多执行了 **N-1** 次逻辑判断，并且打断了循环“流水线”作业，使得编译器不能对循环进行优化处理，降低了效率。

优点：循环的效率 high

缺点：程序不简洁

175

```
void GetMemory(char *p)
{
    p = (char *)malloc(100);
}
void Test(void)
{
    char *str = NULL;
    GetMemory(str);
    strcpy(str, "hello world");
    printf(str);
}
```

请问运行 **Test** 函数会有什么样的结果？

答：程序崩溃。

因为 **GetMemory** 并不能传递动态内存，

**Test** 函数中的 **str** 一直都是 **NULL**。

**strcpy(str, "hello world");**将使程序崩溃。

176

```
char *GetMemory(void)
{
char p[] = "hello world";
return p;
}
void Test(void)
{
char *str = NULL;
```

```
str = GetMemory();
printf(str);
}
```

请问运行 **Test** 函数会有什么样的结果？

答：可能是乱码。

因为 **GetMemory** 返回的是指向“栈内存”的指针，该指针的地址不是 **NULL**，但其原现的内容已经被清除，新内容不可知。

177

```
void GetMemory2(char **p, int num)
{
*p = (char *)malloc(num);
}
void Test(void)
{
char *str = NULL;
GetMemory(&str, 100);
strcpy(str, "hello");
printf(str);
}
```

请问运行 **Test** 函数会有什么样的结果？

答：

- (1) 能够输出 **hello**
- (2) 内存泄漏

178

```
void Test(void)
{
char *str = (char *) malloc(100);
strcpy(str, "hello");
free(str);
if(str != NULL)
{
```

```

        strcpy(str, "world");
        printf(str);
    }
}

```

请问运行 **Test** 函数会有什么样的结果？

答：篡改动态内存区的内容，后果难以预料，非常危险。

因为 **free(str)** 之后，**str** 成为野指针，

**if(str != NULL)** 语句不起作用。

179、请阅读以下一段程序，并给出答案。

```

class A
{
public:
    A(){ doSth(); }
    virtual void doSth(){printf("I am A");}
};
class B:public A
{
public:
    virtual void doSth(){ printf("I am B");}
};
B b;

```

执行结果是什么？为什么？

答：执行结果是 **I am A**

因为 **b** 对象构造时调用基类 **A** 的构造函数 **A()**，得此结果。

180 实现双向链表删除一个节点 **P**，在节点 **P** 后插入一个节点，写出这两个函数；

答：双向链表删除一个节点 **P**

```

template<class type> void list<type>::delnode(int p)
{
    int k=1;
    listnode<type> *ptr,*t;
    ptr=first;

    while(ptr->next!=NULL&& k!=p)
    {
        ptr=ptr->next;
        k++;
    }
    t=ptr->next;
    cout<<"你已经将数据项 "<<t->data<<"删除"<<endl;

```

```

ptr->next=ptr->next->next;
length--;
delete t;
}

```

在节点 P 后插入一个节点：

```

template<class type> bool list<type>::insert(type t,int p)
{
    listnode<type> *ptr;
    ptr=first;

    int k=1;
    while(ptr!=NULL&& k<p)
    {
        ptr=ptr->next;
        k++;
    }
    if(ptr==NULL&&k!=p)
        return false;
    else
    {
        listnode<type> *tp;
        tp=new listnode<type>;
        tp->data=t;
        tp->next=ptr->next;
        ptr->next=tp;
        length++;

        return true;
    }
}

```

### 181. 完成下列程序

```

*

```

```

* *
* *

```

```

* * *
* * *

```

```

* * * *
* * * *

```

```

* * * * *
* * * * *

```

```
* * * * *
*****
```

```
* * * * *
*****
```

```
* * * * *
*****
```

```
#include<iostream>
using namespace std;
```

```
const int n = 8;
```

```
main()
{
    int i;
    int j;
    int k;

    for(i = n; i >= 1; i--)
    {
        for(j = 0; j < n-i+1; j++)
        {
            cout<<"*";
            for(k=1; k < n-i+1; k++)
            {
                cout<<" ";
            }
            cout<<endl;
        }
        system("pause");
    }
}
```

182 完成程序，实现对数组的降序排序

```
#include <iostream>
using namespace std;
```

```
void sort(int* arr, int n);
```

```
int main()
```

```
{
    int array[]={45,56,76,234,1,34,23,2,3};
```

```

    sort(array, 9);
    for(int i = 0; i <= 8; i++)//曾经在这儿出界
        cout<<array[i]<<" ";
    cout<<endl;
    system("pause");
}

void sort(int* arr, int n)
{
    int temp;
    for(int i = 1; i < 9; i++)
    {
        for(int k = 0; k < 9 - i; k++)//曾经在这儿出界
        {
            if(arr[k] < arr[k + 1])
            {
                temp = arr[k];
                arr[k] = arr[k + 1];
                arr[k + 1] = temp;
            }
        }
    }
}

```

183. 以下两条输出语句分别输出什么？ [C++难]

```

float a = 1.0f;
cout << (int)a << endl;
cout << (int&)a << endl;
cout << boolalpha << ( (int)a == (int&)a ) << endl; // 输出什么？
float b = 0.0f;
cout << (int)b << endl;
cout << (int&)b << endl;
cout << boolalpha << ( (int)b == (int&)b ) << endl; // 输出什么？
1
1065353216
boolalpha0
0
0
boolalpha1

```

51. 以下反向遍历 `array` 数组的方法有什么错误？ [STL 易]

```

vector array;
array.push_back( 1 );
array.push_back( 2 );

```

```

array.push_back( 3 );
for( vector::size_type i=array.size()-1; i>=0; --i ) // 反向遍历 array 数组
{
    cout << array[i] << endl;
}

```

184 写一个函数，完成内存之间的拷贝。[考虑问题是否全面]

答：

```

void* mymemcpy( void *dest, const void *src, size_t count )
{
    char* pdest = static_cast<char*>( dest );
    const char* psrc = static_cast<const char*>( src );
    if( pdest>psrc && pdest<psrc+count ) 能考虑到这种情况就行了
    {
        for( size_t i=count-1; i!==-1; --i )
            pdest[i] = psrc[i];
    }
    else
    {
        for( size_t i=0; i<count; ++i )
            pdest[i] = psrc[i];
    }
    return dest;
}

int main( void )
{
    char str[] = "0123456789";
    mymemcpy( str+1, str+0, 9 );
    cout << str << endl;

    system( "Pause" );
    return 0;
}

```

185 对于 C++中类(class) 与结构(struct)的描述正确的为:

A,类中的成员默认是 **private** 的,当是可以声明为 **public,private** 和 **protected**,

结构中定义的成员默认的都是 **public**;

B,结构中不允许定义成员函数,当是类中可以定义成员函数;

C,结构实例使用 **malloc()** 动态创建,类对象使用 **new** 操作符动态分配内存;

D,结构和类对象都必须使用 **new** 创建;

E,结构中不可以定义虚函数,当是类中可以定义虚函数.

F,结构不可以存在继承关系,当是类可以存在继承关系.

答:A,D,F

186. 两个互相独立的类:ClassA 和 ClassB,都各自定义了非静态的公有成员函数 PublicFunc() 和非静态的私有成员函数 PrivateFunc();

现在要在 ClassA 中增加定义一个成员函数 ClassA::AdditionalPunction(ClassA a,ClassB b);则可以在 AdditionalPunction(ClassA x,ClassB y)的实现部分(函数功能体内部)出现的合法的表达是最全的是:

- A,x.PrivateFunc();x.PublicFunc();y.PrivateFunc();y.PublicFunc();
- B,x.PrivateFunc();x.PublicFunc();y.PublicFunc();
- C,x.PrivateFunc();y.PrivateFunc();y.PublicFunc();
- D,x.PublicFunc();y.PublicFunc();

答:B

186.C++程序下列说法正确的有:

A,对调用的虚函数和模板类都进行迟后编译.

B,基类与子类中函数如果要构成虚函数,除了要求在基类中用 virtual 声名,而且必须名字相同且参数类型相同返回类型相同

C,重载的类成员函数都必须要有:或者返回类型不同,或者参数数目不同,或者参数序列的类型不同.

D,静态成员函数和内联函数不能是虚函数,友员函数和构造函数也不能是虚函数,但是析构函数可以是虚函数.

答:A

\*\*\*\*\*

187 头文件的作用是什么?

答: 一、通过头文件来调用库功能。在很多场合,源代码不便(或不准)向用户公布,只要向用户提供头文件和二进制的库即可。用户只需要按照头文件中的接口声明来调用库功能,而不必关心接口怎么实现的。编译器会从库中提取相应的代码。

二、头文件能加强类型安全检查。如果某个接口被实现或被使用时,其方式与头文件中的声明不一致,编译器就会指出错误,这一简单的规则能大大减轻程序员调试、改错的负担。

188、以下为 Windows NT 下的 32 位 C++程序,请计算 sizeof 的值(10 分)

```
char str[] = "Hello";
```

```
char *p = str;
```

```
int n = 10;
```

请计算

sizeof (str) = 6 (2 分)

sizeof ( p ) = 4 (2 分)

sizeof ( n ) = 4 (2 分) void Func ( char str[100])

```
{
```

请计算

sizeof( str ) = 4 (2 分)

```
}
```

```
void *p = malloc( 100 );
```

请计算

```
sizeof ( p ) = 4      ( 2 分 )
```

3 写出下列程序的运行结果。

```
unsigned int i=3;
```

```
cout<<i * -1;
```

189.写出下列程序所有可能的运行结果。

```
int a;
```

```
int b;
```

```
int c;
```

```
void F1()
```

```
{
```

```
b=a*2;
```

```
a=b;
```

```
}
```

```
void F2()
```

```
{
```

```
c=a+1;
```

```
a=c;
```

```
}
```

```
main()
```

```
{
```

```
a=5;
```

```
//Start F1,F2 in parallel
```

```
F1(); F2();
```

```
printf("a=%d\n",a);
```

```
}a=11
```

190 一个链表的操作，注意代码的健壮和安全性。要求：

- (1) 增加一个元素；
- (2) 获得头元素；
- (3) 弹出头元素（获得值并删除）。

191.unsigned short array[]={1,2,3,4,5,6,7};

```
int i = 3;  
*(array + i) = ?
```

答:

4

192

```
class A
```

```
{  
    virtual void func1();  
    void func2();  
}
```

```
Class B: class A
```

```
{  
    void func1(){cout << "fun1 in class B" << endl;}  
    virtual void func2(){cout << "fun2 in class B" << endl;}  
}
```

A, A 中的 func1 和 B 中的 func2 都是虚函数.

B, A 中的 func1 和 B 中的 func2 都不是虚函数.

C, A 中的 func2 是虚函数., B 中的 func1 不是虚函数.

D, A 中的 func2 不是虚函数, B 中的 func1 是虚函数.

答:

A

193 输出下面程序结果。

```
#include <iostream.h>
```

```
class A
```

```
{  
public:  
    virtual void print(void)  
    {  
        cout<<"A::print()"<<endl;  
    }  
};
```

```
class B:public A
```

```
{  
public:  
    virtual void print(void)  
    {  
        cout<<"B::print()"<<endl;  
    }  
};
```

```
};  
class C:public B
```

```
{
```

```

public:
    virtual void print(void)
    {
        cout<<"C::print()"<<endl;
    }
};

void print(A a)
{
    a.print();
}

void main(void)
{
    A a, *pa,*pb,*pc;
    B b;
    C c;

    pa=&a;
    pb=&b;
    pc=&c;

    a.print();
    b.print();
    c.print();

    pa->print();
    pb->print();
    pc->print();

    print(a);
    print(b);
    print(c);
}

```

```

A::print()
A::print()
B::print()
C::print()
A::print()
B::print()
C::print()
A::print()
A::print()
A::print()

```

---

194.程序改错

```
class mml
{
    private:
        static unsigned int x;
    public:
        mml(){ x++; }
        mml(static unsigned int &) {x++;}
        ~mml{x--;}
    pulic:
        virtual mon() {} = 0;
        static unsigned int mmc(){return x;}
        .....
};

class nml:public mml
{
    private:
        static unsigned int y;
    public:
        nml(){ x++; }
        nml(static unsigned int &) {x++;}
        ~nml{x--;}
    public:
        virtual mon() {};
        static unsigned int nnc(){return y;}
        .....
};
```

代码片断:

```
mml* pp = new nml;
.....
delete pp;
```

A:

基类的析构函数应该为虚函数

```
virtual ~mml{x--;}
```

---

195.101 个硬币 100 真、1 假，真假区别在于重量。请用无砝码天平称两次给出真币重还是假币重的结论。

答:

101 个先取出 2 堆,

33,33

第一次称,如果不相等,说明有一堆重或轻

那么把重的那堆拿下来,再放另外 35 个中的 33

如果相等,说明假的重,如果不相等,新放上去的还是重的话,说明假的轻(不可能新放上去的轻)

第一次称,如果相等的话,这 66 个肯定都是真的,从这 66 个中取出 35 个来,与剩下的没称过的 35 个比

下面就不用说了

方法二:

第 3 题也可以拿 A(50),B(50)比一下,一样的话拿剩下的一个和真的比一下。

如果不一样,就拿其中的一堆。比如 A(50)再分成两堆 25 比一下,一样的话就在 B(50)中,不一样就在 A(50)中,结合第一次的结果就知道了。

196. 写出程序结果:

```
void Func(char str[100])
{
    printf("%d\n", sizeof(str));
}
```

答:

4

分析:

指针长度

197. int id[sizeof(unsigned long)];

这个对吗? 为什么??

答:

对

这个 sizeof 是编译时运算符,编译时就确定了  
可以看成和机器有关的常量。

198、 sizeof 应用在结构上的情况

请看下面的结构:

```
struct MyStruct
{
    double dda1;
    char dda;
    int type
};
```

对结构 MyStruct 采用 sizeof 会出现什么结果呢? sizeof(MyStruct)为多少呢? 也许你会这样求:

`sizeof(MyStruct)=sizeof(double)+sizeof(char)+sizeof(int)=13`

但是当在 VC 中测试上面结构的大小时，你会发现 `sizeof(MyStruct)` 为 16。你知道为什么在 VC 中会得出这样一个结果吗？

其实，这是 VC 对变量存储的一个特殊处理。为了提高 CPU 的存储速度，VC 对一些变量的起始地址做了"对齐"处理。在默认情况下，VC 规定各成员变量存放的起始地址相对于结构的起始地址的偏移量必须为该变量的类型所占用的字节数的倍数。下面列出常用类型的对齐方式(vc6.0,32 位系统)。

类型

对齐方式（变量存放的起始地址相对于结构的起始地址的偏移量）

Char

偏移量必须为 `sizeof(char)` 即 1 的倍数

int

偏移量必须为 `sizeof(int)` 即 4 的倍数

float

偏移量必须为 `sizeof(float)` 即 4 的倍数

double

偏移量必须为 `sizeof(double)` 即 8 的倍数

Short

偏移量必须为 `sizeof(short)` 即 2 的倍数

各成员变量在存放的时候根据在结构中出现的顺序依次申请空间，同时按照上面的对齐方式调整位置，空缺的字节 VC 会自动填充。同时 VC 为了确保结构的大小为结构的字节边界数（即该结构中占用最大空间的类型所占用的字节数）的倍？

199

```
#include "stdafx.h"
_Y n P }2{&k O v H ` ,o0
#define SQR(X) X*X
int main(int argc, char* argv[])
{
    int a = 10;
    int k = 2;
    int m = 1;
    a /= SQR(k+m)/SQR(k+m);
    printf("%d\n",a);
    return 0;
}
```

这道题目的结果是什么啊？

`define` 只是定义而已，在编译时只是简单代换 `X*X` 而已，并不经过算术法则的

`a /= k+m*k+m/k+m*k+m;`

`=>a /= (k+m)*1*(k+m);`

`=>a = a/9;`

`=>a = 1;`

200. 下面的代码有什么问题？

```

void DoSomething(...)
{
    char* p;
    p = malloc(1024); // 分配 1K 的空间
    2y x
    if (NULL == p)
        return;
    p = realloc(p, 2048); // 空间不够，重新分配到 2K
    if (NULL == p)
        return;
}

```

A:

p = malloc(1024);      应该写成: p = (char \*) malloc(1024);  
 没有释放 p 的空间，造成内存泄漏。

201 下面的代码有什么问题？并请给出正确的写法。

```

void DoSomething(char* p)
{
    char str[16];
    int n;
    assert(NULL != p);
    sscanf(p, "%s%d", str, n);
    if (0 == strcmp(str, "something"))
    {
    }
}

```

A:

sscanf(p, "%s%d", str, n);      这 句 该 写 成 :      sscanf(p, "%s%d", str, &n);

---

202. 下面代码有什么错误？

```

Void test1()
{

    char string[10];
    char *str1="0123456789";
    strcpy(string, str1);

}

```

数组越界

203. 下面代码有什么问题？

```

Void test2()

{

char string[10], str1[10];

for(i=0; i<10;i++)
{
    str1[i] ='a';
}
strcpy(string, str1);
}
}
数组越界

```

204 下面代码有什么问题?LUPA 开源社区+j H2B F,c U

```

Void test3(char* str1)
{
char string[10];
if(strlen(str1)<=10)
{
    strcpy(string, str1);
}
}
==数组越界

```

==strcpy 拷贝的结束标志是查找字符串中的\0 因此如果字符串中没有遇到\0 的话 会一直复制, 直到遇到\0,上面的 123 都因此产生越界的情况  
建议使用 strncpy 和 memcpy

205.写出运行结果:

```

{
    char str[] = "world"; cout << sizeof(str) << ": ";
    char *p    = str;     cout << sizeof(p) << ": ";
    char i      = 10;      cout << sizeof(i) << ": ";
    void *pp    = malloc(10); cout << sizeof(p) << endl;
}
6: 4: 1: 4

```

---

206.C 和 C++有什么不同?

从机制上: c 是面向过程的(但 c 也可以编写面向对象的程序); c++是面向对象的, 提供了类。但是, c++编写面向对象的程序比 c 容易  
从适用的方向: c 适合要求代码体积小的, 效率高的场合, 如嵌入式; c++适合更上层的,

复杂的：Linux 核心大部分是 C 写的，因为它是系统软件，效率要求极高。  
从名称上也可以看出，C++ 比 C 多了 +，说明 C++ 是 C 的超集；那为什么不叫 C+ 而叫 C++ 呢，  
是因为 C++ 比  
C 来说扩充的东西太多了，所以就在 C 后面放上两个 +；于是就成了 C++  
C 语言是结构化编程语言，C++ 是面向对象编程语言。LUPA 开源社区 } n\*r2C/M8f  
C++ 侧重于对象而不是过程，侧重于类的设计而不是逻辑的设计。

207 在不用第三方参数的情况下，交换两个参数的值

```
#include <stdio.h>
void main()
{
    int i=60;
    int j=50;
    i=i+j;
    j=i-j;
)
    i=i-j;
    printf("i=%d\n",i);
    printf("j=%d\n",j);
}
```

方法二：

```
i^=j;
j^=i;
i^=j;
```

方法三：

```
// 用加减实现，而且不会溢出
a = a+b-(b=a)
```

208. 下面的函数实现在一个固定的数上加上一个数，有什么错误，改正

```
int add_n(int n)
{
    static int i=100;
    i+=n;
    return i;
}
```

答：

因为 static 使得 i 的值会保留上次的值。  
去掉 static 就可了

```
209. union a {
    int a_int1;
    double a_double;
    int a_int2;
```

```
};
typedef struct
{ a a1;
char y;
} b;
```

```
class c
{
double c_double;
b b1;
a a2;
};
```

输出 `cout<<sizeof(c)<<endl;`的结果?

答:

VC6 环境下得出的结果是 32

我(sun)在 VC6.0+win2k 下做过试验:

int-4

float-4

double-8

指针-4

210. `unsigned short array[]={1,2,3,4,5,6,7};`

`int i = 3;`

`*(array + i) = ?`

答:4

211. class A

```
{
virtual void func1();
void func2();
}
```

Class B: class A

```
{
void func1(){cout << "fun1 in class B" << endl;}
virtual void func2(){cout << "fun2 in class B" << endl;}
}
```

A, A 中的 func1 和 B 中的 func2 都是虚函数.

B, A 中的 func1 和 B 中的 func2 都不是虚函数.

C, A 中的 func2 是虚函数., B 中的 func1 不是虚函数.

D, A 中的 func2 不是虚函数, B 中的 func1 是虚函数.

答:

A

212 输出下面程序结果。

```
#include <iostream.h>
class A
{
public:
virtual void print(void)
{
    cout<<"A::print()"<<endl;
}
};

class B:public A
{
public:
virtual void print(void)
{
    cout<<"B::print()"<<endl;
};
};

class C:public
{
public:
virtual void print(void)
{
    cout<<"C::print()"<<endl;
}
};

void print(A a)
{
    a.print();
}

void main(void)
{
    A a, *pa,*pb,*pc;
    pa=&a;
    pb=&b;
    pc=&c;

    a.print();
    b.print();
    c.print();
}
```

```

a->print();
pb->print();
pc->print();

print(a);
print(b);
print(c);
}
A::print()
A::print()

B::print()
C::print()

A::print()

B::print()

C::print()

A::print()

A::print()

A::print()

```

- 213 C++语言是在\_\_ C \_\_语言的基础上发展起来的。
- 214 C++语言的编译单位是扩展名为\_\_ .cpp \_\_的\_\_ 程序 \_\_文件。
215. 行尾使用注释的开始标记符为\_\_ // \_\_。
- 216 多行注释的开始标记符和结束标记符分别为\_\_ /\* \_\_和\_\_ \*/ \_\_。
217. 用于输出表达式值的标准输出流对象是\_\_ cout \_\_。
- 218 用于从键盘上为变量输入值的标准输入流对象是\_\_ cin \_\_。
219. 一个完整程序中必须有一个名为\_\_ main \_\_的函数。
- 220 一个函数的函数体就是一条\_\_ 复合 \_\_语句。
221. 当执行 cin 语句时，从键盘上输入每个数据后必须接着输入一个\_\_ 空白 \_\_符，然后才能继续输入下一个数据。
- 222 在 C++程序中包含一个头文件或程序文件的预编译命令为\_\_ #include \_\_。
223. 程序中的预处理命令是指以\_\_ # \_\_字符开头的命令。
224. 一条表达式语句必须以\_\_ 分号 \_\_作为结束符。
225. 在#include 命令中所包含的头文件，可以是系统定义的头文件，也可以是\_\_ 用户（或编程者） \_\_定义的头文件。
226. 使用#include 命令可以包含一个头文件，也可以包含一个\_\_ 程序 \_\_文件。

227. 一个函数定义由\_\_函数头\_\_\_\_和\_\_函数体\_\_\_\_两部分组成。
228. 若一个函数的定义处于调用它的函数之前,则在程序开始可以省去该函数的\_\_原型(或声明)\_\_\_\_语句。
229. C++头文件和源程序文件的扩展名分别为\_\_\_.h\_\_\_\_和\_\_\_\_.cpp\_\_\_\_。
230. 程序文件的编译错误分为\_\_\_\_警告(warning)\_\_\_\_和\_\_\_\_致命(error)\_\_\_\_两类。
231. 当使用\_\_\_\_void\_\_\_\_保留字作为函数类型时,该函数不返回任何值。
232. 当函数参数表用\_\_\_\_void\_\_\_\_保留字表示时,则表示该参数表为空。
233. 从一条函数原型语句“int fun1(void);”可知,该函数的返回类型为\_\_\_\_\_,该函数带有\_\_\_\_\_个参数。
234. 当执行 cout 语句输出 endl 数据项时,将使 C++显示输出屏幕上的光标从当前位置移动到\_\_\_\_下一行\_\_\_\_的开始位置。
235. 假定 x=5, y=6,则表达式 x++\*++y 的值为\_\_\_\_35\_\_\_\_\_。
236. 假定 x=5, y=6,则表达式 x--\*--y 的值为\_\_\_\_25\_\_\_\_\_。
237. 假定 x=5, y=6,则执行表达式 y\*=x++计算后, x 和 y 的值分别为\_\_\_\_6\_\_\_\_和\_\_\_\_30\_\_\_\_\_。
238. 假定 x=5, y=6,则执行表达式 y+=x--计算后, x 和 y 的值分别为\_\_\_\_4\_\_\_\_和\_\_\_\_11\_\_\_\_\_。
239. C++常数 0x145 对应的十进制值为\_\_\_\_325\_\_\_\_\_。
240. C++常数 0345 对应的十进制值为\_\_\_\_229\_\_\_\_\_。
241. 十进制常数 245 对应的十六进制的 C++表示为\_\_\_\_0xF5\_\_\_\_\_。
242. 十进制常数 245 对应的八进制的 C++表示为\_\_\_\_0365\_\_\_\_\_。
243. signed char 类型的值域范围是\_\_ -128\_\_至\_\_ +127\_\_之间的整数。
244. int 和 float 类型的数据分别占用\_\_\_\_4\_\_\_\_和\_\_\_\_4\_\_\_\_个字节。
245. float 和 double 类型的数据分别占用\_\_\_\_4\_\_\_\_和\_\_\_\_8\_\_\_\_个字节。
246. bool 和 char 类型的数据分别占用\_\_\_\_1\_\_\_\_和\_\_\_\_1\_\_\_\_个字节。
247. unsigned short int 和 int 类型的长度分别为\_\_\_\_2\_\_\_\_和\_\_\_\_4\_\_\_\_\_。
248. 字符串“This’s a book.\n”的长度为\_\_\_\_15\_\_\_\_\_。
249. 字符串“\nThis’s a pen\n\n”的长度为\_\_\_\_15\_\_\_\_\_。
250. 在 C++中存储字符串“abcdef”至少需要\_\_\_\_7\_\_\_\_个字节。
251. 在 C++中存储字符串“a+b=c”至少需要\_\_\_\_6\_\_\_\_个字节。
252. 假定 x 和 y 为整型,其值分别为 16 和 5,则 x%y 和 x/y 的值分别为\_\_\_\_1\_\_\_\_\_和\_\_\_\_3\_\_\_\_\_。
253. 假定 x 和 y 为整型,其值分别为 16 和 5,则 x/y 和 double(x)/y 的值分别为\_\_\_\_3\_\_\_\_\_和\_\_\_\_3.2\_\_\_\_\_。
254. 假定 x 是一个逻辑量,则 x && true 的值为\_\_\_\_x\_\_\_\_\_。
255. 假定 x 是一个逻辑量,则 x || true 的值为\_\_\_\_true(或 1)\_\_\_\_\_。
256. 假定 x 是一个逻辑量,则 x && false 的值为\_\_\_\_false(或 0)\_\_\_\_\_。
257. 假定 x 是一个逻辑量,则 x || false 的值为 x。
258. 假定 x 是一个逻辑量,则 !x || false 的值为\_\_\_\_!x\_\_\_\_\_。
259. 假定 x 是一个逻辑量,则 x && !x 的值为\_\_\_\_false(或 0)\_\_\_\_\_。
260. 假定 x 是一个逻辑量,则 x || !x 的值为\_\_\_\_true(或 1)\_\_\_\_\_。
261. 设 enum Printstatus{ready,busy,error}; 则 cout<<busy 的输出结果是\_\_\_\_1\_\_\_\_\_。
262. 设 enum Printstatus{ready=2,busy,error}; 则 cout<<busy 的输出结果是\_\_\_\_3\_\_\_\_\_。
263. 常数-4.205 和 6.7E-9 分别具有\_\_\_\_4\_\_\_\_和\_\_\_\_2\_\_\_\_位有效数字。
264. 枚举类型中的每个枚举值都是一个\_\_\_\_枚举常量\_\_\_\_,它的值为一个\_\_\_\_整数\_\_\_\_\_。

265. 常数 100 和 3.62 的数据类型分别为 int 和 double。
266. 若  $x=5$ ,  $y=10$ , 则计算  $y*=++x$  表达式后,  $x$  和  $y$  的值分别为 6 和 60。
267. 假定  $x$  和  $ch$  分别为 `int` 型和 `char` 型, 则 `sizeof(x)` 和 `sizeof(ch)` 的值分别为 4 和 1。
268. 假定  $x=10$ , 则表达式  $x \leq 10 ? 20 : 30$  的值为 20。
269. 表达式 `sqrt(81)` 和 `pow(6,3)` 的值分别为 9 和 216。
270. 含随机函数的表达式 `rand()%20` 的值在 0 至 19 区间内。
271. 在 `switch` 语句中, 每个语句标号所含关键字 `case` 后面的表达式必须是 常量。
272. 在 `if` 语句中, 每个 `else` 关键字与它前面同层次并且最接近的 if 关键字相配套。
273. 作为语句标号使用的 C++ 保留字 `case` 和 `default` 只能用于 switch 语句的定义体中。
274. 执行 `switch` 语句时, 在进行作为条件的表达式求值后, 将从某个匹配的标号位置起向下执行, 当碰到下一个标号位置时 (停止/不停止) 不停止 执行。
275. 若 `while` 循环的“头”为“`while(i++<=10)`”, 并且  $i$  的初值为 0, 同时在循环体中不会修改  $i$  的值, 则循环体将被重复执行 11 次后正常结束。
276. 若 `do` 循环的“尾”为“`while(++i<10)`”, 并且  $i$  的初值为 0, 同时在循环体中不会修改  $i$  的值, 则循环体将被重复执行 10 次后正常结束。
277. 当在程序中执行到 `break` 语句时, 将结束本层循环类语句或 `switch` 语句的执行。
278. 当在程序中执行到 continue 语句时, 将结束所在循环语句中循环体的一次执行。
279. 在程序中执行到 return 语句时, 将结束所在函数的执行过程, 返回到调用该函数的位置。
280. 在程序执行完 主(或 main) 函数调用后, 将结束整个程序的执行过程, 返回到 C++ 集成开发窗口。
281. 元素类型为 `int` 的数组 `a[10]` 共占用 40 字节的存储空间。
282. 元素类型为 `double` 的二维数组 `a[4][6]` 共占用 192 字节的存储空间。
283. 元素类型为 `char` 的二维数组 `a[10][30]` 共占用 300 字节的存储空间。
284. 存储字符 'a' 和字符串 "a" 分别需要占用 1 和 2 个字节。

285

```
#include "stdafx.h"
#define SQR(X) X*X
int main(int argc, char* argv[])
{
    int a = 10;
    int k = 2;
    int m = 1;
    a /= SQR(k+m)/SQR(k+m);
    printf("%d\n",a);
    return 0;
}
```

这道题目的结果是什么啊?

`define` 只是定义而已, 在编译时只是简单代换 `X*X` 而已, 并经过算术法则的

```

a /= (k+m)*(k+m)/(k+m)*(k+m);
=>a /= (k+m)*1*(k+m);
=>a = a/9;
=>a = 1;

```

286. 以面向对象方法构造的系统，其基本单位是\_\_\_\_\_对象\_\_\_\_\_。

287. 每个对象都是所属类的一个\_\_实例\_\_。

288. C++支持两种多态性：\_\_\_\_编译\_\_\_\_时的多态性和\_\_\_\_运行\_\_时的多态性。

289. 在 C++中，编译时的多态性是通过\_\_\_\_重载\_\_\_\_实现的，而运行时的多态性则是通过\_\_\_\_虚函数\_\_\_\_实现的。

290. 对于类中定义的任何成员，其隐含访问权限为\_\_\_\_ private（或私有）\_\_。

291. 对于结构中定义的任何成员，其隐含访问权限为\_\_ public(或公有)\_\_\_\_\_。

292. 若在类的定义体中给出了一个成员函数的完整定义，则该函数属于\_\_内联\_\_函数。

293. 为了避免在调用成员函数时修改对象中的任何数据成员，则应在定义该成员函数时，在函数头的后面加上\_\_ const \_\_关键字。

294. 若只需要通过一个成员函数读取数据成员的值，而不需要修改它，则应在函数头的后面加上\_\_ const \_\_关键字。

295. 判断一个字符串是不是回文

```

int IsReverseStr(char *aStr)
{
    int i,j;
    int found=1;
    if(aStr==NULL)
        return -1;
    j=strlen(aStr);
    for(i=0;i<j/2;i++)
        if(*(aStr+i)!=*(aStr+j-i-1))
        {
            found=0;
            break;
        }
    return found;
}

```

296. .写出判断 ABCD 四个表达式的是否正确，若正确，写出经过表达式中 a 的值(3 分)

```
int a = 4;
```

(A) a += (a++); (B) a += (++a); (C) (a++) += a; (D) (++a) += (a++);

a = ?

答：C 错误，左侧不是一个有效变量，不能赋值，可改为(++a) += a;

改后答案依次为 9,10,10,11

298. 动态连接库的两种方式？

答：调用一个 DLL 中的函数有两种方法：

1. 载入时动态链接 (load-time dynamic linking), 模块非常明确调用某个导出函数, 使得他们就像本地函数一样。这需要链接时链接那些函数所在 DLL 的导入库, 导入库向系统提供了载入 DLL 时所需的信息及 DLL 函数定位。
2. 运行时动态链接 (run-time dynamic linking), 运行时可以通过 LoadLibrary 或 LoadLibraryEx 函数载入 DLL。DLL 载入后, 模块可以通过调用 GetProcAddress 获取 DLL 函数的出口地址, 然后就可以通过返回的函数指针调用 DLL 函数了。如此即可避免导入库文件了。

299. 请写出下列代码的输出内容

```
#include
main()
{
    int a,b,c,d;
    a=10;
    b=a++;
    c=++a;
    d=10*a++;
    printf("b, c, d: %d, %d, %d", b, c, d);
    return 0;
}
```

答、10, 12, 120

300. 设有以下说明和定义:

```
typedef union {long i; int k[5]; char c;} DATE;
struct data { int cat; DATE cow; double dog;} too;
DATE max;
```

则语句 printf("%d",sizeof(struct data)+sizeof(max));的执行结果是?

答、结果是: 52。DATE 是一个 union, 变量公用空间。里面最大的变量类型是 int[5], 占用 20 个字节。所以它的大小是 20

data 是一个 struct, 每个变量分开占用空间。依次为 int4 + DATE20 + double8 = 32.

所以结果是 20 + 32 = 52.

当然...在某些 16 位编辑器下, int 可能是 2 字节,那么结果是 int2 + DATE10 + double8 = 20

301. 以下代码有什么问题?

```
cout << (true?"1") << endl;
```

答: 三元表达式“?:”问号后面的两个操作数必须为同一类型。

302. 以下代码能够编译通过吗, 为什么?

```
unsigned int const size1 = 2;
char str1[ size1 ];
unsigned int temp = 0;
cin >> temp;
unsigned int const size2 = temp;
char str2[ size2 ];
```

答：str2 定义出错，size2 非编译器期间常量，而数组定义要求长度必须为编译期常量。

303. 以下反向遍历 array 数组的方法有什么错误？

```
vector array;
array.push_back( 1 );
array.push_back( 2 );
array.push_back( 3 );
for( vector::size_type i=array.size()-1; i>=0; --i ) // 反向遍历 array 数组
{
    cout << array[i] << endl;
}
```

答：首先数组定义有误，应加上类型参数：vector<int> array。其次 vector::size\_type 被定义为 unsigned int，即无符号数，这样做为循环变量的 i 为 0 时再减 1 就会变成最大的整数，导致循环失去控制。

304. 以下代码中的输出语句输出 0 吗，为什么？

```
struct CLS
{
    int m_i;
    CLS( int i ) : m_i(i) {}
    CLS()
    {
        CLS(0);
    }
};
CLS obj;
cout << obj.m_i << endl;
```

答：不能。在默认构造函数内部再调用带参的构造函数属用户行为而非编译器行为，亦即仅执行函数调用，而不会执行其后的初始化表达式。只有在生成对象时，初始化表达式才会随相应的构造函数一起调用。

1. 指出以下变量数据存储位置

全局变量 `int(*g_pFun)(int);g_pFun=myFunction;g_pFun` 存储的位置(A) 为全局的函数指针

指向空间的位置(B) 所有函数代码位于 TEXT 段

函数内部变量 `static int nCount;` (A) 静态变量总是在 DATA 段或 BSS 段中

函数内部变量 `char p[]="AAA";` p 指向空间的位置(C) 局域变量的静态数组,空间在 Stack 中

函数内部变量 `char *p="AAA";` p 指向空间的位置(E), "AAA"为一字符常量空间,不同编译器有不同处理方法,大部分保存在 TEXT(代码段中),也有编译的 rodata 段中

函数内部变量 `char *p=new char;` p 的位置(C) 指向空间的位置(D) 所有 malloc 空间来自于 heap(堆)

A. 数据段

B. 代码段

C. 堆栈

D. 堆

E. 不一定,视情况而定

以上知识参见 C 语言变量的作用域相关课件

2. 以下程序的输出结果为 ( )

```
#include <iostream>
```

```
main( )
```

```
{
```

```
    using namespace std;
```

```
    int num[5]={1,2,3,4,5};
```

```

        cout <<*((int *)&num+1)-1 <<endl;
    }

```

A. 1    B. 2    C. 3    D. 4    E. 5    F. 0    G. 未初始化内存,无法确定

在 C 语言中,一维数组名表示数组的首地址,而且是一个指针.如上例 num,

对&num,表示指针的指针.意味着这里强制转换为二维数组指针.

这样 &num+1 等同于 num[5][1],为代码空间. (&num+1)-1 表示 num[4][0].即 num[4].所以这里答案是 E.

扩展题目:

\*((int \*)&num+1)-1 的值是多少?

Num 是首指针,num+1 是第二个元素指针,-1 后又变成首指针.所以这里是答案是 num[0] 即,A.1

3. 以下哪些是程序间可靠的通讯方式( C ),哪些可以用于跨主机通讯( C,D ,F).Windows 命名管道跨机器也可跨机器.

A. 信号    B. 管道    C. TCP    D. UDP    E. PIPE    F,.串口 I/O

4. class a

```

{
    public:
        virtual void funa( );
        virtual void funb( );
        void fun( );
        static void fund( );
        static int si;

```

private:

```

    int i;
    char c;

```

```

};

```

问: 在 32 位编译器默认情况下,sizeof(a)等于( )字节?

A. 28    B. 25    C.24    D. 20    E. 16    F.12    G. 8

答案在 VC++ 下是 12. 这里需要考虑三个问题,一是虚函数表 vtable 的入口表地址,二是字节对齐.三 ,静态成员是所有对象共享,不计入 sizeof 空间.

在大部分 C++的实现中,带有虚函数的类的前 4 个 BYTE 是虚函数 vtable 表的这个类入口地址.所以 sizeof 必须要加入这个 4 个 byte 的长度,除此外,类的 sizeof()为所有数据成员总的 sizeof 之和,这里是 int i,和 char c.其中 char c 被字节对齐为 4.这样总长度为

Sizeof(a) = sizeof(vtable)+size(int)+sizeof(char + pad) = 12;

5. 32 位 Windows 系统或 Linux 系统下

```

struct
{
    char a;
    char b;
    char c;

```

```

}A;
struct
{
    short a;
    short b;
    short c;
}B;
struct
{
    short a;
    long b;
    char c;
}C;
printf("%d,%d,%d\n",sizeof(A),sizeof(B),sizeof(C)); 的执行结果为: ( )
A. 3,6,7    B. 3,6,8    C. 4,8,12    D. 3,6,12    E. 4,6,7    F. 4,8,9

```

C 语法的字节对齐规则有两种情况要字节对齐, 在 VC++,gcc 测试都是如此

- 1) 对同一个数据类型 (short,int,long) 发生了跨段分布,(在 32CPU 里,即一个数据类型分布在两个段中)才会发生字节对齐.
  - 2) 数据类型的首部和尾部必须有其一是与 4 对齐.而且违反上一规则.
- sizeof(A),sizeof(B)虽然总字节数不能被 4 整除.但刚好所有数据平均分布在以 4 为单位的各个段中.所以无需字节对齐,所以结果是 3 和 6
  - struct {char a;char b;char c;char d;char e;}F; 的 sizeof(F)是等于 5.
  - 用以下实例更加清楚

```

struct {
    char a[20];
    short b;
}A;
struct {
    char a[21];
    short b;
}B;

```

sizeof(A)=22,sizeof(B)=24.因为前者没有发生跨段分布.后者,如果不字节对齐.a[21]占用最后一个段的首地址,b 无法作到与首部与尾部与 4 对齐,只能在 a[21]与 b 之间加入一个 byte,使用 b 的尾部与 4 对齐.

- C 就是比较好理解.要补多个成 12

6. 依据程序,以下选择中那个是对的? ( )

```

class A
{
    int m_nA;
};
class B
{

```

```

        int m_nB;
};
class C:public A,public B
{
    int m_nC;
};
void f(void)
{
    C* pC=new C;
    B* pB=dynamic_cast<B*>(pC);
    A* pA=dynamic_cast<A*>(pC);
}

```

A. pC= =pB,(int)pC= =(int)B

C. pC!=pB,(int)pC= =(int)pB

B. pC= =pB,(int)pC!=(int)pB

D. pC!=pB,(int)pC!=(int)pB

这里主要考多态..将程序变为如下比较易懂

```
#include <stdio.h>
```

```

class A
{
public:
    int m_nA;
};
class B
{
public:
    int m_nB;
};
class C:public A,public B
{
public:
    int m_nC;
};

```

```

void f(void)
{
    C* pC=new C;
    B* pB=dynamic_cast<B*>(pC);
    A* pA=dynamic_cast<A*>(pC);
}

```

```

void f1(void)
{

```

```

C* pC=new C;
pC->m_nA = 1;
pC->m_nB = 2;
pC->m_nC = 3;
B* pB=dynamic_cast<B*>(pC);
A* pA=dynamic_cast<A*>(pC);

printf("A=%x,B=%x,C=%x,iA=%d,iB=%d,iC=%d\n",pA,pB,pC,(int)pA,(int)pB,(int)pC);
}

```

```
void test1();
```

```

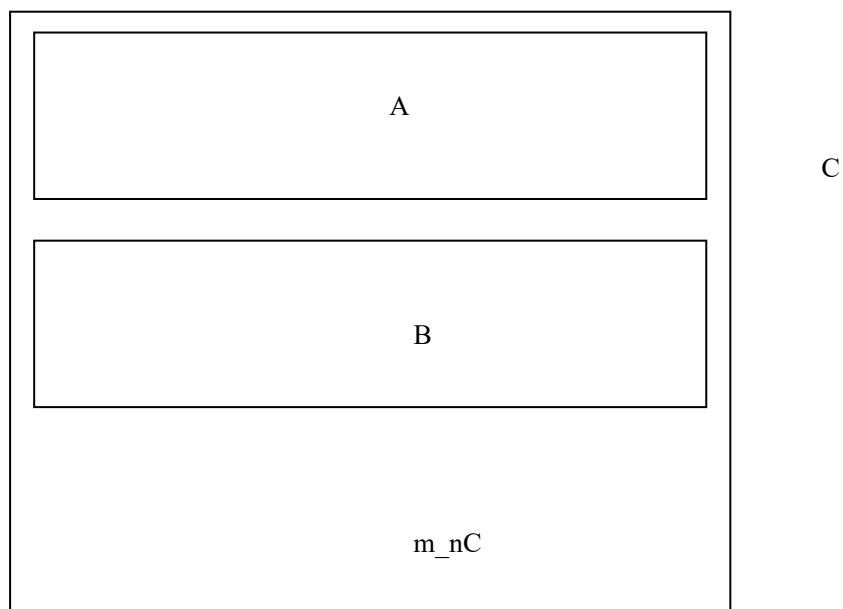
int main()
{
    // test1();
    f1();
    getchar();
    return 0;
}

```

以上程序输出:

**A=4318d0,B=4318d4,C=4318d0,iA=4397264,iB=4397268,iC=4397264**

即 C 从 A,B 继承下来,由下图可以知道 pA=pC.而 pB 强制转换后,只能取到 C 中 B 的部分.  
所以 pB 在 pC 向后偏移 4 个 BYTE,(即 m\_nA)的空间



7, 请写出能匹配 "[10]:dddddd " 和 "[9]:abcdegf ", 不匹配 "[a]:xfdf " 的正则表达式  
\_\_\_\_\_,linux 下支持正则的命令有:\_\_\_find,grep\_\_\_\_\_

8.如下程序:

```
int i=1,k=0;
```

```

long *pl=NULL;
char *pc=NULL;
if(k++&& i++)
    k++, pl++, pc++;
if(i++||k++)
    i++, pl++, pc++;
printf("i=%d,k=%d,pl=%ld,pc=%ld\n",i,k,(long)pl,(long)pc);

```

打印结果为\_\_i=3,k=1,pl=4,pc=1\_\_\_\_\_

主要测试逻辑表达式的短路操作.

&&操作中,前一个表达式为 0,后一表达式不执行

||操作中,前一个表达式为 1,后一表达式不执行

9. 以下程序的输出为\_\_\_\_\_

```

#include<iostream>
using std::cout;
class A
{
public:
    void f(void){
        cout<< "A::f" <<' ';
    }
    virtual void g(void)
    {
        cout <<"A::g" << ' ';
    }
};

class B : public A
{
public:
    void f(void)
    {
        cout << "B :: f " << ' ';
    }
    void g(void)
    {
        cout << "B:: g " << ' ';
    }
};

int main()
{

```

```

A* pA = new B;
pA->f();
pA->g();
B* pB = (B*)pA;
pB->f();
pB->g();
}

```

A::f B::g B::f B::g

多态中虚函数调用.

f()为非虚函数,这样强制转换后,执行本类的同名函数.

G()为虚函数,指针总是执行虚函数,这就是多态..

10.下列代码的作用是删除 list lTest 中值为 6 的元素:

```

list<int> :: iterator Index = lTest .begin();
for( ; Index != lTest .end(); ++ Index)
{
    if((*Index) == 6)
    {
        lTest .erase(Index);
    }
}

```

请问有什么错误\_\_\_\_\_ Index = lTest .erase(Index);\_\_\_\_\_ ,

STL 的游标处理,erase 已经将 Index 破坏掉,需要用新的 Index,否则下一循环的++Index 被破坏掉

请写出正确的代码,或者在原代码上修正.

11.找错误\_以下程序:

```

char* ptr = malloc(100);
if(!ptr)
{
    ...
}
...
//ptr 指向的空间不够需要重新分配
ptr = realloc(ptr,200);
if(!ptr)
{
    ...
}
...

```

请问有什么错误\_\_\_\_\_ if(ptr == NULL)\_\_\_\_\_,请写出正确的代码,或者在原代码上修正.

12.以下为 window NT 下 32 位 C++程序,请填写如下值

class myclass

```

{
    int a ;
    int b;
};
    char *p = "hello";
    char str[] = "world";
    myclass classes[2];
void *p2= malloc(100);

```

```

sizeof(p)=_4__
sizeof(str)=_6_
sizeof(classes)=_16__
sizeof(p2)=_4__

```

13.直接在以下程序中的错误的行数后的填空栏中打叉

程序 1:

```

int main(void)
{
    int i=10;_____
    int *const j=&i;_____
    (*j)++;_____
    j++;___*_____
}

```

程序 2:

```

int main(void)
{
    int i=20;_____
    const int *j=&i;_____
    *j++;_____
    (*j)++;___*_____
}

```

主要考 **const** 出现在\*前后不同含意,**const** 在\*后表示指针本身不能改,**const** 在\*前面指针内容不能改,程序 1 中 j 不能修改指针,所以 j++是错,程序 2,j 不能改改内容,所以

14.用 C/C++代码实现以下要求:从 1-100 中挑选出 10 个不同的数字,请把可能的所有组合打印出来.

15.有一个非常大的全局数组 `int a[]`,长度 `n` 超过 2 的 24 次方,写一个针对该数组的查找算法 `unsigned search(int value)`(返回值下标),插入算法 `insert(int value,unsigned index)`.再次注意该数组的长度很长.

题目不太清,可能可以把数值本身作下标.并且按顺序排序.

16.有两个单向链表,表头 `pHeader1`,`pHeader2`,请写一个函数判断这两个链表是否有交叉.如果有交叉,给出交叉点.程序不能改变链表的内容,可以使用额外的空间,时间复杂度尽量小,最

好给出两种解.(双重循环的解由于时间复杂度高, 不算正解).

1.移动链表指针,如果最终

17.编写程序,将一棵树从根节点到叶子的所有最长路径都打印出来.比如一棵树从跟到最末端的叶子最远要经

过 4 个节点,那么就打印出所有要经过 4 个节点才能到达的叶子的搜索路径(所有途径节点)都分别打印出来.

18.请分别对一个链表和一个数组进行排序,并指出你排序的算法名称以及为何选择该算法  
数组可用交换法排序

19.有单向链表,其中节点结构为 `Node{int value;Node *pNext};`只知道指向某个节点的指针 `pCurrent`;并且知道该节点不是尾节点,有什么办法把他删除吗?要求不断链.

从链表头开始,找到 `pCurrent` 上一个结点 `pPrev`,然后 `pPrev->pNext = pCurrent->pNext;`

20.问题 A:用什么方法避免 c/c++编程中的头文件重复包含?问题 B:假设解决了重复包含问题,但是又需要在两个不同的头文件中引用各声明的类,应该如何处理?具体代码如下:

在头文件 `Man.h` 中

```
....  
Class Cman  
{  
    ....  
    CFace m_face;  
};  
....
```

在头文件 `Face.h` 中

```
...  
Class CFace  
{  
    ...  
    Cman *m_owner;  
};  
....
```

这样类 `CMan.CFace` 就相互引用了,该如何处理呢?

1.`#ifndef ....`

`#define .....`

2.类的前向声明

21.多线程和单线程各自分别在什么时候效率更高?

多线程在并发,并且各线程无需访问共享数据情况详细最高

如果多线程过于频繁切换,或共享数据很多情况下,使用单线程较好

22.在程序设计中,对公共资源(比如缓冲区等)的操作和访问经常需要使用锁来进行保护,但在大并发系统中过多的锁会导致效率很低,通常有那些方法可以尽量避免或减少锁的使用?  
减少锁的粒度,每次尽可能减少锁范围  
采用队列处理,这样无需使用锁.

23.请详细阐述如何在 **release** 版本(windows 程序或 linux 程序都可以)中,查找段错误问题.  
可以用编译器生成 **map** 文件来定位源码.通过地址反查源码

24.假设你编译链接 **release** 版本后得到一个可执行程序(由多个 **cpp** 文件和 **H** 文件编译),结果可执行程序文件非常大,你如何找到造成文件太大的可能原因,可能的原因是什么?  
使用一个已经初始化的巨大的全局数组

25.在编写 **C++**赋值运算符时有哪些要注意的地方?

返回值,参数最好用引用

减少友元函数使用,移植有问题.

26.假设你是参与设计嫦娥卫星的嵌入式单板软件工程师,其中有一个快速搜索可能要用到哈希变或者平衡二叉树,要求不管什么条件下,单板必须在指定的短时间内有输出,你会采取那种算法?为什么用这种算法,为什么不用另一种算法?

**HASH.HASH** 访问速度较快.

27.**strcpy()**容易引起缓冲区溢出问题,请问有什么函数可以替代以减少风险,为什么?

**strncpy**

28.请指出 **spinlock,mutex,semaphore,critical section** 的作用与区别,都在哪些场合使用.

**spin\_lock** Linux 内核自旋锁. **Mutex** Windows 互质量, **semaphore** POSIX ,**critical section** Windows

29.在哪些方法使阻塞模式的 **recv** 函数在没有收到数据的情况下返回(不能将 **socket** 修改为非阻塞模式)请描述得详细点.

使用 **select**

30.有 3 个红色球,2 个白色球,1 个绿色球.取出两个不同颜色的球就能变成两个第三种颜色的球(比如:取出 1 红球,1 白球,就能变成 2 个绿球).问,最少几次变化能将所有球都变成同一颜色,说明步骤和原因?

31.单向链表的反转是一个经常被问到的一个面试题,也是一个非常基础的问题.比如一个链表是这样的: 1->2->3->4->5 通过反转后成为 5->4->3->2->1。

最容易想到的方法遍历一遍链表,利用一个辅助指针,存储遍历过程中当前指针指向的下一个元素,然后将当前节点元素的指针反转后,利用已经存储的指针往后面继续遍历。  
源代码如下:

```
1.      struct linka {
2.          int data;
3.          linka* next;
4.      };
5.      void reverse(linka*& head) {
```

```

6.      if(head ==NULL)
7.          return;
8.      linka *pre, *cur, *ne;
9.      pre=head;
10.     cur=head->next;
11.     while(cur)
12.     {
13.         ne = cur->next;
14.         cur->next = pre;
15.         pre = cur;
16.         cur = ne;
17.     }
18.     head->next = NULL;
19.     head = pre;
20. }

```

还有一种利用递归的方法。这种方法的基本思想是在反转当前节点之前先调用递归函数反转后续节点。源代码如下。不过这个方法有一个缺点，就是在反转后的最后一个结点会形成一个环，所以必须将函数的返回的节点的 `next` 域置为 `NULL`。因为要改变 `head` 指针，所以我用了引用。算法的源代码如下：

```

1.      linka* reverse(linka* p,linka*& head)
2.      {
3.          if(p == NULL || p->next == NULL)
4.          {
5.              head=p;
6.              return p;
7.          }
8.          else
9.          {
10.             linka* tmp = reverse(p->next,head);
11.             tmp->next = p;

```

```
12.         return p;
13.     }
14. }
```

**32.已知 String 类定义如下:**

```
class String
{
public:
String(const char *str = NULL); // 通用构造函数
String(const String &another); // 拷贝构造函数
~ String(); // 析构函数
String & operator =(const String &rhs); // 赋值函数
private:
char *m_data; // 用于保存字符串
};
```

尝试写出类的成员函数实现。

答案:

```
String::String(const char *str)
{
if ( str == NULL ) //strlen 在参数为 NULL 时会抛异常才会有这步判断
{
m_data = new char[1] ;
m_data[0] = '\0' ;
}
else
{
m_data = new char[strlen(str) + 1];
strcpy(m_data,str);
}

}
```

```
String::String(const String &another)
{
m_data = new char[strlen(another.m_data) + 1];
strcpy(m_data,other.m_data);
}
```

```
String& String::operator =(const String &rhs)
{
    if ( this == &rhs)
        return *this ;
    delete []m_data; //删除原来的数据，新开一块内存
    m_data = new char[strlen(rhs.m_data) + 1];
    strcpy(m_data,rhs.m_data);
    return *this ;
}
```

```
String::~~String()
{
    delete []m_data ;
}
```

**33.求下面函数的返回值（微软）**

```
int func(x)
{
    int countx = 0;
    while(x)
    {
        countx ++;
        x = x&(x-1);
    }
    return countx;
}
```

假定  $x = 9999$ 。 答案： 8

思路：将  $x$  转化为 2 进制，看含有的 1 的个数。

#### 34. 什么是“引用”？申明和使用“引用”要注意哪些问题？

答：引用就是某个目标变量的“别名”(alias)，对应用的操作与对变量直接操作效果完全相同。申明一个引用的时候，切记要对其进行初始化。引用声明完毕后，相当于目标变量名有两个名称，即该目标原名称和引用名，不能再把该引用名作为其他变量名的别名。声明一个引用，不是新定义了一个变量，它只表示该引用名是目标变量名的一个别名，它本身不是一种数据类型，因此引用本身不占存储单元，系统也不给引用分配存储单元。不能建立数组的引用。

#### 45. 将“引用”作为函数参数有哪些特点？

(1) 传递引用给函数与传递指针的效果是一样的。这时，被调函数的形参就成为原来主调函数中的实参变量或对象的一个别名来使用，所以在被调函数中对形参变量的操作就是对其相应的目标对象（在主调函数中）的操作。

(2) 使用引用传递函数的参数，在内存中并没有产生实参的副本，它是直接对实参操作；而使用一般变量传递函数的参数，当发生函数调用时，需要给形参分配存储单元，形参变量是实参变量的副本；如果传递的是对象，还将调用拷贝构造函数。因此，当参数传递的数据较大时，用引用比用一般变量传递参数的效率和所占空间都好。

(3) 使用指针作为函数的参数虽然也能达到与使用引用的效果，但是，在被调函数中同样要给形参分配存储单元，且需要重复使用“\*指针变量名”的形式进行运算，这很容易产生错误且程序的阅读性较差；另一方面，在主调函数的调用点处，必须用变量的地址作为实参。而引用更容易使用，更清晰。

#### 36. 在什么时候需要使用“常引用”？

如果既要利用引用提高程序的效率，又要保护传递给函数的数据不在函数中被改变，就应使用常引用。常引用声明方式：**const** 类型标识符 &引用名=目标变量名；

例 1

```
int a ;  
  
const int &ra=a;  
  
ra=1; //错误  
  
a=1; //正确
```

例 2

```
string foo( );  
  
void bar(string & s);
```

那么下面的表达式将是非法的：

```
bar(foo( ));  
  
bar("hello world");
```

原因在于 `foo( )` 和 `"hello world"` 串都会产生一个临时对象，而在 C++ 中，这些临时对象都是 `const` 类型的。因此上面的表达式就是试图将一个 `const` 类型的对象转换为非 `const` 类型，这是非法的。

引用型参数应该在能被定义为 `const` 的情况下，尽量定义为 `const` 。

### 37. 将“引用”作为函数返回值类型的格式、好处和需要遵守的规则？

格式：类型标识符 &函数名（形参列表及类型说明）{ //函数体 }

好处：在内存中不产生被返回值的副本；（注意：正是因为这点原因，所以返回一个局部变量的引用是不可取的。因为随着该局部变量生存期的结束，相应的引用也会失效，产生 `run time error`！

注意事项:

(1) 不能返回局部变量的引用。这条可以参照 **Effective C++[1]**的 Item 31。主要原因是局部变量会在函数返回后被销毁，因此被返回的引用就成为了"无所指"的引用，程序会进入未知状态。

(2) 不能返回函数内部 **new** 分配的内存的引用。这条可以参照 **Effective C++[1]**的 Item 31。虽然不存在局部变量的被动销毁问题，可对于这种情况（返回函数内部 **new** 分配内存的引用），又面临其它尴尬局面。例如，被函数返回的引用只是作为一个临时变量出现，而没有被赋予一个实际的变量，那么这个引用所指向的空间（由 **new** 分配）就无法释放，造成 **memory leak**。

(3) 可以返回类成员的引用，但最好是 **const**。这条原则可以参照 **Effective C++[1]**的 Item 30。主要原因是当对象的属性是与某种业务规则（**business rule**）相关联的时候，其赋值常常与某些其它属性或者对象的状态有关，因此有必要将赋值操作封装在一个业务规则当中。如果其它对象可以获得该属性的非常量引用（或指针），那么对该属性的单纯赋值就会破坏业务规则的完整性。

(4) 流操作符重载返回值申明为“引用”的作用：

流操作符<<和>>，这两个操作符常常希望被连续使用，例如：`cout << "hello" << endl;`因此这两个操作符的返回值应该是一个仍然支持这两个操作符的流引用。可选的其它方案包括：返回一个流对象和返回一个流对象指针。但是对于返回一个流对象，程序必须重新（拷贝）构造一个新的流对象，也就是说，连续的两个<<操作符实际上是针对不同对象的！这无法让人接受。对于返回一个流指针则不能连续使用<<操作符。因此，返回一个流对象引用是惟一选择。这个唯一选择很关键，它说明了引用的重要性以及无可替代性，也许这就是C++语言中引入引用这个概念的原因吧。赋值操作符=。这个操作符象流操作符一样，是可以连续使用的，例如：`x = j = 10;`或者`(x=10)=100;`赋值操作符的返回值必须是一个左值，以便可以被继续赋值。因此引用成了这个操作符的惟一返回值选择。

### 例 3

```
#include <iostream.h>

int &put(int n);

int vals[10];

int error=-1;

void main()
{
    put(0)=10; //以 put(0)函数值作为左值，等价于 vals[0]=10;
    put(9)=20; //以 put(9)函数值作为左值，等价于 vals[9]=20;
    cout<<vals[0];
    cout<<vals[9];
}

int &put(int n)
{
    if (n>=0 && n<=9 ) return vals[n];
    else { cout<<"subscript error"; return error; }
}
```

(5) 在另外的一些操作符中，却千万不能返回引用：**+-\*/** 四则运算符。它们不能返回引用，**Effective C++[1]**的 **Item23** 详细的讨论了这个问题。主要原因是这四个操作符没有 **side effect**，因此，它们必须构造一个对象作为返回值，可选的方案包括：返回一个对象、返回一个局部变量的引用，返回一个 **new** 分配的对象的引用、返回一个静态对象引用。根据前面提到的引用作为返回值的三个规则，第 2、3 两个方案都被否决了。静态对象的引用又因为 **((a+b) == (c+d))** 会永远为 **true** 而导致错误。所以可选的只剩下返回一个对象了。

### 38. “引用”与多态的关系？

引用是除指针外另一个可以产生多态效果的手段。这意味着，一个基类的引用可以指向它的

派生类实例。

#### 例 4

```
Class A; Class B : Class A{...}; B b; A& ref = b;
```

### 39. “引用”与指针的区别是什么？

指针通过某个指针变量指向一个对象后，对它所指向的变量间接操作。程序中使用指针，程序的可读性差；而引用本身就是目标变量的别名，对引用的操作就是对目标变量的操作。此外，就是上面提到的对函数传 `ref` 和 `pointer` 的区别。

### 40. 什么时候需要“引用”？

流操作符<<和>>、赋值操作符=的返回值、拷贝构造函数的参数、赋值操作符=的参数、其它情况都推荐使用引用。

以上 2-8 参考：<http://blog.csdn.net/wfwd/archive/2006/05/30/763551.aspx>

### 41. 结构与联合有和区别？

1. 结构和联合都是由多个不同的数据类型成员组成，但在任何同一时刻，联合中只存放了一个被选中的成员（所有成员共用一块地址空间），而结构的所有成员都存在（不同成员的存放地址不同）。
2. 对于联合的不同成员赋值，将会对其它成员重写，原来成员的值就不存在了，而对于结构的成员赋值是互不影响的。

### 42. 下面关于“联合”的题目的输出？

a)

```
#include <stdio.h>
```

```
union
```

```
{
```

```
int i;
```

```
char x[2];
```

```
}a;
```

```
void main()
```

```
{
```

```
a.x[0] = 10;
```

```
a.x[1] = 1;
```

```
printf("%d",a.i);
```

```
}
```

答案：266（低位低地址，高位高地址，内存占用情况是 0x010A）

b)

```
main()
```

```
{
```

```
union{ /*定义一个联合*/
```

```
int i;
```

```
struct{ /*在联合中定义一个结构*/
```

```
char first;
```

```
char second;
```

```
}half;
```

```
}number;
```

```
number.i=0x4241; /*联合成员赋值*/
```

```
printf("%c%c\n", number.half.first, number.half.second);
```

```
number.half.first='a'; /*联合中结构成员赋值*/
```

```

number.half.second='b';

printf("%x\n", number.i);

getch();

}

```

答案： AB (0x41 对应'A',是低位； 0x42 对应'B',是高位)

6261 (number.i 和 number.half 共用一块地址空间)

**43. 已知 strcpy 的函数原型: char \*strcpy(char \*strDest, const char \*strSrc)其中 strDest 是目的字符串, strSrc 是源字符串。不调用 C++/C 的字符串库函数, 请编写函数 strcpy。**

答案:

```

char *strcpy(char *strDest, const char *strSrc)
{
    if ( strDest == NULL || strSrc == NULL)
        return NULL ;
    if ( strDest == strSrc)
        return strDest ;
    char *tempPtr = strDest ;
    while( (*strDest++ = *strSrc++) != '\0')
        ;
    return tempPtr ;
}

```

**44. .h 头文件中的 ifndef/define/endif 的作用?**

答: 防止该头文件被重复引用。

**45. #include<file.h> 与 #include "file.h"的区别?**

答：前者是从 **Standard Library** 的路径寻找和引用 **file.h**，而后者是从当前工作路径搜寻并引用 **file.h**。

#### 46.在 C++ 程序中调用被 C 编译器编译后的函数，为什么要加 **extern "C"**？

首先，作为 **extern** 是 C/C++ 语言中表明函数和全局变量作用范围（可见性）的关键字，该关键字告诉编译器，其声明的函数和变量可以在本模块或其它模块中使用。

通常，在模块的头文件对本模块提供给其它模块引用的函数和全局变量以关键字 **extern** 声明。例如，如果模块 **B** 欲引用该模块 **A** 中定义的全局变量和函数时只需包含模块 **A** 的头文件即可。这样，模块 **B** 中调用模块 **A** 中的函数时，在编译阶段，模块 **B** 虽然找不到该函数，但是并不会报错；它会在连接阶段中从模块 **A** 编译生成的目标代码中找到此函数

**extern "C"**是连接申明(linkage declaration),被 **extern "C"**修饰的变量和函数是按照 C 语言方式编译和连接的,来看看 C++中对类似 C 的函数是怎样编译的：

作为一种面向对象的语言，C++支持函数重载，而过程式语言 C 则不支持。函数被 C++编译后在符号库中的名字与 C 语言的不同。例如，假设某个函数的原型为：

```
void foo( int x, int y );
```

该函数被 C 编译器编译后在符号库中的名字为 **\_foo**，而 C++编译器则会产生像 **\_foo\_int\_int** 之类的名字（不同的编译器可能生成的名字不同，但是都采用了相同的机制，生成的新名字称为“**mangled name**”）。

**\_foo\_int\_int** 这样的名字包含了函数名、函数参数数量及类型信息，C++就是靠这种机制来实现函数重载的。例如，在 C++中，函数 **void foo( int x, int y )**与 **void foo( int x, float y )**编译生成的符号是不相同的，后者为 **\_foo\_int\_float**。

同样地，C++中的变量除支持局部变量外，还支持类成员变量和全局变量。用户所编写程序的类成员变量可能与全局变量同名，我们以"."来区分。而本质上，编译器在进行编译时，与函数的处理相似，也为类中的变量取了一个独一无二的名字，这个名字与用户程序中同名的全局变量名字不同。

未加 `extern "C"`声明时的连接方式

假设在 C++中，模块 A 的头文件如下：

```
// 模块 A 头文件 moduleA.h

#ifndef MODULE_A_H
#define MODULE_A_H

int foo( int x, int y );

#endif
```

在模块 B 中引用该函数：

```
// 模块 B 实现文件 moduleB.cpp

#include "moduleA.h"

foo(2,3);
```

实际上，在连接阶段，连接器会从模块 A 生成的目标文件 `moduleA.obj` 中寻找 `_foo_int_int` 这样的符号！

加 `extern "C"`声明后的编译和连接方式

加 `extern "C"`声明后，模块 A 的头文件变为：

```
// 模块 A 头文件 moduleA.h

#ifndef MODULE_A_H
#define MODULE_A_H

extern "C" int foo( int x, int y );

#endif
```

在模块 B 的实现文件中仍然调用 `foo( 2,3 )`，其结果是：

(1) 模块 A 编译生成 `foo` 的目标代码时，没有对其名字进行特殊处理，采用了 C 语言的方式：

(2) 连接器在为模块 B 的目标代码寻找 `foo(2,3)`调用时，寻找的是未经修改的符号名 `_foo`。

如果在模块 A 中函数声明了 `foo` 为 `extern "C"`类型，而模块 B 中包含的是 `extern int foo( int x, int y )`，则模块 B 找不到模块 A 中的函数；反之亦然。

所以，可以用一句话概括 `extern "C"`这个声明的真实目的（任何语言中的任何语法特性的诞生都不是随意而为的，来源于真实世界的需求驱动。我们在思考问题时，不能只停留在这个语言是怎么做的，还要问一问它为什么要这么做，动机是什么，这样我们可以更深入地理解许多问题）：实现 C++与 C 及其它语言的混合编程。

明白了 C++中 `extern "C"`的设立动机，我们下面来具体分析 `extern "C"`通常的使用技巧：

`extern "C"`的惯用法

(1) 在 C++中引用 C 语言中的函数和变量，在包含 C 语言头文件（假设为 `cExample.h`）时，需进行下列处理：

```
extern "C"
{
#include "cExample.h"
```

```
}
```

而在 C 语言的头文件中，对其外部函数只能指定为 **extern** 类型，C 语言中不支持 **extern "C"** 声明，在.c 文件中包含了 **extern "C"**时会出现编译语法错误。

C++引用 C 函数例子工程中包含的三个文件的源代码如下：

```
/* c 语言头文件：cExample.h */
```

```
#ifndef C_EXAMPLE_H
```

```
#define C_EXAMPLE_H
```

```
extern int add(int x,int y);
```

```
#endif
```

```
/* c 语言实现文件：cExample.c */
```

```
#include "cExample.h"
```

```
int add( int x, int y )
```

```
{
```

```
return x + y;
```

```
}
```

```
// c++实现文件，调用 add: cppFile.cpp
```

```
extern "C"
```

```
{
```

```
#include "cExample.h"
```

```
}
```

```
int main(int argc, char* argv[])
```

```
{
```

```
add(2,3);
```

```
return 0;
}
```

如果 C++调用一个 C 语言编写的.DLL 时，当包括.DLL 的头文件或声明接口函数时，应加 `extern "C" { }`。

(2) 在 C 中引用 C++语言中的函数和变量时，C++的头文件需添加 `extern "C"`，但是在 C 语言中不能直接引用声明了 `extern "C"`的该头文件，应该仅将 C 文件中将 C++中定义的 `extern "C"`函数声明为 `extern` 类型。

C 引用 C++函数例子工程中包含的三个文件的源代码如下：

```
//C++头文件 cppExample.h
#ifndef CPP_EXAMPLE_H
#define CPP_EXAMPLE_H
extern "C" int add( int x, int y );
#endif
```

```
//C++实现文件 cppExample.cpp
#include "cppExample.h"
int add( int x, int y )
{
    return x + y;
}
```

```
/* C 实现文件 cFile.c
/* 这样会编译出错： #include "cExample.h" */
extern int add( int x, int y );
```

```
int main( int argc, char* argv[] )  
{  
    add( 2, 3 );  
    return 0;  
}
```

**15 题目的解答请参考《C++中 extern “C”含义深层探索》注解：**

#### **47. 关联、聚合(Aggregation)以及组合(Composition)的区别？**

涉及到 UML 中的一些概念：关联是表示两个类的一般性联系，比如“学生”和“老师”就是一种关联关系；聚合表示 has-a 的关系，是一种相对松散的关系，聚合类不需要对被聚合类负责，如下图所示，用空的菱形表示聚合关系：

从实现的角度讲，聚合可以表示为：

```
class A {...} class B { A* a; .....}
```

而组合表示 contains-a 的关系，关联性强于聚合：组合类与被组合类有相同的生命周期，组合类要对被组合类负责，采用实心的菱形表示组合关系：

实现的形式是：

```
class A{...} class B{ A a; ...}
```

参考文章：<http://blog.csdn.net/wfwd/archive/2006/05/30/763753.aspx>

<http://blog.csdn.net/wfwd/archive/2006/05/30/763760.aspx>

#### 48.面向对象三个基本特征，并简单叙述之？

1. 封装：将客观事物抽象成类，每个类对自身的数据和方法实行 `protection(private, protected, public)`
2. 继承：广义的继承有三种实现形式：实现继承（指使用基类的属性和方法而无需额外编码的能力）、可视继承（子窗体使用父窗体的外观和实现代码）、接口继承（仅使用属性和方法，实现滞后到子类实现）。前两种（类继承）和后一种（对象组合=>接口继承以及纯虚函数）构成了功能复用的两种方式。
3. 多态：是将父对象设置成为和一个或更多的他的子对象相等的技术，赋值之后，父对象就可以根据当前赋值给它的子对象的特性以不同的方式运作。简单的说，就是一句话：允许将子类类型的指针赋值给父类类型的指针。

#### 49. 重载 (overload)和重写(overried, 有的书也叫做“覆盖”)的区别？

常考的题目。从定义上来说：

重载：是指允许存在多个同名函数，而这些函数的参数表不同（或许参数个数不同，或许参数类型不同，或许两者都不同）。

重写：是指子类重新定义复类虚函数的方法。

从实现原理上来说：

重载：编译器根据函数不同的参数表，对同名函数的名称做修饰，然后这些同名函数就成了不同的函数（至少对于编译器来说是这样的）。如，有两个同名函数：`function func(p:inte`

ger):integer;和 `function func(p:string):integer;`。那么编译器做过修饰后的函数名称可能是这样的：`int_func`、`str_func`。对于这两个函数的调用，在编译器间就已经确定了，是静态的。也就是说，它们的地址在编译期就绑定了（早绑定），因此，重载和多态无关！

重写：和多态真正相关。当子类重新定义了父类的虚函数后，父类指针根据赋给它的不同的子类指针，动态的调用属于子类的该函数，这样的函数调用在编译期间是无法确定的（调用的子类的虚函数的地址无法给出）。因此，这样的函数地址是在运行期绑定的（晚绑定）。

## 50. 多态的作用？

主要是两个：1. 隐藏实现细节，使得代码能够模块化；扩展代码模块，实现代码重用；2.

接口重用：为了类在继承和派生的时候，保证使用家族中任一类的实例的某一属性时的正确调用。

## 51. New delete 与 malloc free 的联系与区别？

答案：都是在堆(heap)上进行动态的内存操作。用 `malloc` 函数需要指定内存分配的字节数并且不能初始化对象，`new` 会自动调用对象的构造函数。`delete` 会调用对象的 `destructor`，而 `free` 不会调用对象的 `destructor`。

## 52. 有哪几种情况只能用 initialization list 而不能用 assignment？

答案：当类中含有 `const`、`reference` 成员变量；基类的构造函数都需要初始化表。

## 53. C++是不是类型安全的？

答案：不是。两个不同类型的指针之间可以强制转换（用 `reinterpret cast`）。C#是类型安全的。

## 54. main 函数执行以前，还会执行什么代码？

答案：全局对象的构造函数会在 `main` 函数之前执行。

## 55. 描述内存分配方式以及它们的区别？

- 1) 从静态存储区域分配。内存在程序编译的时候就已经分配好，这块内存存在程序的整个运行期间都存在。例如全局变量，`static` 变量。
- 2) 在栈上创建。在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集。
- 3) 从堆上分配，亦称动态内存分配。程序在运行的时候用 `malloc` 或 `new` 申请任意多少的内存，程序员自己负责在何时用 `free` 或 `delete` 释放内存。动态内存的生存期由程序员决定，使用非常灵活，但问题也最多。

## 56.struct 和 class 的区别

答案：`struct` 的成员默认是公有的，而类的成员默认是私有的。`struct` 和 `class` 在其他方面是功能相当的。

从感情上讲，大多数的开发者感到类和结构有很大的差别。感觉上结构仅仅象一堆缺乏封装和功能的开放的内存位，而类就象活的并且可靠的社会成员，它有智能服务，有牢固的封装屏障和一个良好定义的接口。既然大多数人都这么认为，那么只有在你的类有很少的方法并且有公有数据（这种事情在良好设计的系统中是存在的!）时，你也许应该使用 `struct` 关键字，否则，你应该使用 `class` 关键字。

## 57. 当一个类 A 中没有生命任何成员变量与成员函数,这时 sizeof(A)的值是多少,如果不是零,请解释一下编译器为什么没有让它为零。（Autodesk）

答案：肯定不是零。举个反例，如果是零的话，声明一个 `class A[10]` 对象数组，而每一个对象占用的空间是零，这时就没办法区分 `A[0]`, `A[1]`... 了。

## 58. 在 8086 汇编下，逻辑地址和物理地址是怎样转换的？（Intel）

答案：通用寄存器给出的地址，是段内偏移地址，相应段寄存器地址\*10H+通用寄存器内地址，就得到了真正要访问的地址。

## 59. 比较 C++ 中的 4 种类型转换方式？

请参考：<http://blog.csdn.net/wfwd/archive/2006/05/30/763785.aspx>，重点是 `static_cast`, `dynamic_cast` 和 `reinterpret_cast` 的区别和应用。

**60.分别写出 `BOOL`,`int`,`float`,指针类型的变量 `a` 与“零”的比较语句。**

答案：

`BOOL` : `if ( !a ) or if(a)`

`int` : `if ( a == 0)`

`float` : `const EXPRESSION EXP = 0.000001`

`if ( a < EXP && a >-EXP)`

`pointer` : `if ( a != NULL) or if(a == NULL)`

**61.请说出 `const` 与 `#define` 相比，有何优点？**

答案：1) `const` 常量有数据类型，而宏常量没有数据类型。编译器可以对前者进行类型安全检查。而对后者只进行字符替换，没有类型安全检查，并且在字符替换可能会产生意料不到的错误。

2) 有些集成化的调试工具可以对 `const` 常量进行调试，但是不能对宏常量进行调试。

**62.简述数组与指针的区别？**

数组要么在静态存储区被创建（如全局数组），要么在栈上被创建。指针可以随时指向任意类型的内存块。

(1)修改内容上的差别

```
char a[] = "hello";
```

```
a[0] = 'X';
```

```
char *p = "world"; // 注意 p 指向常量字符串
```

```
p[0] = 'X'; // 编译器不能发现该错误，运行时错误
```

(2) 用运算符 `sizeof` 可以计算出数组的容量（字节数）。`sizeof(p)`,`p` 为指针得到的是一个指针变量的字节数，而不是 `p` 所指的内存容量。`C++/C` 语言没有办法知道指针所指的内存

容量，除非在申请内存时记住它。注意当数组作为函数的参数进行传递时，该数组自动退化为同类型的指针。

```
char a[] = "hello world";  
  
char *p = a;  
  
cout<< sizeof(a) << endl; // 12 字节  
cout<< sizeof(p) << endl; // 4 字节  
  
计算数组和指针的内存容量  
  
void Func(char a[100])  
{  
    cout<< sizeof(a) << endl; // 4 字节而不是 100 字节  
}
```

### 63.类成员函数的重载、覆盖和隐藏区别？

答案：

a.成员函数被重载的特征：

- (1) 相同的范围（在同一个类中）；
- (2) 函数名字相同；
- (3) 参数不同；
- (4) **virtual** 关键字可有可无。

b.覆盖是指派生类函数覆盖基类函数，特征是：

- (1) 不同的范围（分别位于派生类与基类）；
- (2) 函数名字相同；
- (3) 参数相同；
- (4) 基类函数必须有 **virtual** 关键字。

c.“隐藏”是指派生类的函数屏蔽了与其同名的基类函数，规则如下：

- (1) 如果派生类的函数与基类的函数同名，但是参数不同。此时，不论有无 **virtual** 关键字，基类的函数将被隐藏（注意别与重载混淆）。
- (2) 如果派生类的函数与基类的函数同名，并且参数也相同，但是基类函数没有 **virtual** 关键字。此时，基类的函数被隐藏（注意别与覆盖混淆）。

**64. 如何打印出当前源文件的文件名以及源文件的当前行号？**

答案：

```
cout << __FILE__ ;
```

```
cout<<__LINE__ ;
```

\_\_FILE\_\_和\_\_LINE\_\_是系统预定义宏，这种宏并不是在某个文件中定义的，而是由编译器定义的。

**65. main 主函数执行完毕后，是否可能会再执行一段代码，给出说明？**

答案：可以，可以用\_onexit 注册一个函数，它会在 main 之后执行 int fn1(void), fn2(void), fn3(void), fn4 (void);

```
void main( void )
```

```
{
```

```
String str("zhanglin");
```

```
_onexit( fn1 );
```

```
_onexit( fn2 );
```

```
_onexit( fn3 );
```

```
_onexit( fn4 );
```

```
printf( "This is executed first.\n" );
```

```
}
```

```
int fn1()
```

```
{
```

```
printf( "next.\n" );
```

```
return 0;
```

```
}
```

```
int fn2()
```

```
{
```

```
printf( "executed " );
```

```
return 0;
```

```
}
```

```
int fn3()
```

```

{
printf( "is " );
return 0;
}

int fn4()
{
printf( "This " );
return 0;
}

```

The `_onexit` function is passed the address of a function (func) to be called when the program terminates normally. Successive calls to `_onexit` create a register of functions that are executed in LIFO (last-in-first-out) order. The functions passed to `_onexit` cannot take parameters.

**66. 如何判断一段程序是由 C 编译程序还是由 C++编译程序编译的？**

答案：

```

#ifdef __cplusplus
cout<<"c++";
#else
cout<<"c";
#endif

```

**67.文件中有一组整数，要求排序后输出到另一个文件中**

答案：

```

#include<iostream>

#include<fstream>

using namespace std;

```

```

void Order(vector<int>& data) //bubble sort
{
    int count = data.size() ;
    int tag = false ; // 设置是否需要继续冒泡的标志位
    for ( int i = 0 ; i < count ; i++)
    {
        for ( int j = 0 ; j < count - i - 1 ; j++)
        {
            if ( data[j] > data[j+1])
            {
                tag = true ;
                int temp = data[j] ;
                data[j] = data[j+1] ;
                data[j+1] = temp ;
            }
        }
        if ( !tag )
            break ;
    }
}

```

```

void main( void )
{
    vector<int>data;
    ifstream in("c:\\data.txt");
    if ( !in)
    {

```

```

cout<<"file error!";

exit(1);

}

int temp;

while (!in.eof())

{

in>>temp;

data.push_back(temp);

}

in.close(); //关闭输入文件流

Order(data);

ofstream out("c:\\result.txt");

if ( !out)

{

cout<<"file error!";

exit(1);

}

for ( i = 0 ; i < data.size() ; i++)

out<<data[i]<<" ";

out.close(); //关闭输出文件流

}

```

#### 68. 链表题：一个链表的结点结构

```

struct Node

{

int data ;

Node *next ;

};

```

```
typedef struct Node Node ;
```

(1)已知链表的头结点 head,写一个函数把这个链表逆序 ( Intel)

```
Node * ReverseList(Node *head) //链表逆序
{
    if ( head == NULL || head->next == NULL )
        return head;

    Node *p1 = head ;
    Node *p2 = p1->next ;
    Node *p3 = p2->next ;
    p1->next = NULL ;
    while ( p3 != NULL )
    {
        p2->next = p1 ;
        p1 = p2 ;
        p2 = p3 ;
        p3 = p3->next ;
    }
    p2->next = p1 ;
    head = p2 ;
    return head ;
}
```

(2)已知两个链表 head1 和 head2 各自有序，请把它们合并成一个链表依然有序。(保留所有结点，即便大小相同)

```
Node * Merge(Node *head1 , Node *head2)
{
    if ( head1 == NULL)
        return head2 ;
```

```

if ( head2 == NULL)
return head1 ;
Node *head = NULL ;
Node *p1 = NULL;
Node *p2 = NULL;
if ( head1->data < head2->data )
{
head = head1 ;
p1 = head1->next;
p2 = head2 ;
}
else
{
head = head2 ;
p2 = head2->next ;
p1 = head1 ;
}
Node *pcurrent = head ;
while ( p1 != NULL && p2 != NULL)
{
if ( p1->data <= p2->data )
{
pcurrent->next = p1 ;
pcurrent = p1 ;
p1 = p1->next ;
}
else
{
pcurrent->next = p2 ;
pcurrent = p2 ;

```

```

p2 = p2->next ;
}
}
if ( p1 != NULL )
pcurrent->next = p1 ;
if ( p2 != NULL )
pcurrent->next = p2 ;
return head ;
}

```

(3)已知两个链表 head1 和 head2 各自有序，请把它们合并成一个链表依然有序，这次要求用递归方法进行。 (Autodesk)

答案：

```

Node * MergeRecursive(Node *head1 , Node *head2)
{
if ( head1 == NULL )
return head2 ;
if ( head2 == NULL )
return head1 ;
Node *head = NULL ;
if ( head1->data < head2->data )
{
head = head1 ;
head->next = MergeRecursive(head1->next,head2);
}
else
{
head = head2 ;
head->next = MergeRecursive(head1,head2->next);
}
return head ;
}

```

```
}
```

**69. 分析一下这段程序的输出 (Autodesk)**

```
class B
```

```
{
```

```
public:
```

```
B()
```

```
{
```

```
cout<<"default constructor"<<endl;
```

```
}
```

```
~B()
```

```
{
```

```
cout<<"destructed"<<endl;
```

```
}
```

```
B(int i):data(i) //B(int) works as a converter ( int -> instance of B)
```

```
{
```

```
cout<<"constructed by parameter " << data <<endl;
```

```
}
```

```
private:
```

```
int data;
```

```
};
```

```
B Play( B b)
```

```
{
```

```
return b ;
```

```
}
```

(1) results:

```
int main(int argc, char* argv[]) constructed by parameter 5
```

```

{ destructed B(5)形参析构
B t1 = Play(5); B t2 = Play(t1);    destructed t1 形参析构
return 0;                            destructed t2 注意顺序!
} destructed t1

```

(2) results:

```

int main(int argc, char* argv[]) constructed by parameter 5
{ destructed B(5)形参析构
B t1 = Play(5); B t2 = Play(10);    constructed by parameter 10
return 0;                            destructed B(10)形参析构
} destructed t2 注意顺序!

destructed t1

```

## 70. 写一个函数找出一个整数数组中，第二大的数（microsoft）

答案：

```

const int MINNUMBER = -32767 ;
int find_sec_max( int data[] , int count)
{
    int maxnumber = data[0] ;
    int sec_max = MINNUMBER ;
    for ( int i = 1 ; i < count ; i++)
    {
        if ( data[i] > maxnumber )
        {
            sec_max = maxnumber ;
            maxnumber = data[i] ;
        }
        else
        {

```

```

if ( data[i] > sec_max )
sec_max = data[i] ;
}
}
return sec_max ;
}

```

**71. 写一个在一个字符串(n)中寻找一个子串(m)第一个位置的函数。**

KMP 算法效率最好，时间复杂度是  $O(n+m)$ 。

**72. 多重继承的内存分配问题：**

比如有 `class A : public class B, public class C {}`

那么 A 的内存结构大致是怎么样的？

这个是 **compiler-dependent** 的，不同的实现其细节可能不同。

如果不考虑有虚函数、虚继承的话就相当简单；否则的话，相当复杂。

可以参考《深入探索 C++对象模型》，或者：

<http://blog.csdn.net/wfwd/archive/2006/05/30/763797.aspx>

**73. 如何判断一个单链表是有环的？（注意不能用标志位，最多只能用两个额外指针）**

```

struct node { char val; node* next;}

```

```

bool check(const node* head) {} //return false : 无环; true: 有环

```

一种  $O(n)$  的办法就是（搞两个指针，一个每次递增一步，一个每次递增两步，如果有环的话两者必然重合，反之亦然）：

```

bool check(const node* head)
{

```

```

if(head==NULL) return false;

node *low=head, *fast=head->next;

while(fast!=NULL && fast->next!=NULL)

{

low=low->next;

fast=fast->next->next;

if(low==fast) return true;

}

return false;

}

```

74.不使用库函数，编写函数 int strcmp(char \*source, char \*dest)相等返回 0，不等返回-1

```

int StrCmp(char *source, char *dest)
{
    assert(source !=NULL) ;
    assert(dest!=NULL) ;
    while(*source==*dest&&*source&&*dest)
    {
        source++;
        dest++;
    }
    return (*source!=*dest)?-1:0;
}

```

75.写一个函数，实现将一个字符串中的'\t'替换成四个'\*'， '\t'个数不定。如 char \*p="ht\t\tdsf\t\t\t\tfdsfdfsw\tew\t",替换后 p="ht\*\*\*\*hdsf\*\*\*\*\*tt\*\*\*\*\*fdsdfsw\*\*\*\*ew\*\*\*\*\*"。

```

char *Replace(char *Sorce)
{
    char *pTemp=Sorce;
    int iCount=0;
    int iSizeOfSorce=0;

    while(*pTemp!='\0')
    {
        if('\t'==*pTemp)

```

```

        iCount++;
        pTemp++;
    }
    iSizeOfSorce=pTemp-Sorce;

    char *pNewStr=new char[iSizeOfSorce+3*iCount*sizeof(char)+1];
    char *pTempNewStr=pNewStr;

    pTemp=Sorce;
    while(*pTemp!='\0')
    {
        if('\t'==*pTemp)
        {
            for(int iLoop=0; iLoop<4; iLoop++)
            {
                *pTempNewStr='*';
                pTempNewStr++;
            }
            pTemp++;
        }
        else
        {
            *pTempNewStr=*pTemp;
            pTemp++;
            pTempNewStr++;
        }
    }
    *pTempNewStr='\0';

    return pNewStr;
}

```

**76. 写一函数实现将一个字符串中的数字字符全部去掉。**

```

void RemoveNum(char strSrc[])
{
    char *p=strSrc;
    char *q;
    while(*p!='\0')
    {
        if(*p>='0'&&*p<='9')

```

```

    {
        q=p;
        while(*q!='\0')
        {
            *q=*(q+1);
            q++;
        }
    }
    else
    {
        p++;
    }
}
}

```

77、链表节点结构如下：

```

struct STUDENT
{
    long num;
    float score;
    STUDENT *pNext;
};

```

编写实现将两棵有序（按学号从小到大）的链表合并的函数，要求合并后的链表有序（按学号从小到大）

```

STUDENT *EmergeList(STUDENT *pHead1,STUDENT *pHead2)
{
    //取小者为表头
    STUDENT * pHead;
    STUDENT *p1;
    STUDENT *p2;
    STUDENT *pCur;
    STUDENT *pTemp;

    if (pHead1->num<= pHead2->num)
    {
        pHead = pHead1;
        p2 = pHead2;
        p1=pHead1->pNext;
    }
}

```

```

else
{
    pHead = pHead2;
    p1 = pHead1;
    p2=pHead2->pNext;
}
pCur=pHead;
while(p1!=NULL&& p2!=NULL)
{
    if(p2->num<p1->num)
    {
        pCur->pNext=p2;
        p2=p2->pNext;
        pCur=pCur->pNext;
    }
    else if(p2->num>p1->num)
    {
        pCur->pNext=p1;
        p1=p1->pNext;
        pCur=pCur->pNext;
    }
    else if(p2->num==p1->num)
    {
        pCur->pNext=p1;
        p1=p1->pNext;
        pCur=pCur->pNext;
        pTemp= p2;
        p2=p2->pNext;
        delete pTemp;
        pTemp = NULL;
    }
}
if(NULL==p1)
{
    pCur->pNext=p2;
}
else if(NULL==p2)
{
    pCur->pNext=p1;
}

return pHead;
}

```

78、封装 CMyString 类，要求声明和实现分开，声明见 MyString.h，该类的声明可以添加内容，完成 STLTest 文件夹下 main 文件的要求。

参见 STLTest Answer 文件夹

79.请你分别画出 OSI 的七层网络结构图和 TCP/IP 的五层结构图。

80.请你详细地解释一下 IP 协议的定义，在哪个层上面？主要有什么作用？TCP 与 UDP 呢？

81.请问交换机和路由器各自的实现原理是什么？分别在哪个层次上面实现的？

82.请问 C++ 的类和 C 里面的 struct 有什么区别？

83.全局变量和局部变量有什么区别？是怎么实现的？操作系统和编译器是怎么知道的？

84. 非 C++ 内建型别 A 和 B，在哪几种情况下 B 能隐式转化为 A？[C++ 中等]

a. class B : public A { ..... } // B 公有继承自 A，可以是间接继承的

b. class B { operator A( ); } // B 实现了隐式转化为 A 的转化

c. class A { A( const B& ); } // A 实现了 non-explicit 的参数为 B（可以有其他带默认值的参数）构造函数

d. A& operator= ( const A& ); // 赋值操作，虽不是正宗的隐式类型转换，但也可以勉强算一个

85. 以下代码中的两个 sizeof 用法有问题吗？[C 易]

void UpperCase( char str[] ) // 将 str 中的小写字母转换成大写字母

```
{
    for( size_t i=0; i<sizeof(str)/sizeof(str[0]); ++i )
        if( 'a'<=str[i] && str[i]<='z' )
            str[i] -= ('a'-'A' );
}
```

char str[] = "aBcDe";

cout << "str 字符长度为: " << sizeof(str)/sizeof(str[0]) << endl;

UpperCase( str );

cout << str << endl;

86. 以下代码有什么问题? [C 难]

```
void char2Hex( char c ) // 将字符以 16 进制表示
{
    char ch = c/0x10 + '0'; if( ch > '9' ) ch += ('A'-'9'-1);
    char cl = c%0x10 + '0'; if( cl > '9' ) cl += ('A'-'9'-1);
    cout << ch << cl << ' ';
}

char str[] = "I love 中国";
for( size_t i=0; i<strlen(str); ++i )
    char2Hex( str[i] );

cout << endl;
```

87. 以下代码有什么问题? [C++ 易]

```
struct Test
{
    Test( int ) {}
    Test() {}
    void fun() {}
};

void main( void )
{
    Test a(1);
    a.fun();
    Test b();
    b.fun();
}
```

88. 以下代码有什么问题? [C++ 易]

```
cout << (true?1:"1") << endl;
```

89. 以下代码能够编译通过吗，为什么？ [C++易]

```
unsigned int const size1 = 2;
char str1[ size1 ];
unsigned int temp = 0;
cin >> temp;
unsigned int const size2 = temp;
char str2[ size2 ];
```

90. 以下代码中的输出语句输出 0 吗，为什么？ [C++易]

```
struct CLS
{
    int m_i;
    CLS( int i ) : m_i(i) {}
    CLS()
    {
        CLS(0);
    }
};
CLS obj;
cout << obj.m_i << endl;
```

91. C++中的空类，默认产生哪些类成员函数？ [C++易]

答：

```

class Empty
{
public:
    Empty();                // 缺省构造函数
    Empty( const Empty& );  // 拷贝构造函数
    ~Empty();               // 析构函数
    Empty& operator=( const Empty& ); // 赋值运算符
    Empty* operator&();      // 取址运算符
    const Empty* operator&() const; // 取址运算符 const
};

```

92. 以下两条输出语句分别输出什么？[C++难]

```

float a = 1.0f;
cout << (int)a << endl;
cout << (int&)a << endl;
cout << boolalpha << ( (int)a == (int&)a ) << endl; // 输出什么？
float b = 0.0f;
cout << (int)b << endl;
cout << (int&)b << endl;
cout << boolalpha << ( (int)b == (int&)b ) << endl; // 输出什么？

```

93. 以下反向遍历 `array` 数组的方法有什么错误？[STL 易]

```

vector array;
array.push_back( 1 );
array.push_back( 2 );
array.push_back( 3 );
for( vector::size_type i=array.size()-1; i>=0; --i ) // 反向遍历 array 数组
{

```

```

        cout << array[i] << endl;
    }

```

94. 以下代码有什么问题? [STL 易]

```

typedef vector<int> IntArray;
IntArray array;
array.push_back( 1 );
array.push_back( 2 );
array.push_back( 2 );
array.push_back( 3 );
// 删除 array 数组中所有的 2
for( IntArray::iterator itor=array.begin(); itor!=array.end(); ++itor )
{
    if( 2 == *itor ) array.erase( itor );
}

```

95. 写一个函数，完成内存之间的拷贝。[考虑问题是否全面]

答:

```

void* mymemcpy( void *dest, const void *src, size_t count )
{
    char* pdest = static_cast<char*>( dest );
    const char* psrc = static_cast<const char*>( src );
    if( pdest>psrc && pdest<psrc+count ) 能考虑到这种情况就行了
    {
        for( size_t i=count-1; i!=-1; --i )
            pdest[i] = psrc[i];
    }
}

```

```

        else
        {
            for( size_t i=0; i<count; ++i )
                pdest[i] = psrc[i];
        }
        return dest;
    }
}

```

```

int main( void )
{
    char str[] = "0123456789";
    mymemcpy( str+1, str+0, 9 );
    cout << str << endl;
    system( "Pause" );
    return 0;
}

```

华为 C/C++ 笔试题(附答案)2008 年 02 月 15 日 星期五 18:001. 写出判断 ABCD 四个表达式的是否正确, 若正确, 写出经过表达式中 a 的值(3 分)

96.int a = 4;

(A) a += (a++); (B) a += (++a); (C) (a++) += a; (D) (++a) += (a++);

a = ?

答: C 错误, 左侧不是一个有效变量, 不能赋值, 可改为(++a) += a;

改后答案依次为 9,10,10,11

97. 某 32 位系统下, C++ 程序, 请计算 sizeof 的值(5 分).

```
char str[] = "http://www.ibegroup.com/";
```

```
char *p = str;
```

```
int n = 10;
```

请计算

sizeof (str) = ? (1)

sizeof ( p ) = ? (2)

sizeof ( n ) = ? (3)

```
void Foo ( char str[100]){
```

请计算

sizeof( str ) = ? (4)

```
}  
void *p = malloc( 100 );
```

请计算

sizeof ( p ) = ? ( 5 )

答: (1) 17 (2) 4 (3) 4 (4) 4 (5) 4

98. 回答下面的问题. (4 分)

(1).头文件中的 `ifndef/define/endif` 干什么用? 预处理

答: 防止头文件被重复引用

(2). `#include` 和 `#include "filename.h"` 有什么区别?

答: 前者用来包含开发环境提供的库头文件, 后者用来包含自己编写的头文件。

(3).在 C++ 程序中调用被 C 编译器编译后的函数, 为什么要加 `extern "C"` 声明?

答: 函数和变量被 C++ 编译后在符号库中的名字与 C 语言的不同, 被 `extern "C"` 修饰的变量和函数是按照 C 语言方式编译和连接的。由于编译后的名字不同, C++ 程序不能直接调用 C 函数。C++ 提供了一个 C 连接交换指定符号 `extern "C"` 来解决这个问题。

(4). `switch()` 中不允许的数据类型是?

答: 实型

99. 回答下面的问题(6 分)

(1). `Void GetMemory(char **p, int num){`

```
*p = (char *)malloc(num);
```

```
}
```

```
void Test(void){
```

```
char *str = NULL;
```

```
GetMemory(&str, 100);
```

```
strcpy(str, "hello");
```

```
printf(str);
```

```
}
```

请问运行 `Test` 函数会有什么样的结果?

答: 输出“hello”

(2). `void Test(void){`

```
char *str = (char *) malloc(100);
```

```
strcpy(str, "hello");
```

```
free(str);
```

```
if(str != NULL){
```

```
strcpy(str, "world");
```

```
printf(str);
```

```
}
```

```
}
```

请问运行 `Test` 函数会有什么样的结果?

答: 输出“world”

100. `char *GetMemory(void){`

```
char p[] = "hello world";
```

```

return p;
}
void Test(void){
char *str = NULL;
str = GetMemory();
printf(str);
}

```

请问运行 **Test** 函数会有什么样的结果？

答：无效的指针，输出不确定

#### 101. 编写 **strcat** 函数(6 分)

已知 **strcat** 函数的原型是 `char *strcat (char *strDest, const char *strSrc);`

其中 **strDest** 是目的字符串，**strSrc** 是源字符串。

(1) 不调用 C++/C 的字符串库函数，请编写函数 **strcat**

答：

VC 源码：

```

char * __cdecl strcat (char * dst, const char * src)
{
char * cp = dst;
while( *cp )
cp++; /* find end of dst */
while( *cp++ = *src++ ); /* Copy src to end of dst */
return( dst ); /* return dst */
}

```

(2) **strcat** 能把 **strSrc** 的内容连接到 **strDest**，为什么还要 `char *` 类型的返回值？

答：方便赋值给其他变量

#### 102.MFC 中 **CString** 是类型安全类么？

答：不是，其它数据类型转换到 **CString** 可以使用 **CString** 的成员函数 **Format** 来转换

#### 103.C++中为什么用模板类。

答：(1)可用来创建动态增长和减小的数据结构

(2)它是类型无关的，因此具有很高的可复用性。

(3)它在编译时而不是运行时检查数据类型，保证了类型安全

(4)它是平台无关的，可移植性

(5)可用于基本数据类型

#### 104.CSingleLock 是干什么的。

答：同步多个线程对一个数据类的同时访问

#### 105.NEWTEXTMETRIC 是什么。

答：物理字体结构，用来设置字体的高宽大小

106.程序什么时候应该使用线程，什么时候单线程效率高。

答：1. 耗时的操作使用线程，提高应用程序响应

2. 并行操作时使用线程，如 C/S 架构的服务器端并发线程响应用户的请求。

3. 多 CPU 系统中，使用线程提高 CPU 利用率

4. 改善程序结构。一个既长又复杂的进程可以考虑分为多个线程，成为几个独立或半独立的运行部分，这样的程序会利于理解和修改。

其他情况都使用单线程。

107.Windows 是内核级线程么。

答：见下一题

108.Linux 有内核级线程么。

答：线程通常被定义为一个进程中代码的不同执行路线。从实现方式上划分，线程有两种类型：“用户级线程”和“内核级线程”。用户线程指不需要内核支持而在用户程序中实现的线程，其不依赖于操作系统核心，应用进程利用线程库提供创建、同步、调度和管理线程的函数来控制用户线程。这种线程甚至在象 DOS 这样的操作系统中也可实现，但线程的调度需要用户程序完成，这有些类似 Windows 3.x 的协作式多任务。另外一种则需要内核的参与，由内核完成线程的调度。其依赖于操作系统核心，由内核的内部需求进行创建和撤销，这两种模型各有其好处和缺点。用户线程不需要额外的内核开支，并且用户态线程的实现方式可以被定制或修改以适应特殊应用的要求，但是当线程因 I/O 而处于等待状态时，整个进程就会被调度程序切换为等待状态，其他线程得不到运行的机会；而内核线程则没有各个限制，有利于发挥多处理器的并发优势，但却占用了更多的系统开支。

Windows NT 和 OS/2 支持内核线程。Linux 支持内核级的多线程

109.C++中什么数据分配在栈或堆中，New 分配数据是在近堆还是远堆中？

答：栈：存放局部变量，函数调用参数,函数返回值，函数返回地址。由系统管理

堆：程序运行时动态申请，new 和 malloc 申请的内存就在堆上

110.使用线程是如何防止出现大的波峰。

答：意思是如何防止同时产生大量的线程，方法是使用线程池，线程池具有可以同时提高调度效率和限制资源使用的好处，线程池中的线程达到最大数时，其他线程就会排队等候。

111.一般数据库若出现日志满了，会出现什么情况，是否还能使用？

答：只能执行查询等读操作，不能执行更改，备份等写操作，原因是任何写操作都要记录日志。也就是说基本上处于不能使用的状态。

112 SQL Server 是否支持行级锁，有什么好处？

答：支持，设立封锁机制主要是为了对并发操作进行控制，对干扰进行封锁，保证数据的一致性和准确性，行级封锁确保在用户取得被更新的行到该行进行更新这段时间内不被其它用户所修改。因而行级锁即可保证数据的一致性又能提高数据操作的进发性。

113. 关于内存对齐的问题以及 `sizeof()` 的输出

答：编译器自动对齐的原因：为了提高程序的性能，数据结构（尤其是栈）应该尽可能地在自然边界上对齐。原因在于，为了访问未对齐的内存，处理器需要作两次内存访问；然而，对齐的内存访问仅需要一次访问。

114. `int i=10, j=10, k=3; k*=i+j; k` 最后的值是？

答：60，此题考察优先级，实际写成：`k*=(i+j);`，赋值运算符优先级最低

115. 对数据库的一张表进行操作,同时要对另一张表进行操作,如何实现？

答：将操作多个表的操作放入到事务中进行处理

116. TCP/IP 建立连接的过程?(3-way shake)

答：在 TCP/IP 协议中，TCP 协议提供可靠的连接服务，采用三次握手建立一个连接。

第一次握手：建立连接时，客户端发送 `syn` 包(`syn=j`)到服务器，并进入 `SYN_SEND` 状态，等待服务器确认；

第二次握手：服务器收到 `syn` 包，必须确认客户的 `SYN` (`ack=j+1`)，同时自己也发送一个 `SYN` 包 (`syn=k`)，即 `SYN+ACK` 包，此时服务器进入 `SYN_RECV` 状态；

第三次握手：客户端收到服务器的 `SYN+ACK` 包，向服务器发送确认包 `ACK(ack=k+1)`，此包发送完毕，客户端和服务器进入 `ESTABLISHED` 状态，完成三次握手。

117. ICMP 是什么协议,处于哪一层？

答：Internet 控制报文协议，处于网络层（IP 层）

118. 触发器怎么工作的？

答：触发器主要是通过事件进行触发而被执行的，当对某一表进行诸如 `UPDATE`、`INSERT`、`DELETE` 这些操作时，数据库就会自动执行触发器所定义的 `SQL` 语句，从而确保对数据的处理必须符合由这些 `SQL` 语句所定义的规则。

119. winsock 建立连接的主要实现步骤？

答：服务器端：`socket()`建立套接字，绑定（`bind`）并监听（`listen`），用 `accept()` 等待客户端连接。

客户端：`socket()`建立套接字，连接（`connect`）服务器，连接上后使用 `send()`和 `recv()`，在套接字上写读数据，直至数据交换完毕，`closesocket()`关闭套接字。

服务器端：`accept()`发现有客户端连接，建立一个新的套接字，自身重新开始等待连接。该新产生的套接字使用 `send()`和 `recv()`写读数据，直至数据交换完毕，`closesocket()`关闭套接字。

120. 动态连接库的两种方式？

答：调用一个 DLL 中的函数有两种方法：

1. 载入时动态链接（load-time dynamic linking），模块非常明确调用某个导出函数，使得他们就像本地函数一样。这需要链接时链接那些函数所在 DLL 的导入库，导入库向系统提供了载入 DLL 时所需的信息及 DLL 函数定位。
2. 运行时动态链接（run-time dynamic linking），运行时可以通过 `LoadLibrary` 或 `LoadLibraryEx` 函数载入 DLL。DLL 载入后，模块可以通过调用 `GetProcAddress` 获取 DLL 函

数的

出口地址，然后就可以通过返回的函数指针调用 DLL 函数了。如此即可避免导入库文件了。

#### 121.IP 组播有那些好处？

答：Internet 上产生的许多新的应用，特别是高带宽的多媒体应用，带来了带宽的急剧消耗和网络拥挤问题。组播是一种允许一个或多个发送者（组播源）发送单一的数据包到多个接收者（一次的，同时的）的网络技术。组播可以大大的节省网络带宽，因为无论有多少个目标地址，在整个网络的任何一条链路上只传送单一的数据包。所以说组播技术的核心就是针对如何节约网络资源的前提下保证服务质量。

#### 122. 以下代码中的两个 sizeof 用法有问题吗？[C 易]

```
void UpperCase( char str[] ) // 将 str 中的小写字母转换成大写字母
{
    for( size_t i=0; i<sizeof(str)/sizeof(str[0]); ++i )
        if( 'a'<=str[i] && str[i]<='z' )
            str[i] -= ('a'-'A' );
}
char str[] = "aBcDe";
cout << "str 字符长度为: " << sizeof(str)/sizeof(str[0]) << endl;
UpperCase( str );
cout << str << endl;
```

答：函数内的 sizeof 有问题。根据语法，sizeof 如用于数组，只能测出静态数组的大小，无法检测动态分配的或外部数组大小。函数外的 str 是一个静态定义的数组，因此其大小为

123，函数内的 str 实际只是一个指向字符串的指针，没有任何额外的与数组相关的信息，因此 sizeof 作用于上只将其当指针看，一个指针为 4 个字节，因此返回 4。

一个 32 位的机器,该机器的指针是多少位

指针是多少位只要看地址总线的位数就行了。80386 以后的机子都是 32 的数据总线。所以指针的位数就是 4 个字节了。

#### 124.

```
main()
{
    int a[5]={1,2,3,4,5};
    int *ptr=(int *)(&a+1);

    printf("%d,%d",*(a+1),*(ptr-1));
}
```

输出：2,5

\*(a+1) 就是 a[1]，\*(ptr-1)就是 a[4],执行结果是 2，5

&a+1 不是首地址+1，系统会认为加一个 a 数组的偏移，是偏移了一个数组的大小（本例是

5 个 int)

```
int *ptr=(int *)(&a+1);
```

则 ptr 实际是&(a[5]),也就是 a+5

原因如下:

&a 是数组指针, 其类型为 int (\*)[5];

而指针加 1 要根据指针类型加上一定的值,

不同类型的指针+1 之后增加的大小不同

a 是长度为 5 的 int 数组指针, 所以要加 5\*sizeof(int)

所以 ptr 实际是 a[5]

但是 prt 与(&a+1)类型是不一样的(这点很重要)

所以 prt-1 只会减去 sizeof(int\*)

a,&a 的地址是一样的, 但意思不一样, a 是数组首地址, 也就是 a[0]的地址, &a 是对象(数组)首地址, a+1 是数组下一元素的地址, 即 a[1],&a+1 是下一个对象的地址, 即 a[5].

125.请问以下代码有什么问题:

```
int main()
{
    char a;
    char *str=&a;
    strcpy(str,"hello");
    printf(str);
    return 0;
}
```

没有为 str 分配内存空间, 将会发生异常

问题出在将一个字符串复制进一个字符变量指针所指地址。虽然可以正确输出结果, 但因为越界进行内在读写而导致程序崩溃。

```
char* s="AAA";
printf("%s",s);
s[0]='B';
printf("%s",s);
```

有什么错?

"AAA"是字符串常量。s 是指针, 指向这个字符串常量, 所以声明 s 的时候就有问题。

```
const char* s="AAA";
```

然后又因为是常量, 所以对 s[0]的赋值操作是不合法的。

126、关键字 volatile 有什么含意? 并举出三个不同的例子?

提示编译器对象的值可能在编译器未监测到的情况下改变。

127、将树序列化 转存在数组或 链表中

128.下面哪个实体不是 SIP 协议定义的 (A)。

A ) MGC      B ) UA      C ) proxy      D ) Redirector

UA = User Agent 用户代理,指客户端的协议栈

PROXY = SIP Proxy ,指 SIP 服务器

Redirector = 重定向模块。一般用于跨服务器通讯

129. VOIP 中本端与对端呼叫接通后, 将通话转接到第三方称之为 (C)。

A) 呼叫转接 B) 呼叫前转 C) 呼叫转移 D) 三方通话

跟普通电话相同, A, B 都是没有接通前就把呼叫转接。D 是指三方同时在通话。

130. VOIP 的主要优点是 (D)

A) 价格便宜并且能为客户提供更好的增值服务。

B) 语音质量比传统的 PSTN 电话好。

C) 通话的安全性和可靠性有更高的保障。

D) 不需要服务提供商就可以使用。

音质, 可靠性是传统电话好。这题的问题在于, 增值服务是指什么? 如是 SP 则 VOIP 不支持。还是服务器提供商是指什么? VOIP 需要服务器。

131. 下面哪个技术不属于语音处理技术 (D)

A) 静音检测

B) 分组丢失补偿

C) 同声消除

D) 网路穿越

D 是网络传输的问题, 主要是穿透 NAT 网关。特别是比较复杂的网络。

132. SIP 协议是使用下面哪种方式编解码的 ( )

A) ASN.1 B) BER C) ABNF D) PER

网络应用

133. 在不同网络中 MTU 会发生变化, 由于 MTU 的变化, 会导致那些值也相应发生变化 (A)

A) IP 总长度字段

B) TCP MSS 字段

C) UDP 长度字段

D) TCP 长度字段

待查, MTU 变化会让 IP 分片或重组。因此变化就是 IP

134. 下列关于 TCP 和 UDP 的正确的说法是 (C)

A) TCP 实时性比 UDP 好

B) TCP 比 UDP 效率高

C) TCP 首部比 UDP 的首部长

D) TCP 安全性比 UDP 高

实时性, 效率。安全性, TCP 不见得比 UDP 高

135. 一个套接口应该匹配 (D)

A) 源目标 IP

B) 源目标 IP 端口

C) 目标 IP 和目标端口

D) 本地 IP 和本地端口

SOCKET 相当一 IP 连接上用端口标识队列

136. TCP 服务器出现异常并马上重启, 通常会出现那些情况 ( )

A) socket 调用失败

B) bind 调用失败

C) listen 调用失败

D) select 调用失败

此题有问题，一般软件很难自动重启。而 TCP 服务器可以在任何一个阶段出问题，上述哪一个都能出现，这个本意应该是指 Select.

底层开发

137. 在一台以字节为最小存储单位的机器上，如果把 0x12345678 写到从 0x0000 开始的地址上，下列关于 big—endian 和 little—endian 说法正确的是 (B)

A) 在 big—endian 模式下，地址 0x0000 到 0x0003 存储的数据依次为：0x56, 0x78, 0x12, 0x34

B) 在 big—endian 模式下，地址 0x0000 到 0x0003 存储的数据依次为：0x12, 0x34, 0x56, 0x78

C) 在 little—endian 模式下，地址 0x0000 到 0x0003 存储的数据依次为：0x34, 0x12, 0x78, 0x56

D) 在 little—endian 模式下，地址 0x0000 到 0x0003 存储的数据依次为：0x56, 0x78, 0x12, 0x34

138. 以下关于交叉编译器概述正确的是 (A)

A) 交叉编译器一般按照 CPU 类型分类，不同的 CPU 就有不同的交叉编译器

B) 交叉编译器的速度比其他编译器的速度要快

C) linux 开发环境中的交叉编译器不是 gcc 编译器

D) 交叉编译器编译出来的目标文件一般也能在开发机（普通 PC）上运行

139. 以下关于 linux 下中断的说法正确的是 ( )

A) 中断只能由硬件产生

B) 中断服务例程不能运行参数传递

C) 中断服务例程可以在函数结束的时候返回一个值

D) 中断服务例程是不可中断的

D,B?

140. 以下关于 linux 下系统调用的说法错误的是 ( )

A) 应用程序通过系统调用访问内核

B) 每个系统调用都有一个唯一的系统调用号

C) 用户可以定制自己的系统调用

D)

可能是 A，系统调用在内核执行。但这里访问比较模糊。

141. 关于 SPI 说法正确的是 ( )

A) SPI 工作在全双工模式下

B)

C)

D) SPI 接口一般工作在主从模式下

C 语言

142. Char Test[10];char \*pTest=test;问：&Test 在数值上等于 (A)

A) Test B) Test[0] C)&pTest D)符号表某个符号的地址

&Test 相当于二维指针首指针，TEST 是一维的首指针

143. 在顺序表{3,6,8,10,12,15,16,18,21,25,30}中, 用二分法查找关键码值 11, 所需的关键码比较次数为 ( ) B?

A) 2    B) 3    C) 4    D) 5

144. 单链表中每个结点中包括一个指针 link, 它指向该结点的下一个结点, 现要将指针 q 指向的新结点放到指针 p 指向的单链表接点之后, 下面的操作序列中哪一个是正确的 (C)

A) q:=p^.link;p^.link:=q^.link

B)p^.link:=q^.link;q:=p^.link

C)q^.link:=p^.link;p^.link:=q

D)p^.link:=q;q^.link:=p^.link

145. 以下叙述正确的是 (C)

A) 在 C 程序中, main 函数必须位于程序的最前面

B) C 程序的每行中只能写一条语句

C) C 语言本身没有输入输出语句

D) 在对一个 C 程序进行编译的过程中, 可发现注释中的拼写错误

146. 有以下程序

```
Main()
{
    Char a[]="programming",b[]="language";
    Char *p1,*p2;
    Int i;
    P1=a;p2=b;
    For(i=0;i<7;i++)
        If(*(p1+i)==*(p2+i))
            Printf("%c",*(p1+i));
}
```

打印出什么 ( )

147. 请简述以下两个 for 循环的优缺点 (6 分)

|                                                                                                                         |                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// 第一个 for (i=0; i&lt;N; i++) {     if (condition)         DoSomething();     else         DoOtherthing(); }</pre> | <pre>// 第二个 if (condition) {     for (i=0; i&lt;N; i++)         DoSomething(); } else {     for (i=0; i&lt;N; i++)         DoOtherthing(); }</pre> |
| 优点:<br>当 N 较大时, 效率较高<br>缺点:<br>每一个循环都要做一次 if 判断                                                                         | 优点:<br>当 N 较小时, 效率较高<br>缺点:<br>循环次数较多                                                                                                              |

|  |  |
|--|--|
|  |  |
|--|--|

148. 位运算：给定一个整型变量 a，（1）给 bit3 置数（2）清除 bit3（6 分）

```
a|=(0x1<<3);
a&=~(0x1<<3);
```

149. 评述下面的代码（6 分）

```
Main ( )
{
    Int a[100];
    Int *p;
    P=( unsigned int *) a+1);
    Printf("0x%x",*p);
}
```

- 1.数组没有赋初值，
- 2.指针类型转换不一致
- 3..打印一个未赋值的整数值,p=a[1]

```
#include <stdio.h>
```

```
main()
{
    int a[100]={1,2,3};
    unsigned int *p;
    p=((unsigned int *)a+1);
    printf("%x,%x\n",a,p);
    printf("0x%x",*p);
    getchar();
}
```

150. 编程题；(10 分)

从键盘输入一组字符；  
字母按升序排列；其余的字符按升序排列  
字母放前，其余的放后

例如：输入：\_@AB-@ab 结果：ABab-@@\_

151.简述需求分析的过程和意义

152.网状、层次数据模型与关系数据模型的最大的区别是什末

153.软件质量保证体系是什末 国家标准中与质量保证管理相关的几个标准是什末 编号和全称是什末号和全称是什末

153 文件格式系统有哪几种类型? 分别说说 win95、win98、winMe、w2k、winNT、winXP 分别支持那些文件系统

154. 我现在有个程序, 发现在 WIN98 上运行得很慢, 怎么判别是程序存在问题还是软硬件系统存在问题?

155. 有关 P2P 点对点文件传输的原理

156.一台计算机的 IP 是 192.168.10.71 子网掩码 255.255.255.64 与 192.168.10.201 是同一局域网吗?

157.internet 中 e-mail 协议,IE 的协议,NAT 是什么,有什么好处,能带来什么问题?DNS 是什么,它是如何工作的?

158.PROXY 是如何工作的?

169.win2k 系统内 AT 命令完成什么功能,Messenger 服务是做什么,怎么使用?

170 进程,线程的定义及区别

171,32 位操作系统内,1 进程地址空间多大,进程空间与物理内存有什么关系?

172.网络攻击常用的手段,防火墙如何保证安全.

173.如何配静态 IP,如何测网络内 2 台计算机通不通,PING 一次返几个数据包?

174.WIN9X 与 WINNT 以上操作系统有"服务"吗,服务是什么,如何停止服务?

175.AD 在 WIN2KSERVER 上建需什么文件格式,AD 是什么?XP 多用户下"注销"与"切换"的区别.

176. UDP 可以跨网段发送吗?

177.最简单的确认远程计算机(win2K 以上)某个监听端口是正常建立的?

178. 找错

```
void test1()
{
```

```

char string[10];
char* str1="0123456789";
strcpy(string, str1);
}

```

答：表面上并且编译都不会错误。但如果 **string** 数组原意表示的是字符串的话，那这个赋值就没有达到意图。最好定义为 **char string[11]**，这样最后一个元素可以存储字符串结尾符'\0'；

```

void test2()
{
    char string[10], str1[10];
    for(int l=0; l<10;l++)
    {
        str1[l] ='a';
    }
    strcpy(string, str1);
}

```

答：**strcpy** 使用错误，**strcpy** 只有遇到字符串末尾的'\0'才会结束，而 **str1** 并没有结尾标志，导致 **strcpy** 函数越界访问，不妨让 **str1[9]='\0'**，这样就正常了。

```

void test3(char* str1)
{
    char string[10];
    if(strlen(str1)<=10)
    {
        strcpy(string, str1);
    }
}

```

答：这又会出现第一道改错题的错误了。**strlen(str1)**算出来的值是不包含结尾符'\0'的，如果 **str1** 刚好为 10 个字符 + 1 结尾符，**string** 就得不到结尾符了。可将 **strlen(str1)<=10** 改为 **strlen(str1)<10**。

179. 找错

```

#define MAX_SRM 256

```

```

DSN get_SRM_no()
{
    static int SRM_no;
    int l;

```

```

    for(l=0;l<MAX_SRM;l++,SRM_no++)
    {
        SRM_no %= MAX_SRM;
        if(MY_SRM.state==IDLE)
        {
            break;
        }
    }
    if(l>=MAX_SRM)
        return (NULL_SRM);
    else
        return SRM_no;
}

```

答：我不知道这段代码的具体功能，但明显有两个错误

- 1, SRM\_no 没有赋初值
- 2, 由于 **static** 的声明，使该函数成为不可重入（即不可预测结果）函数，因为 SRM\_no 变量放在程序的全局存储区中，每次调用的时候还可以保持原来的赋值。这里应该去掉 **static** 声明。

180. 写出程序运行结果

```

int sum(int a)
{
    auto int c=0;
    static int b=3;
    c+=1;
    b+=2;
    return(a+b+c);
}
void main()
{
    int l;
    int a=2;
    for(l=0;l<5;l++)
    {
        printf("%d,", sum(a));
    }
}

```

答：8,10,12,14,16

该题比较简单。只要注意 **b** 声明为 **static** 静态全局变量，其值在下次调用时是可以保持住原来的赋值的就可以。

181.

```
int func(int a)
{
    int b;
    switch(a)
    {
        case 1: b=30;
        case 2: b=20;
        case 3: b=16;
        default: b=0;
    }
    return b;
}
```

则 func(1)=?

答: func(1)=0, 因为没有 break 语句, switch 中会一直计算到 b=0。这是提醒我们不要忘了 break。呵呵。

182:

```
int a[3];
a[0]=0; a[1]=1; a[2]=2;
int *p, *q;
p=a;
q=&a[2];
```

则 a[q-p]=?

答: a[q-p]=a[2]=2;这题是要告诉我们指针的运算特点

183.

定义 int \*\*a[3][4], 则变量占有的内存空间为: \_\_\_\_\_

答: 此处定义的是指向指针的指针数组, 对于 32 位系统, 指针占内存空间 4 字节, 因此总空间为  $3 \times 4 \times 4 = 48$ 。

184.

编写一个函数, 要求输入年月日时分秒, 输出该年月日时分秒的下一秒。如输入 2004 年 12 月 31 日 23 时 59 分 59 秒, 则输出 2005 年 1 月 1 日 0 时 0 分 0 秒。

答:

```
/*输入年月日时分秒, 输出年月日时分秒的下一秒, 输出仍然在原内存空间*/
bool NextMinute(int *nYear,int *nMonth,int *nDate,int *nHour,int *nMinute,int *nSecond)
```

```

{
    if(*nYear<0 || *nMonth>12 || *nMonth<0 || *nHour>23 || *nHour<0 || *nMinute<0 ||
*nMinute>59 || *nSecond<0 || *nSecond>59) return false;
    int nDays;
    switch(*nMonth)
    {
        case 1:
        case 3:
        case 5:
        case 7:
        case 8:
        case 10:
        case 12:
            nDays=31;
            break;
        case 2:// 判断闰年
            if(*nYear%400==0||*nYear%100!=0&&*nYear%4==0)
            {
                nDays=29;
            }
            else
            {
                nDays=28;
            }
            break;
        default:
            nDays=30;
            break;
    }
    if(*nDate<0 || *nDate>nDays) return false;

```

(\*nSecond)++; // 秒加 1

if(\*nSecond>=60) // 秒满 60，做出特殊处理，下面时，日，月等类同

```

{
    *nSecond=0;
    (*nMinute)++;
    if(*nMinute>=60)
    {
        *nMinute=0;
        (*nHour)++;
        if(*nHour>=24)
        {
            *nHour=0;
            (*nDate)++;

```

```

        if(*nDate>nDays)
        {
            *nDate=1;
            (*nMonth)++;
            if(*nMonth>12)
            {
                *nMonth=1;
                (*nYear)++;
            }
        }
    }
}
return true;
}
/*示例可运行代码*/
void main()
{
    int nYear=2004,nMonth=12,nDate=31,nHour=23,nMinute=59,nSecond=59;
    bool res = NextMinute(&nYear,&nMonth,&nDate,&nHour,&nMinute,&nSecond);
    if(res)
        printf("The
result:%d-%d-%d %d:%d:%d",nYear,nMonth,nDate,nHour,nMinute,nSecond);
    else
        printf("Input error!\n");
}

```

185. 写一个函数，判定运算环境（16 位以上字长）是 little-endian 还是 big-endian

186. 操作系统的主要组成部分？

187. 操作系统中进程调度策略有哪几种？

188. 进程间主要的通讯方式？

189. 写出进程的主要状态？

190. 以太网物理地址和 IP 地址转换采用什么协议？

191. IP 地址的编码分为哪两部分？

192. 写出以太网数据帧格式

193. 8031 和 8051 的主要区别？

194. C++中的空类，默认产生哪些类成员函数？

分析以下程序的执行结果

```
#include<iostream.h>
class base
{
public:
base(){cout<< "constructing base class"<<endl;}
~base(){cout<<"destructing base class"<<endl;}
};
class subs:public base
{
public:
subs(){cout<<"constructing sub class"<<endl;}
~subs(){cout<<"destructing sub class"<<endl;}
};
void main()
{
subs s;
}
```

195.指出下面程序的错误

```
#define SIZE 5
struct String
{
char *pData;
};
void main()
{
char *pData;
};
void main()
{
char acValue1[SIZE]={'H','E','L','L','O'};
char acValue2[SIZE]={'W','O','L','D'};
struct String a,b,c;
a.pData=malloc(SIZE*sizeof(char));
memcpy(a.pData,acValue1,SIZE);
b.pData=malloc(SIZE*sizeof(char));
memcpy(b.pData,acValue2,SIZE);
b=a;
```

```

    free(a.pData);
    c=b;
}

```

196.指出下面两段程序的区别

**【1】**

```

main()
{
    int loop=1;
    int arr[10];
    int i=0;
    while(loop<5)
    {
        for(;i<=10;i++)
        {
            arr[i]=1;
        }
        loop++;
    }
}

```

**【2】**

```

main()
{
    int arr[10];
    int loop=1;
    int i=0;
    while(loop<5)
    {
        for(i=0;i<=10;i++)
        {
            arr[i]=1;
        }
        loop++;
    }
}

```

197.指出下面程序的错误（函数 GetValue 返回 unsigned char 类型的值）

```

#define MAXNUM 400;
unsigned char ucIndex,ucInputVar,aucArray[MAXNUM];
for(ucIndx =0;ucIndex<=MAXNUM;ucIndex++)
{
    aucArray[ucIndex]=aucArray[ucIndex]+1;
}

```

```

ucInputVar=GetValue();
for(ucIndex=0;ucIndex>(ucInputVar-1);ucIndex++)
{
    aucArray[ucIndex]=aucArray[ucIndex]*2+1;
}

```

#### 198.什么是 com 和 ActiveX，简述 DCOM。

答：COM（Component Object Mode）即组件对象模型，是组件之间相互接口的规范。其作用是使各种软件构件和应用软件能够用一种统一的标准方式进行交互。COM 不是一种面向对象的语言，而是一种与源代码无关的二进制标准。

ActiveX 是 Microsoft 提出的一套基于 COM 的构件技术标准，实际上是对象嵌入与链接（OLE）的新版本。基于分布式环境下的 COM 被称作 DCOM(Distribute COM，分布式组件对象模型)，它实现了 COM 对象与远程计算机上的另一个对象之间直接进行交互。DCOM 规范定义了分散对象创建和对象间通信的机制，DCOM 是 ActiveX 的基础，因为 ActiveX 主要是针对 Internet 应用开发（相比 OLE）的技术，当然也可以用于普通的桌面应用程序。

#### 199.列出 3 个常用网络协议使用的端口。

答：HTTP 协议用 80 端口，FTP 协议用 21 端口，POP3 协议用 110 端口

#### 199 什么是 ODBC?

答：ODBC(Open Database Connectivity，开放数据库互连)是微软公司开放服务结构(WOSA，Windows Open Services Architecture)中有关数据库的一个组成部分，它建立了一组规范，并提供了一组对数据库访问的标准 API（应用程序编程接口）。ODBC 的最大优点是能以统一的方式(用它提供的 API 访问数据库)处理所有的数据库。

#### 200 结构化编程和 goto 语句的区别和关系？

答：结构化编程设计思想采用了模块分解与功能抽象和自顶向下、分而治之的方法，从而有效地将一个较复杂的程序系统设计任务分解成许多易于控制和处理的子程序，便于开发和维护。goto 语句可以实现无条件跳转，改变程序流向，破坏结构化编程设计风格。但 goto 语句在结构化编程中并非不可使用，只是要受到限制的使用。

#### 201 MFC 中 SendMessage 和 PostMessage 的区别？

答：PostMessage 和 SendMessage 的区别主要在于是否等待应用程序做出消息处理。PostMessage 只是把消息放入队列，然后继续执行；而 SendMessage 必须等待应用程序处理消息后才返回继续执行。这两个函数的返回值也不同，PostMessage 的返回值表示 PostMessage 函数执行是否正确，而 SendMessage 的返回值表示其他程序处理消息后的返回值。

#### 202. 改错

```

#include
#include
class CBuffer
{
char * m_pBuffer;

```

```

int m_size;
public:
CBuffer()
{
m_pBuffer=NULL;
}
~CBuffer()
{
Free();
}
void Allocte(int size) (3) {
m_size=size;
m_pBuffer= new char[size];
}
private:
void Free()
{
if(m_pBuffer!=NULL)
{
delete m_pBuffer;
m_pBuffer=NULL;
}
}
public:
void SaveString(const char* pText) const
{
strcpy(m_pBuffer, pText);
}
char* GetBuffer() const
{
return m_pBuffer;
}
};

void main (int argc, char* argv[])
{
CBuffer buffer1;
buffer1.SaveString("Microsoft");
printf(buffer1.GetBuffer());
}

```

答：改正后

主要改正 **SaveString** 函数  
将

```
void SaveString(const char* pText) const
{
    strcpy(m_pBuffer, pText);
}
```

改为

```
void SaveString(const char* pText) (1)
{
    Allocte(strlen(pText)+1); (2)
    strcpy(m_pBuffer, pText);
}
```

原因：

- (1) **const** 成员函数表示不会修改数据成员，而 **SaveString** 做不到，去掉 **const** 声明
- (2) **m\_pBuffer** 指向 **NULL**，必须用 **Allocte** 分配空间才能赋值。
- (3) 另外需要将 **Allocte** 成员函数声明为私有成员函数更符合实际

203. 下来程序想打印“Welcome MSR Asia”，改正错误

```
#include
#include
char * GetName (void)
{
    //To return "MSR Asia" String
    char name[]="MSR Asia";
    return name;
}
void main(int argc, char* argv[])
{
    char name[32];
    //Fill in zeros into name
    for(int i=0;i<=32;i++)
    {
        name[i]='\0';
    }
    //copy "Welcome" to name
    name="Welcome";
    //Append a blank char
    name[8]=" ";
    //Append string to name
    strcat(name,GetName());
    //print out
    printf(name);
}
```

答：改正后为

```
#include
```

```

#include
char * GetName (void)
{
//To return "MSR Asia" String
//char name[]="MSR Asia"; (1)
char *name=(char *)malloc(strlen("MSR Asia")+1);
strcpy(name,"MSR Asia");
return name;
}
void main(int argc, char* argv[])
{
char name[32];
//Fill in zeros into name
for(int i=0;i<=32;i++)
{
name[i]='\0';
}
//copy "Welcome" to name
//name="Welcome"; (2)
strcat(name,"Welcome ");
//Append a blank char
// name[8]=' '; (3)
//Append string to name
char *p=GetName(); (4)
strcat(name,p);
free (p);
//print out
printf(name);
}

```

原因：(1) 在函数内部定义的变量在函数结束时就清空了，必须动态分配内存

(2) 字符串赋值语句错误，应该用 **strcat**

(3) 该语句无效，可去掉

(4) 定义一个指针指向动态分配的内存，用完后需用 **free** 语句释放

204. 写出下面程序的输出结果

```

#include
class A
{
public:
void FuncA()
{
printf("FuncA called\n");
}
}

```

```
virtual void FuncB()
{
printf("FuncB called\n");
}
};
```

```
class B: public A
{
public:
void FuncA()
{
A::FuncA();
printf("FuncAB called\n");
}
virtual void FuncB()
{
printf("FuncBB called\n");
}
};
```

```
void main(void)
{
B b;
A *pa;
pa=&b;
A *pa2=new A;
b.FuncA(); (1)
b.FuncB(); (2)
pa->FuncA(); (3)
pa->FuncB(); (4)
pa2->FuncA(); (5)
pa2->FuncB();
delete pa2;
}
```

答:

1. b.FuncA(); 输出

FuncA called

FuncAB called

2. b.FuncB();输出

FuncBB called

上两者好理解，直接调用类 B 的相应成员函数

3. pa->FuncA();输出

FuncA called 调用类 A 的 FuncA()

4. pa->FuncB();输出

FuncBB called 调用类 B 的 FuncB(), 原因是 C++ 的动态决议机制, 当基类函数声明为 virtual 时, 指向派生类对象的基类指针来调用该函数会选择派生类的实现, 除非派生类没有才调用基类的虚函数。还有一点注意的是: 指向基类类型的指针可以指向基类对象也可以指向派生类对象, 如 pa=&b;

5. pa2->FuncA();

pa2->FuncB(); 输出

FuncA called

FuncB called

这也好理解, 直接调用类 A 的相应成员函数

206. In the main() function, after ModifyString(text) is called, what's the value of 'text'?

```
#include
```

```
#include
```

```
int FindSubString(char* pch)
```

```
{
```

```
int count=0;
```

```
char* p1=pch;
```

```
while(*p1!='\0')
```

```
{
```

```
if(*p1==p1[1]-1)
```

```
{
```

```
p1++;
```

```
count++;
```

```
}
```

```
else
```

```
{
```

```
break;
```

```
}
```

```
}
```

```
int count2=count;
```

```
while(*p1!='\0')
```

```
{
```

```
if(*p1==p1[1]+1)
```

```
{
```

```
p1++;
```

```
count2--;
```

```
}
```

```
else
```

```
{
```

```
break;
```

```
}
```

```
}
```

```
if(count2==0)
```

```
return count;
```

```

return 0;
}

void ModifyString(char* pText)
{
char* p1=pText;
char* p2=p1;
while(*p1!='\0')
{
int count=FindSubString(p1);
if(count>0)
{
*p2++=*p1;
sprintf(p2, "%I", count);
while(*p2!= '\0')
{
p2++;
}
p1+=count+count+1;
}
else
{
*p2++=*p1++;
}
}
}
void main(void)
{
char text[32]="XYBCDCBABABA";
ModifyString(text);
printf(text);
}

```

答：我不知道这个结构混乱的程序到底想考察什么，只能将最后运行结果写出来是XYBCDCBAIBAAP

## 207. Programming (Mandatory)

### Linked list

- Implement a linked list for integers, which supports the insertafter (insert a node after a specified node) and removeafter (remove the node after a specified node) methods;
- Implement a method to sort the linked list to descending order.

答：题目的意思是实现一个整型链表，支持插入，删除操作（有特殊要求，都是在指定节点后进行操作），并写一个对链表数据进行降序排序的方法。

那我们不妨以一个线性链表进行编程。

// 单链表结构体为

```
typedef struct LNode
{
    int data;
    struct LNode *next;
}LNode, *pLinkList;
```

// 单链表类

```
class LinkList
{
private:
    pLinkList m_pList;
    int m_listLength;
public:
    LinkList();
    ~LinkList();
    bool InsertAfter(int afternode, int data);//插入
    bool RemoveAfter(int removenode);//删除
    void sort();//排序
};
```

实现方法

//insert a node after a specified node

```
bool LinkList::InsertAfter(int afternode, int data)
{
    LNode *pTemp = m_pList;
    int curPos = -1;
    if (afternode > m_listLength ) // 插入点超过总长度
    {
        return false;
    }
    while (pTemp != NULL) // 找到指定的节点
    {
        curPos++;
        if (curPos == afternode)
            break;
        pTemp = pTemp->next;
    }
    if (curPos != afternode) // 节点未寻到，错误退出
    {
        return false;
    }
    LNode *newNode = new LNode; // 将新节点插入指定节点后
    newNode->data = data;
    newNode->next = pTemp->next;
```

```
pTemp->next = newNode;
m_listLength++;
return true;
}
```

//remove the node after a specified node

```
bool LinkList::RemoveAfter(int removenode)
{
    LNode *pTemp = m_pList;
    int curPos=-1;
    if (removenode > m_listLength) // 删除点超过总长度
    {
        return false;
    }
```

// 找到指定的节点后一个节点，因为删除的是后一个节点

```
while (pTemp != NULL)
{
    curPos++;
    if (curPos == removenode+1)
        break;
    pTemp = pTemp->next;
}
if (curPos != removenode) // 节点未寻到，错误退出
{
    return false;
}
LNode *pDel = NULL; // 删除节点
pDel = pTemp->next;
pTemp->next = pDel->next;
delete pDel;
m_listLength--;
return true;
}
```

//sort the linked list to descending order.

```
void LinkList::sort()
{
    if (m_listLength<=1)
    {
        return;
    }
    LNode *pTemp = m_pList;
    int temp;
```

```
// 选择法排序
for(int i=0;i<M_LISTLENGTH-1;i++)
for(int j=i+1;j<M_LISTLENGTH;j++)
if (pTemp[i].data<PTEMP[j].DATA)
{
temp=pTemp[i].data;
pTemp[i].data=pTemp[j].data;
pTemp[j].data=temp;
}
}
```

前两个函数实现了要求 a，后一个函数 sort()实现了要求 b

## 208. Debugging (Mandatory)

a. For each of the following recursive methods, enter Y in the answer box if the method terminates (assume i=5), Otherwise enter N.

(题目意思：判断下面的递归函数是否可以结束)

```
static int f(int i){
return f(i-1)*f(i-1);
}
```

**Ansewr:** N,明显没有返回条件语句，无限递归了

```
static int f(int i){
if(i==0){return 1;}
else {return f(i-1)*f(i-1);}
}
```

**Ansewr:**Y，当 i=0 时可结束递归

```
static int f(int i){
if(i==0){return 1;}
else {return f(i-1)*f(i-2);}
}
```

**Ansewr:**N，因为 i=1 时，f(i-2)=f(-1),进入一个无限递归中

## 209. 编程

将整数转换成字符串：void itoa(int,char);

例如 itoa(-123,s[])则 s="-123";

答：

```
char* itoa(int value, char* string)
{
char tmp[33];
char* tp = tmp;
int i;
unsigned v;
```



```

-----
return 0;
}
答: #define N 8
int main()
{
    int i;
    int j;
    int k;

    for(i=0;i<N;i++)
    {
        for(j=0;j<i+1;j++)
        {
            printf("***");
            for(k=0;k<i;k++)
                printf(".");
        }
        printf("\n");
    }
    return 0;
}

```

211. 下列程序运行时会崩溃，请找出错误并改正，并且说明原因。

```

#include "stdio.h"
#include "malloc.h"
typedef struct TNode
{
    TNode* left;
    TNode* right;
    int value;
}TNode;

TNode* root=NULL;
void append(int N);

int main()
{
    append(63);
    append(45);
    append(32);
    append(77);
    append(96);
}

```

```

append(21);
append(17); // Again, 数字任意给出
return 0;
}

void append(int N)
{
    TNode* NewNode=(TNode *)malloc(sizeof(TNode));
    NewNode->value=N;
    NewNode->left=NULL; //新增
    NewNode->right=NULL; //新增
    if(root==NULL)
    {
        root=NewNode;
        return;
    }
    else
    {
        TNode* temp;
        temp=root;
        while((N>=temp->value && temp->left!=NULL)|| (Nvalue && temp-
>right!=NULL))
        {
            while(N>=temp->value && temp->left!=NULL)
                temp=temp->left;
            while(Nvalue && temp->right!=NULL)
                temp=temp->right;
        }
        if(N>=temp->value)
            temp->left=NewNode;
        else
            temp->right=NewNode;
        return;
    }
}

```

答：因为新节点的左右指针没有赋 NULL 值，致使下面的 while 循环不能正确结束而导致内存越界，最后崩溃（注意结束条件是 temp->left!=NULL 或 temp->right!=NULL）。改正就是增加两条赋值语句，如上文红色部分字体就是新增的两条语句。

212.打印如下图案，共 19 行，只能有一个 for 循环（题目已经提供）

```
for(i=0;i<19;i++)
{
}
答:
#include "stdio.h"
void main()
{
for(int i=0;i<19;i++)
{
int j=0;
while (j<19)
{
if (i<=9)
{
if (j<=9)
{
if (i+j>=9)
printf("***");
else
printf(" ");
}
else
if (j-i<=9)
printf("***");
```

```

else
printf(" ");
}
else
{
if (j<=9)
{
if (i-j<=9)
printf("");
else
printf(" ");
}
else
if (j+i<=27)
printf("");
else
printf(" ");
}
j++;
}
printf("\n");
}
}

```

213.stack data （栈）存在于

A.rom, B .flash C .eeprom D.ram E .none of the above

答：D.ram。这题稍微涉及到一点硬件知识，ROM 的全称是 Read Only Memory，即只读存储器，flash，eeprom 都是 ROM 家族的一员，RAM 是 Random Access Memory 的简称，意为随机存取存储器，也就是内存了。不管是堆还是栈都是放在内存里的。

214.

```

int i;
int x=0x12345678;
unsigned char *p=(unsigned char *)&x;
for(i=0;i<SIZEOF(X);i++)
printf("%2x",*(p+i));

```

在 80x86pc 机器上运行结果？

答：x 在 PC 机上的内存存放顺序为 78 56 34 12，高字节在前，低字节在后，因此输出 78563412

Sun Sparc Unix 上运行结果？

215.

```

char a[2][2][3]={{{1,6,3},{5,4,15}},{{3,5,33},{23,12,7}}};
for(int i=0;i<12;i++)

```

```
printf("%d ",_____);
```

在空格处填上合适的语句，顺序打印出 **a** 中的数字

答:  $*(*(a+i/6)+(i/3\%2))+i\%3$

这题主要是要将输出的序号依次写出一些，如 000,001,002,010,011,012,100,101...然后找序号变化规律

216.请用标准 C 语言实现一个双向循环链表的查找与删除。

```
typedef struct doublecyclelink{
    int key;
    struct doublecyclelink *prev;
    struct doublecyclelink *next;
}DoubleCycleLinkT;
```

```
DoubleCycleLinkT *findKey(DoubleCycleLinkT *link,int key);
```

遍历整个双向循环链表，将第一个与 **key** 值相同的结点移出链表，并返回。

若没有找到则返回 **NULL**。

答:

函数为

```
DoubleCycleLinkT *findKey(DoubleCycleLinkT *link,int key)
{
    DoubleCycleLinkT *p;
    p=link->next;
    while (p->next!=link) // 链表结尾
    {
        if (p->key==key) // 查找到 key 值相同,删除该节点,并返回
        {
            p->prev->next=p->next;
            p->next->prev=p->prev;
            free(p);
            return link;
        }
        else
            p=p->next; // 否则查找下一节点
    }
    if (p->next == link) return NULL; //没找到，返回 NULL

}
```

217、请用标准 C 语言实现下列标准库函数，设计中不得使用其他库函数。

```
char *strstr(char *str1,char *str2);
```

在字符串 **str1** 中，寻找字串 **str2**，若找到返回找到的位置，否则返回 **NULL**。

答:

函数为

```

char * strstr ( const char * str1, const char * str2 )
{
    char *cp = (char *) str1;
    char *s1, *s2;
    if ( !*str2 )
        return((char *)str1);
    while (*cp)
    {
        s1 = cp;
        s2 = (char *) str2;
        while ( *s1 && *s2 && !(*s1-*s2) )
            s1++, s2++;
        if (!*s2)
            return(cp);
        cp++;
    }
    return(NULL);
}

```

218.实现双向链表删除一个节点 P，在节点 P 后插入一个节点，写出这两个函数；

答：

假设线性表的双向链表存储结构

```

typedef struct DuNode{
    struct DuNode *prior; //前驱指针
    ElemType data; //数据
    struct DuNode *next; //后继指针
}DuNode,*DuLinkedList;

```

删除操作

```

Status ListDelete_DuL(DuLinkedList &L,int i,ElemType &e)
{
    if(!(p=GetElemP_DuL(L,i)))
        return ERROR;
    e=p->data;
    p->prior->next=p->next;
    p->next->prior=p->prior;
    free(p);
    return OK;
}

```

插入操作

```

Status ListInsert_DuL(DuLinkedList &L,int i,ElemType &e)
{
    if(!(p=GetElemP_DuL(L,i)))
        return ERROR;
    if(!(s=(DuLinkedList)malloc(sizeof(DuNode))))

```

```
return ERROR;
```

```
s->data=e;
s->prior=p->prior;
p->prior->next=s;
s->next=p;
p->prior=s;
return OK;
}
```

219.写一个函数，将其中的\t 都转换成 4 个空格。

答：

该函数命名为 **convert**,参数的意义为：

**\*strDest** 目的字符串,**\*strSrc** 源字符串,**length** 源字符串的长度

函数实现为：

```
char* convert(char *strDest, const char *strSrc,int length)
{
    char * cp = strDest;
    int i=0;
    while(*strSrc && i
    {
        if (*strSrc=='\t') //将\t 转换成 4 个空格
        {
            for(int j=0;j<4;j++)
                *cp++=' ';
        }
        else //否则直接拷贝
            *cp++=*strSrc;
        strSrc++;
        i++;
    }
    return strDest;
}
```

230.Windows 程序的入口是哪里？ 写出 Windows 消息机制的流程。

答：

Windows 程序的入口是 **WinMain** 函数

消息机制：系统将会维护一个或多个消息队列，所有产生的消息都会被放入或是插入队列中。系统会在队列中取出每一条消息，根据消息的接收句柄而将该消息发送给拥有该窗口的程序的消息循环。每一个运行的程序都有自己的消息循环，在循环中得到属于自己的消息并根据接收窗口的句柄调用相应的窗口过程。而在没有消息时消息循环就将控制权交给系统。

231.如何定义和实现一个类的成员函数为回调函数？

答：

所谓的回调函数，就是预先在系统的对函数进行注册，让系统知道这个函数的存在，以后，当某个事件发生时，再调用这个函数对事件进行响应。

定义一个类的成员函数时在该函数前加 **CALLBACK** 即将其定义为回调函数，函数的实现和普通成员函数没有区别

**232.C++**里面是不是所有的动作都是 **main()**引起的？如果不是，请举例。

答：不是，比如中断引起的中断处理不是直接由 **main()**引起的，而是由外部事件引起的。

**233.C++**里面如何声明 **const void f(void)**函数为 C 程序中的库函数

答：在该函数前添加 **extern "C"**声明

**234.** 内联函数在编译时是否做参数类型检查

答：做类型检查，因为内联函数就是在程序编译时，编译器将程序中出现的内联函数的调用表达式用内联函数的函数体来代替。

**235.** 请你详细地解释一下 IP 协议的定义,在哪个层上面？主要有什么作用？TCP 与 UDP 呢？

答：IP 是 Internet Protocol 的简称，是网络层的主要协议，作用是提供不可靠、无连接的数据报传送。TCP 是 Transmit Control Protocol（传输控制协议）的缩写，在运输层，TCP 提供一种面向连接的，可靠的字节流服务；UDP 是 User Datagram Protocol（用户数据报协议）的缩写，在运输层，UDP 提供不可靠的传输数据服务

**236.** 请问交换机和路由器各自的实现原理是什么？分别在哪个层次上面实现的？

答：交换机属于 O S I 第二层即数据链路层设备。它根据 M A C 地址寻址，通过站表选择路由，站表的建立和维护由交换机自动进行。路由器属于 O S I 第三层即网络层设备，它根据 I P 地址进行寻址，通过路由表路由协议产生。交换机最大的好处是快速，路由器最大的好处是控制能力强。

**237.** 全局变量和局部变量有什么区别？是怎么实现的？操作系统和编译器是怎么知道的？

答：一些变量在整个程序中都是可见的，它们称为全局变量。一些变量只能在一个函数中可知，称为局部变量。这就是他们的区别。

在任何函数外面定义的变量就是全局变量，在函数内部定义的变量是局部变量，这是它们在程序中的实现过程。

操作系统和编译器是根据程序运行的内存区域知道他们的，程序的全局数据放在所分配内存的全局数据区，程序的局部数据放在栈区。

**238.** 有两个文件 **a.txt,b.txt**.**a.txt** 中存储的是 **aaaaaa**, **b.txt** 中存储的是 **bbb**. 将两个文件合并成 **c.txt** 如果是 **a** 并 **b** 的话存储为 **abababaaa**.要是 **b** 并 **a** 的话就是 **bababaaaa**.用 c 语言编程实现。

```
#include "stdio.h"

void fmerge(FILE *fa,FILE *fb,FILE *fc)
{
    char cha, chb;
    cha=fgetc(fa);
```

```

chb=fgetc(fb);
while ((cha!=EOF)&&(chb!=EOF))
{
    fputc(chb,fc);
    fputc(chb,fc);
    cha=fgetc(fa);
    chb=fgetc(fb);
}
while (cha!=EOF)
{
    fputc(chb,fc);
    cha=fgetc(fa);
}
while (chb!=EOF)
{
    fputc(chb,fc);
    chb=fgetc(fb);
}
}
int main()
{
    FILE *fa,*fb,*fc;
    fa=fopen("a.txt","r");
    fb=fopen("b.txt","r");
    fc=fopen("c.txt","w");
    fmerge(fa,fb,fc);
    fclose(fa);
    fclose(fb);
    fclose(fc);
    return 0;
}

```

239.C++:memset ,memcpy 和 strcpy 的根本区别?

#include "memory.h"

**memset** 用来对一段内存空间全部设置为某个字符,一般用在对定义的字符串进行初始化为 ' '或'\0'; 例:char a[100];memset(a, '\0', sizeof(a));

**memcpy** 用来做内存拷贝,你可以拿它拷贝任何数据类型的对象,可以指定拷贝的数据长度; 例: char a[100],b[50]; memcpy(b, a, sizeof(b));注意如用 sizeof(a),会造成 b 的内存地址溢出。

**strcpy** 就只能拷贝字符串了,它遇到'\0'就结束拷贝; 例: char a[100],b[50];strcpy(a,b);如用 strcpy(b,a),要注意 a 中的字符串长度(第一个'\0'之前)是否超过 50 位,如超过,则会造成 b 的内存地址溢出。

### strcpy

原型: `extern char *strcpy(char *dest, char *src);`

用法: `#include`

功能: 把 `src` 所指由 `NULL` 结束的字符串复制到 `dest` 所指的数组中。

说明: `src` 和 `dest` 所指内存区域不可以重叠且 `dest` 必须有足够的空间来容纳 `src` 的字符串。

返回指向 `dest` 的指针。

### memcpy

原型: `extern void *memcpy(void *dest, void *src, unsigned int count);`

用法: `#include`

功能: 由 `src` 所指内存区域复制 `count` 个字节到 `dest` 所指内存区域。

说明: `src` 和 `dest` 所指内存区域不能重叠, 函数返回指向 `dest` 的指针。

### memset

原型: `extern void *memset(void *buffer, char c, int count);`

用法: `#include`

功能: 把 `buffer` 所指内存区域的前 `count` 个字节设置成字符 `c`。

说明: 返回指向 `buffer` 的指针。

## 240.ASSERT()是干什么用的

`ASSERT()` 是一个调试程序时经常使用的宏, 在程序运行时它计算括号内的表达式, 如果表达式为 `FALSE (0)`, 程序将报告错误, 并终止执行。如果表达式不为 `0`, 则继续执行后面的语句。这个宏通常原来判断程序中是否出现了明显非法的数据, 如果出现了终止程序以免导致严重后果, 同时也便于查找错误。例如, 变量 `n` 在程序中不应该为 `0`, 如果为 `0` 可能导致错误, 你可以这样写程序:

.....

```
ASSERT( n != 0);
```

```
k = 10/ n;
```

.....

`ASSERT` 只有在 `Debug` 版本中才有效, 如果编译为 `Release` 版本则被忽略。

`assert()` 的功能类似, 它是 `ANSI C` 标准中规定的函数, 它与 `ASSERT` 的一个重要区别是可以用在 `Release` 版本中。

## 241. 二分查找算法:

1、递归方法实现:

```
int BSearch(elemtype a[],elemtype x,int low,int high)
```

```
/*在下届为 low, 上界为 high 的数组 a 中折半查找数据元素 x*/
```

```
{
```

```
int mid;
```

```
if(low>high) return -1;
```

```
mid=(low+high)/2;
```

```
if(x==a[mid]) return mid;
```

```

if(x<a[mid]) return(BSearch(a,x,low,mid-1));
else return(BSearch(a,x,mid+1,high));
}

```

2、非递归方法实现：

```

int BSearch(elemtype a[],keytype key,int n)
{
    int low,high,mid;
    low=0;high=n-1;
    while(low<=high)
    {
        mid=(low+high)/2;
        if(a[mid].key==key) return mid;
        else if(a[mid].key<key) low=mid+1;
        else high=mid-1;
    }
    return -1;
}

```

242，写出下面代码段的输出结果，并说明理由：

```
char str1[] = "abc";
```

```
char str2[] = "abc";
```

```
const char str3[] = "abc";
```

```
const char str4[] = "abc";
```

```
const char *str5 = "abc";
```

```
const char *str6 = "abc";
```

```
char *str7 = "abc";
```

```
char *str8 = "abc";
```

```
cout << ( str1 == str2 ) << endl;
```

```
cout << ( str3 == str4 ) << endl;
```

```
cout << ( str5 == str6 ) << endl;
```

```
cout << ( str7 == str8 ) << endl;
```

- 1, `str1, str2, str3, str4` 是数组变量, 它们有各自的内存空间;  
而 `str5, str6, str7, str8` 是指针, 它们指向相同的常量区域。

243. 以下代码中的两个 `sizeof` 用法有问题吗?

```
void UpperCase( char str[] )  
  
{  
    for( size_t i=0; i<sizeof(str)/sizeof(str[0]); ++i )  
        if( 'a'<=str[i] && str[i]<='z' )  
            str[i] -= ('a'-'A' );  
}  
  
char str[] = "aBcDe";  
cout << "str 字符长度为: " << sizeof(str)/sizeof(str[0]) << endl;  
UpperCase( str );  
cout << str << endl;
```

函数内的 `sizeof` 有问题。根据语法, `sizeof` 如用于数组, 只能测出静态数组的大小, 无法检测动态分配的或外部数组大小。函数外的 `str` 是一个静态定义的数组, 因此其大小为 6, 函数内的 `str` 实际只是一个指向字符串的指针, 没有任何额外的与数组相关的信息, 因此 `sizeof` 作用于上只将其当指针看,

244, 下面程序输出结果是多少:

```
main()  
{  
    int a[5]={1,2,3,4,5};  
    int *ptr=(int *)(&a+1);  
  
    printf("%d,%d", *(a+1), *(ptr-1));  
}
```

```
}
```

2,5

\*(a+1) 就是 a[1], \*(ptr-1)就是 a[4],执行结果是 2, 5

&a+1 不是首地址+1, 系统会认为加一个 a 数组的偏移, 是偏移了一个数组的大小 (本例是 5 个 int)

```
int *ptr=(int *)(&a+1);
```

则 ptr 实际是&(a[5]),也就是 a+5

原因如下:

&a 是数组指针, 其类型为 int (\*)[5];

而指针加 1 要根据指针类型加上一定的值,

不同类型的指针+1 之后增加的大小不同

a 是长度为 5 的 int 数组指针, 所以要加 5\*sizeof(int)

所以 ptr 实际是 a[5]

但是 prt 与(&a+1)类型是不一样的(这点很重要)

所以 prt-1 只会减去 sizeof(int\*)

a,&a 的地址是一样的, 但意思不一样, a 是数组首地址, 也就是 a[0]的地址, &a 是对象 (数组) 首地址, a+1 是数组下一元素的地址, 即 a[1],&a+1 是下一个对象的地址, 即 a[5].

245, 请问运行 Test 函数会有什么样的结果?

```
void GetMemory(char *p)
```

```
{
```

```
p = (char *)malloc(100);
```

```
}
```

```
void Test(void)
```

```
{
```

```
char *str = NULL;
```

```
GetMemory(str);
```

```
strcpy(str, "hello world");
```

```
printf(str);
```

```
}
```

, 请问运行 Test 函数会有什么样的结果?

```
char *GetMemory(void)
```

```
{
```

```
char p[] = "hello world";
```

```
return p;
```

```
}
```

```
void Test(void)
```

```
{
```

```
char *str = NULL;
```

```
str = GetMemory();
```

```
printf(str);
```

```
}
```

，请问运行 Test 函数会有什么样的结果？

```
Void GetMemory2(char **p, int num)
{
    *p = (char *)malloc(num);
}
void Test(void)
{
    char *str = NULL;
    GetMemory(&str, 100);
    strcpy(str, "hello");
    printf(str);
}
```

，请问运行 Test 函数会有什么样的结果？

```
void Test(void)
{
    char *str = (char *) malloc(100);
    strcpy(str, "hello");
    free(str);
    if(str != NULL)
    {
        strcpy(str, "world");
        printf(str);
    }
}
```

几种排序：

### 1.稳定性比较

插入排序、冒泡排序、二叉树排序、二路归并排序及其他线形排序是稳定的

选择排序、希尔排序、快速排序、堆排序是不稳定的

### 2.时间复杂性比较

插入排序、冒泡排序、选择排序的时间复杂性为  $O(n^2)$

其它非线形排序的时间复杂性为  $O(n\log_2 n)$

线形排序的时间复杂性为  $O(n)$ ;

### 3.辅助空间的比较

线形排序、二路归并排序的辅助空间为  $O(n)$ ,其它排序的辅助空间为  $O(1)$ ;

### 4.其它比较

插入、冒泡排序的速度较慢,但参加排序的序列局部或整体有序时,这种排序能达到较快的速度。

反而在这种情况下,快速排序反而慢了。

当  $n$  较小时,对稳定性不作要求时宜用选择排序,对稳定性有要求时宜用插入或冒泡排序。

若待排序的记录的关键字在一个明显有限范围内时,且空间允许是用桶排序。

当  $n$  较大时,关键字元素比较随机,对稳定性没要求宜用快速排序。

当  $n$  较大时,关键字元素可能出现本身是有序的,对稳定性有要求时,空间允许的情况下。

宜用归并排序。

当  $n$  较大时,关键字元素可能出现本身是有序的,对稳定性没有要求时宜用堆排序。

\*\*\*\*\*

## 重温经典排序思想--C 语言常用排序全解

/\*

=====

=====

相关知识介绍(所有定义只为帮助读者理解相关概念,并非严格定义):

### 1、稳定排序和非稳定排序

简单地讲就是所有相等的数经过某种排序方法后,仍能保持它们在排序之前的相对次序,我们就

说这种排序方法是稳定的。反之,就是非稳定的。

比如:一组数排序前是  $a_1, a_2, a_3, a_4, a_5$ , 其中  $a_2 = a_4$ , 经过某种排序后为  $a_1, a_2, a_4, a_3, a_5$ , 则我们说这种排序是稳定的,因为  $a_2$  排序前在  $a_4$  的前面,排序后它还是在  $a_4$  的前面。假如变成  $a_1, a_4$ ,

$a_2, a_3, a_5$  就不是稳定的了。

### 2、内排序和外排序

在排序过程中，所有需要排序的数都在内存，并在内存中调整它们的存储顺序，称为内排序；在排序过程中，只有部分数被调入内存，并借助内存调整数在外存中的存放顺序排序方法称为外排序。

### 3、算法的时间复杂度和空间复杂度

所谓算法的时间复杂度，是指执行算法所需要的计算工作量。

一个算法的空间复杂度，一般是指执行这个算法所需要的内存空间。

```
=====
=====
```

```
*/
```

```
/*
```

```
=====
```

功能：选择排序

输入：数组名称（也就是数组首地址）、数组中元素个数

```
=====
```

```
*/
```

```
/*
```

```
=====
```

算法思想简单描述：

在要排序的一组数中，选出最小的一个数与第一个位置的数交换；然后在剩下的数当中再找最小的与第二个位置的数交换，如此循环到倒数第二个数和最后一个数比较为止。

选择排序是不稳定的。算法复杂度  $O(n^2)$ —[n 的平方]

```
=====
```

```
*/
```

```
void select_sort(int *x, int n)
```

```
{
```

```
int i, j, min, t;
```

```
for (i=0; i<n-1; i++) /*要选择的次数： 0~n-2 共 n-1 次*/
```

```
{
```

```
    min = i; /*假设当前下标为 i 的数最小，比较后再调整*/
```

```
    for (j=i+1; j<n; j++) /*循环找出最小的数的下标是哪个*/
```

```
    {
```

```
        if (*(x+j) < *(x+min))
```

```
        {
```

```
            min = j; /*如果后面的数比前面的小，则记下它的下标*/
```

```
        }
```

```
    }
```

```
}
```

```

        if (min != i) /*如果 min 在循环中改变了，就需要交换数据*/
        {
            t = *(x+i);
            *(x+i) = *(x+min);
            *(x+min) = t;
        }
    }
}

```

```

/*
=====
功能：直接插入排序
输入：数组名称（也就是数组首地址）、数组中元素个数
=====
*/
/*

```

=====

算法思想简单描述：

在要排序的一组数中，假设前面(n-1) [n>=2] 个数已经是排好顺序的，现在要把第 n 个数插到前面的有序数中，使得这 n 个数也是排好顺序的。如此反复循环，直到全部排好顺序。

直接插入排序是稳定的。算法时间复杂度  $O(n^2)$ --[n 的平方]

```

=====
*/
void insert_sort(int *x, int n)
{
    int i, j, t;

    for (i=1; i<n; i++) /*要选择的次数： 1~n-1 共 n-1 次*/
    {
        /*
            暂存下标为 i 的数。注意：下标从 1 开始，原因就是开始时
            第一个数即下标为 0 的数，前面没有任何数，单一个，认为
            它是排好顺序的。
        */
        t=*(x+i);
        for (j=i-1; j>=0 && t<*(x+j); j--) /*注意： j=i-1, j--, 这里就是下标为 i 的数，在它前面有
            序列中找插入位置。*/
        {
            *(x+j+1) = *(x+j); /*如果满足条件就往后挪。最坏的情况就是 t 比下标为 0 的数都小，
            它要放在最前面， j=-1, 退出循环*/
        }
    }
}

```

```

    }

    *(x+j+1) = t; /*找到下标为 i 的数的放置位置*/
}
}

/*
=====
功能：冒泡排序
输入：数组名称（也就是数组首地址）、数组中元素个数
=====
*/
/*
=====
算法思想简单描述：

```

在要排序的一组数中，对当前还未排好序的范围内的全部数，自上而下对相邻的两个数依次进行比较和调整，让较大的数往下沉，较小的往上冒。即：每当两相邻的数比较后发现它们的排序与排序要求相反时，就将它们互换。

下面是一种改进的冒泡算法，它记录了每一遍扫描后最后下沉数的位置 **k**，这样可以减少外层循环扫描的次数。

冒泡排序是稳定的。算法时间复杂度  $O(n^2)$ --[n 的平方]

```

=====
*/

void bubble_sort(int *x, int n)
{
    int j, k, h, t;

    for (h=n-1; h>0; h=k) /*循环到没有比较范围*/
    {
        for (j=0, k=0; j<h; j++) /*每次预置 k=0，循环扫描后更新 k*/
        {
            if (*(x+j) > *(x+j+1)) /*大的放在后面，小的放到前面*/
            {
                t = *(x+j);
                *(x+j) = *(x+j+1);
                *(x+j+1) = t; /*完成交换*/
                k = j; /*保存最后下沉的位置。这样 k 后面的都是排序排好了的。*/
            }
        }
    }
}

```

```

    }
}
}

```

```

/*

```

```

=====
功能：希尔排序

```

```

输入：数组名称（也就是数组首地址）、数组中元素个数

```

```

*/

```

```

/*

```

```

=====
算法思想简单描述：

```

在直接插入排序算法中，每次插入一个数，使有序序列只增加 1 个节点，并且对插入下一个数没有提供任何帮助。如果比较相隔较远距离（称为增量）的数，使得数移动时能跨过多个元素，则进行一次比较就可能消除多个元素交换。D.L.shell 于 1959 年在以他名字命名的排序算法中实现了这一思想。算法先将要排序的一组数按某个增量  $d$  分成若干组，每组中记录的下标相差  $d$ 。对每组中全部元素进行排序，然后再用一个较小的增量对它进行，在每组中再进行排序。当增量减到 1 时，整个要排序的数被分成一组，排序完成。

下面的函数是一个希尔排序算法的一个实现，初次取序列的一半为增量，以后每次减半，直到增量为 1。

希尔排序是不稳定的。

```

=====
*/

```

```

void shell_sort(int *x, int n)

```

```

{

```

```

    int h, j, k, t;

```

```

    for (h=n/2; h>0; h=h/2) /*控制增量*/

```

```

    {

```

```

        for (j=h; j<n; j++) /*这个实际上就是上面的直接插入排序*/

```

```

        {

```

```

            t = *(x+j);

```

```

            for (k=j-h; (k>=0 && t<*(x+k)); k-=h)

```

```

            {

```

```

                *(x+k+h) = *(x+k);

```

```

            }

```

```

            *(x+k+h) = t;

```

```

    }
}
}

```

```

/*

```

```

=====
功能：快速排序

```

```

输入：数组名称（也就是数组首地址）、数组中起止元素的下标

```

```

*/

```

```

/*

```

```

=====
算法思想简单描述：

```

快速排序是对冒泡排序的一种本质改进。它的基本思想是通过一趟扫描后，使得排序序列的长度能大幅度地减少。在冒泡排序中，一次扫描只能确保最大数值的数移到正确位置，而待排序序列的长度可能只减少 1。快速排序通过一趟扫描，就能确保某个数（以它为基准点吧）的左边各数都比它小，右边各数都比它大。然后又用同样的方法处理它左右两边的数，直到基准点的左右只有一个元素为止。它是由 C.A.R.Hoare 于 1962 年提出的。

显然快速排序可以用递归实现，当然也可以用栈化解递归实现。下面的函数是用递归实现的，有兴趣的朋友可以改成非递归的。

快速排序是不稳定的。最理想情况算法时间复杂度  $O(n\log_2 n)$ ，最坏  $O(n^2)$

```

=====
*/

```

```

void quick_sort(int *x, int low, int high)

```

```

{

```

```

    int i, j, t;

```

```

    if (low < high) /*要排序的元素起止下标，保证小的放在左边，大的放在右边。这里以下标为
low 的元素为基准点*/

```

```

    {

```

```

        i = low;

```

```

        j = high;

```

```

        t = *(x+low); /*暂存基准点的数*/

```

```

        while (i<j) /*循环扫描*/

```

```

        {

```

```

            while (i<j && *(x+j)>t) /*在右边的只要比基准点大仍放在右边*/

```

```

{
    j--; /*前移一个位置*/
}

if (i<j)
{
    *(x+i) = *(x+j); /*上面的循环退出：即出现比基准点小的数，替换基准点的数*/
    i++; /*后移一个位置，并以此为基准点*/
}

while (i<j && *(x+i)<=t) /*在左边的只要小于等于基准点仍放在左边*/
{
    i++; /*后移一个位置*/
}

if (i<j)
{
    *(x+j) = *(x+i); /*上面的循环退出：即出现比基准点大的数，放到右边*/
    j--; /*前移一个位置*/
}
}

*(x+i) = t; /*一遍扫描完后，放到适当位置*/
quick_sort(x,low,i-1); /*对基准点左边的数再执行快速排序*/
quick_sort(x,i+1,high); /*对基准点右边的数再执行快速排序*/
}
}

```

```

/*
=====
功能：堆排序
输入：数组名称（也就是数组首地址）、数组中元素个数
=====
*/
/*
=====
算法思想简单描述：

```

堆排序是一种树形选择排序，是对直接选择排序的有效改进。

堆的定义如下：具有  $n$  个元素的序列  $(h_1, h_2, \dots, h_n)$ ，当且仅当满足  $(h_i \geq h_{2i}, h_i \geq h_{2i+1})$  或  $(h_i \leq h_{2i}, h_i \leq h_{2i+1})$  ( $i=1, 2, \dots, n/2$ ) 时称之为堆。在这里只讨论满足前者条件的堆。

由堆的定义可以看出，堆顶元素（即第一个元素）必为最大项。完全二叉树可以很直观地表示堆的结构。堆顶为根，其它为左子树、右子树。

初始时把要排序的数的序列看作是一棵顺序存储的二叉树，调整它们的存储顺序，使之成为一个堆，这时堆的根节点的数最大。然后将根节点与堆的最后一个节点交换。然后对前面(n-1)个数重新调整使之成为堆。

依此类推，直到只有两个节点

的堆，并对它们作交换，最后得到有 n 个节点的有序序列。

从算法描述来看，堆排序需要两个过程，一是建立堆，二是堆顶与堆的最后一个元素交换位置。所以堆排序有两个函数组成。一是建堆的渗透函数，二是反复调用渗透函数实现排序的函数。

堆排序是不稳定的。算法时间复杂度  $O(n\log_2 n)$ 。

```
*/
/*
功能：渗透建堆
输入：数组名称（也就是数组首地址）、参与建堆元素的个数、从第几个元素开始
*/
void sift(int *x, int n, int s)
{
    int t, k, j;

    t = *(x+s); /*暂存开始元素*/
    k = s;      /*开始元素下标*/
    j = 2*k + 1; /*右子树元素下标*/

    while (j<n)
    {
        if (j<n-1 && *(x+j) < *(x+j+1)) /*判断是否满足堆的条件：满足就继续下一轮比较，否则调整。*/
        {
            j++;
        }

        if (t<*(x+j)) /*调整*/
        {
            *(x+k) = *(x+j);
            k = j; /*调整后，开始元素也随之调整*/
            j = 2*k + 1;
        }
        else /*不需要调整了，已经是个堆了，退出循环。*/
        {
            break;
        }
    }
}
```

```
    }  
}
```

```
*(x+k) = t; /*开始元素放到它正确位置*/  
}
```

```
/*
```

功能：堆排序

输入：数组名称（也就是数组首地址）、数组中元素个数

```
*/
```

```
void heap_sort(int *x, int n)
```

```
{
```

```
int i, k, t;
```

```
int *p;
```

```
for (i=n/2-1; i>=0; i--)
```

```
{
```

```
    sift(x,n,i); /*初始建堆*/
```

```
}
```

```
for (k=n-1; k>=1; k--)
```

```
{
```

```
    t = *(x+0); /*堆顶放到最后*/
```

```
    *(x+0) = *(x+k);
```

```
    *(x+k) = t;
```

```
    sift(x,k,0); /*剩下的数再建堆*/
```

```
}
```

```
}
```